AXC3000 MIPI CSI-2

Software Requirements: Quartus® Prime Pro version 25.1.1

Hardware Requirements: AXC3000 Evaluation Kit with USB-C cable,
amsOSRAM MIRA220_RGB_mini_SB Camera kit,
Trenz CRUVI TEC0278 adapter

Document Control

Document Version:	Version 1.0
Document Date:	10/20/2025
Document Author(s):	Steven Kravatsky, Erika Peter, Naji Naufel, ESC Team
Document Classification:	Released
Document Distribution:	This document is still under development. All specifications, procedures, and processes described in this document are subject to change without prior notice
Prior Version History:	0.0

Please read the legal disclaimer at the end of this document.

Table of Contents

1 Introduction

This project demonstrates the MIPI-CSI2 video path from a camera through a frame buffer located in the on-chip SRAM. It uses the MIPI D-PHY and on-chip SRAM hard IPs, MIPI-CSI2 soft IP, and a few elements of the Altera® Video and Vision IP suite.

1.1 Hardware and Software needs

The target hardware is the Arrow AXC3000 Evaluation kit.

For the video input path, an ams OSRAM Mira220 RGB camera with a 15-pin flat ribbon cable is used as the imager. The Flat Ribbon cable goes through an adapter (Trenz TEC0278-01) which plugs into the CRUVI-HS interface of the AXC3000 board. Video data goes through a few IP blocks before reaching the video frame buffer located in on-chip M20K SRAM memory.

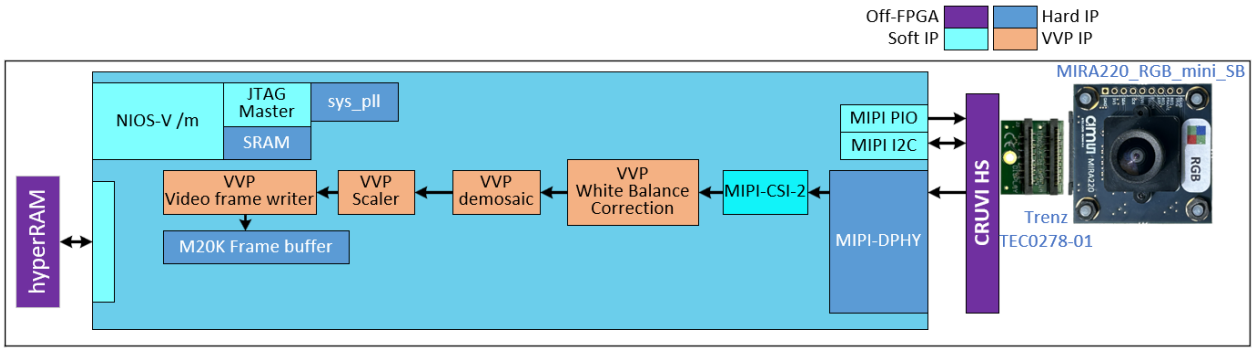


Figure 1-1 : Hardware Project Block Diagram

Lab Notes: Many of the names that the lab asks you to choose for files, components, and other objects in this exercise must be spelled exactly as directed. This nomenclature is necessary because the pre-written software application includes variables that use the names of the hardware peripherals. Naming the components differently can cause errors.

2 What is MIPI

“MIPI” stands for Mobile Industry Processor Interface, and it's a global standards body (MIPI Alliance) and a set of interface specifications.

It provides robust, interoperable hardware and protocol interfaces between components like processors, FPGAs, image sensors, and displays.

Popular specifications include CSI-2 (camera), DSI-2 (display), I3C (sensor and control bus), and more.

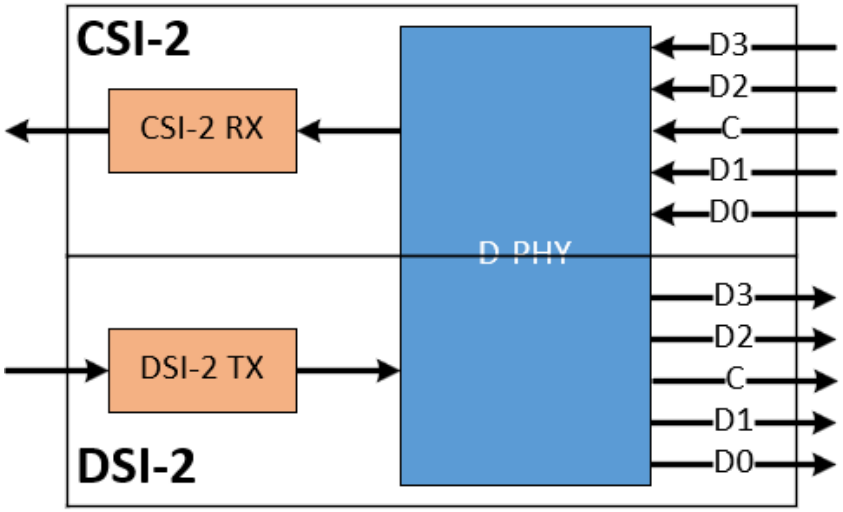


Figure 2-1 : MIPI Camera and Display components

2.1 CSI-2

MIPI CSI-2 is a high-speed, lane-scalable serial link interface used to transfer image and video data from camera (image sensors) to a host processor, or FPGA, in embedded systems.

It is implemented over a D-PHY physical layer for short-reach applications.

The lane scalability supports 1, 2, or 4 data lanes plus a clock lane. These lanes use differential signaling with data rates up to 3.5 Gbps per lane (Agilex-5 E-Series capability).

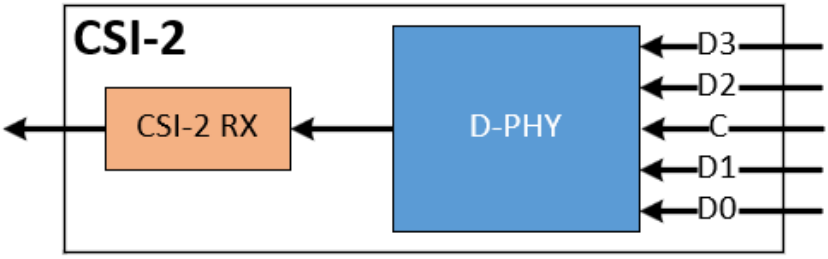


Figure 2-2 : MIPI CSI-2 Path

2.1.1 D-PHY

The D-PHY is the Physical-Layer interface developed by the MIPI Alliance to carry high-speed and low-power data streams for camera (CSI-2) and display (DSI-2) applications. In CSI-2 applications, it:

- Uses low-swing, high-speed differential pair input signaling.
- Deserializes incoming data streams back into parallel data for processing.
- Includes mechanisms for clock recovery and data alignment.
- Delivers 8-bit or 16-bit parallel data to the CSI-2 RX module.
- In High-Speed (HS) mode, it enables rapid data transfer.
- In Low Power (LP) mode, it facilitates control signaling and low-speed data communication, operating at lower voltages and conserving power.
- It supports scalable lane configurations from 1 to 4 lanes, allowing flexibility in balancing data rate and power consumption based on application requirements.

2.1.2 CSI-2 RX

The CSI-2 RX (Receiver) decodes incoming images and video data from a camera module over a D-PHY and delivers a formatted pixel stream to the downstream system interface.

- Operates as the bridge between the high-speed MIPI physical layer and the host processor/image signal processor/FPGA.
- Performs lane de-skew and word alignment, synchronizing all active lanes to a common byte clock domain.
- Generates Start-of-Packet (SOP) and End-of-Packet (EOP) markers to mark data boundaries.
- Merges bytes from multiple data lanes and consolidates them into complete CSI-2 packets
- Unpacks video payloads (e.g., RAW10/12/14, RGB, YUV) based on packet format.
- Transfers data across clock domains via FIFO, producing pixel-stream data.
- Supports formats like 1–8 pixels per clock, buffering modes (line buffer, fill-level, low-latency), and up to 8 independent stream outputs.

2.2 DSI-2

MIPI DSI-2 is a high-speed serial link used for display panels. It is comprised of 2 components, the Transport Layer (CSI-2 Transmitter), and Physical Layer (PHY).

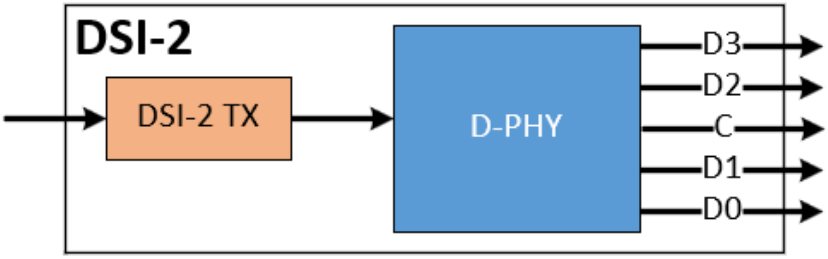


Figure 2-3 : MIPI DSI-2 Path

2.2.1 DSI-2 TX

A DSI-2 TX (Transmitter) takes incoming pixel data streams (e.g. RAW, YUV, RGB) and packages them into compliant DSI-2 protocol packets.

- Handles packet formatting, virtual channel multiplexing, error control, and interfaces with the physical layer (typically D-PHY) via PPI (PHY-Protocol Interface).
- Receives video pixel streams from 1–4 pixel ports.
- Converts packet bytes to D-PHY byte interface data (PPI).
- Controls the D-PHY lane states: transitions between Low-Power (LP), High-Speed (HS), and ULPS/Idle.
- Packs image data (e.g., RAW10/12/14, RGB, YUV) into DSI-2 standardized packets (short/long).
- Formats data with proper word lengths, headers, and trailer control fields.

2.2.2 D-PHY

The D-PHY is the Physical-Layer interface developed by the MIPI Alliance to carry high-speed and low-power data streams for camera (CSI-2) and display (DSI-2) applications. In MIPI applications, it:

- Uses low-swing, high-speed differential pair output signaling.
- Serializes parallel data from the DSI-2 TX to outgoing data.
- Receives 8-bit or 16-bit parallel data from the DSI-2 TX module.
- In High-Speed (HS) mode, it enables rapid data transfer.
- In Low Power (LP) mode, it facilitates control signaling and low-speed data communication, operating at lower voltages and conserving power.
- It supports scalable lane configurations from 1 to 4 lanes, allowing flexibility in balancing data rate and power consumption based on application requirements.

2.3 MIPI CSI-2 Input Path for this Project

In addition to the D-PHY and CSI-2 RX components, there are other IP blocks needed for transporting the image/video data to the frame buffer.

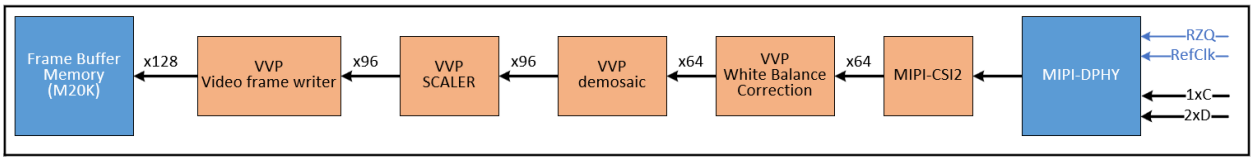


Figure 2-4 : MIPI CSI-2 Input Path

The ***MIPI D-PHY*** uses a 1 clock lane and 2 data lane interface to the camera operating at 1.5Gbps, which gets transformed into 16-bit parallel pixel data output running at 93.75MHz.

The ***MIPI-CSI2 RX*** IP module take the data from the D-PHY, buffers it and outputs 4 pixels in parallel in RAW12 format.

The ***White Balance Correction*** IP scales individual color channels of a 2x2 color filter array to correct color distortion.

The ***Demosaic*** IP converts the data from a 2x2 Bayer filter array video stream into an RGB video stream.

Since the camera initialization is being done for a 1280x720 frame size, the ***Scaler*** IP is used to convert the horizontal resolution from 1280 to 320 and the vertical resolution from 720 to 200.

The ***video frame writer*** is the last IP in the video input path. It writes video data in 128-bit width into 1 frame buffer located in the on-chip SRAM. The 320x200 24-bit color frame needs 1,536,000 bits (192,000 bytes).

3 Using the ams OSRAM Mira220 RGB camera

Chapter 3 is intended as informational. It describes the derivation of the camera settings using the ams OSRAM Sensor Configuration Tool. For the Workshop, the settings are provided for you within the C-code that runs on the Nios-V CPU. Going through the steps in this chapter is optional.

This project uses the ams OSRAM Mira220 2.2MP RGB image sensor.

Camera features:

- Global Shutter
- 1600 x 1400 effective resolution with selectable Region of Interest (ROI)
- Selectable bit depth of 8/10/12-bit
- Maximum frame rate of 90fps

For this Workshop, we will use a MIRA220_DEM_KT_RGB (Q6511A1615) which includes the camera with a lens on PCB, 15-pin flat ribbon cable, a tripod mounting bracket, and a tripod.

The NIOS-V C program used in this project already has the camera setup parameters loaded in, but if you choose, you can learn how they were derived using the ***ams OSRAM Sensor Configuration Tool v1.4.0***.

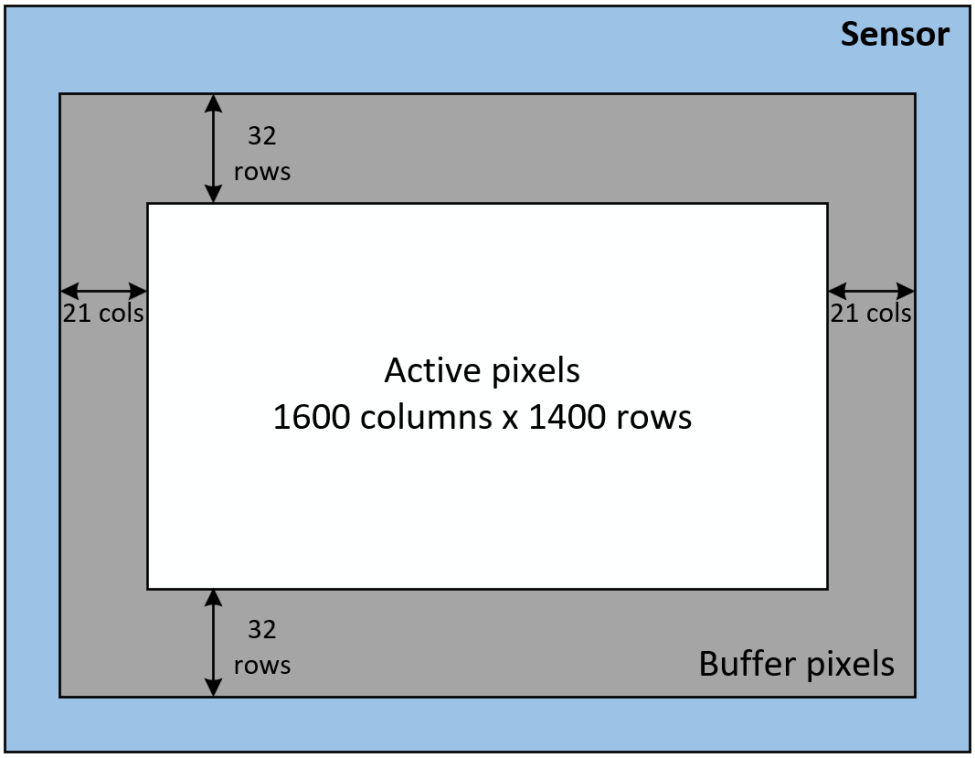


Figure 3-1 : Image Sensor format

The Mira 220 can do windowing by using the Region of Interest (ROI) as seen in the figure below. We will use this feature to set the camera to capture 1280x720 (16:9) images.

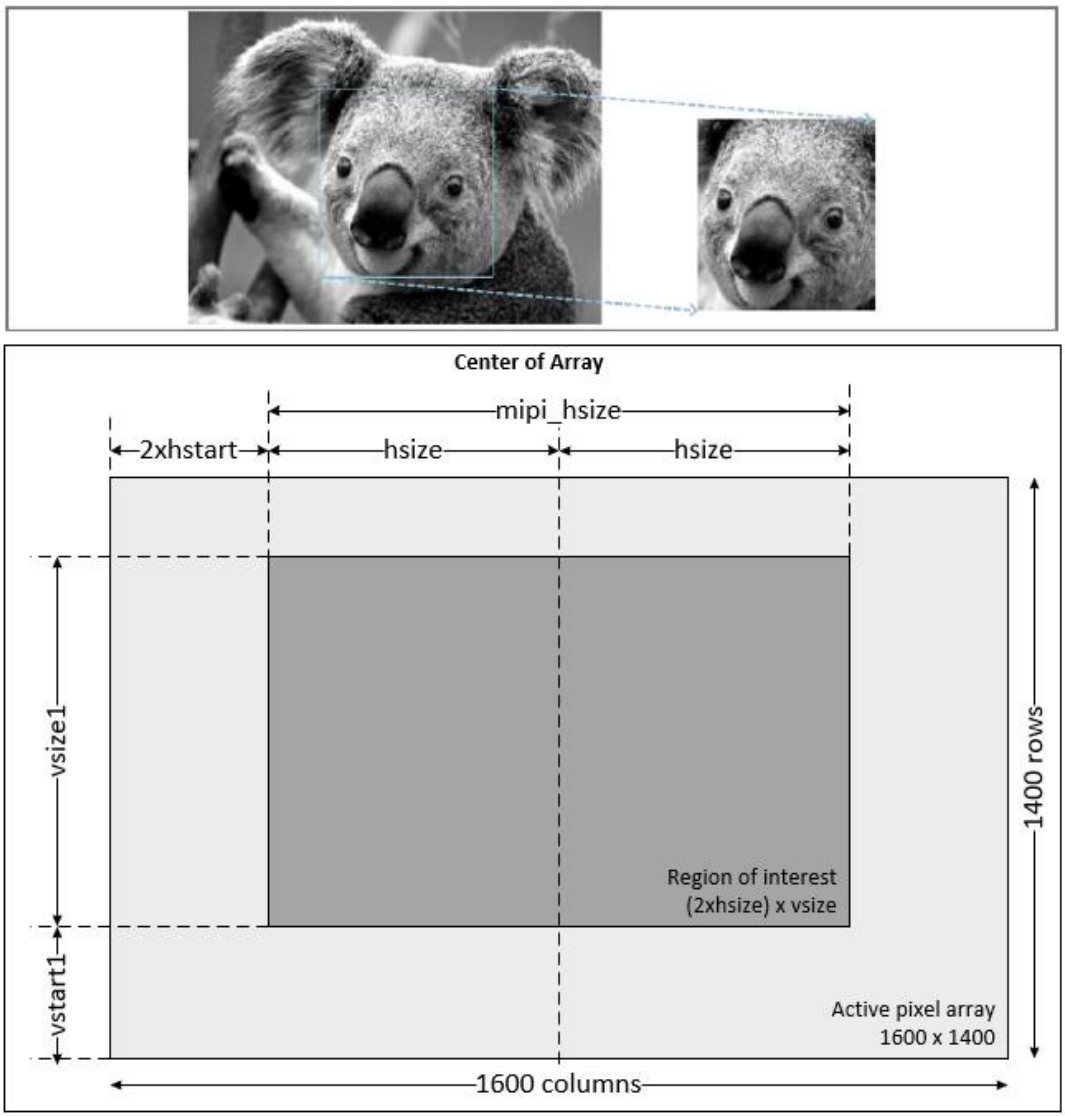


Figure 3-2 : ROI Windowing

3.1 Using the Camera Configuration Tool

At the time of this document creation, the tool was in v1.4.0.

Open the ams OSRAM Sensor Configuration Tool

On the startup screen **click *New*** on the menu bar and select **Mira220**. This action loads the parameter set up screens for the Mira 220 image sensor.

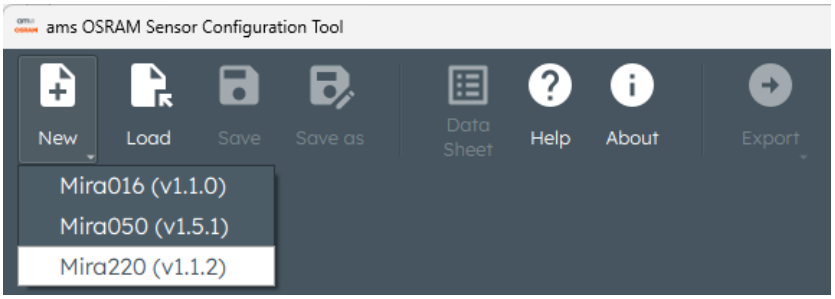


Figure 3-3 : Select Mira 220 image sensor

The configuration tool has many screens, but most of them will remain untouched (defaults).

3.1.1 Horizontal ROI Screen

On this screen, we need to achieve 1280 columns. All values need to add up to 1600.

As you saw in Figure 3-2, 1600 = 2xHSTART (Front Porch) + 2xHSIZE + 2xHSTART (Back Porch).

So, we enter **640** (1280/2) for ***HSIZE***, and **80** for ***HSTART***.

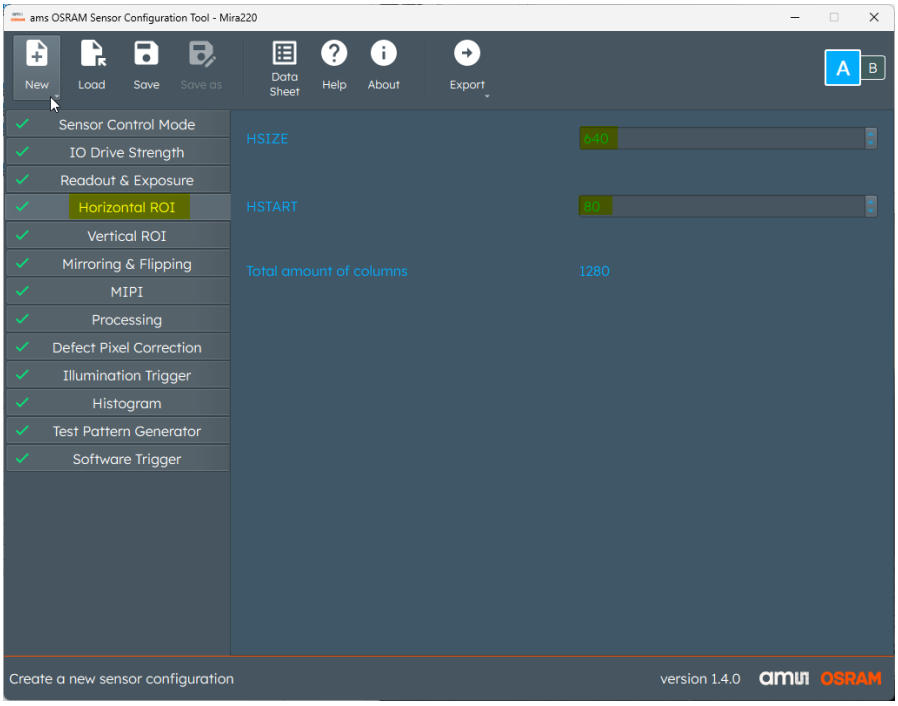


Figure 3-4 : Horizontal ROI Screen Settings

3.1.2 Vertical ROI Screen

On this screen, we need to achieve 720 rows. All values need to add up to 1400.

As you saw in Figure 3-2, ***1400 = VSIZE + 2xVSTART (Vertical Blanking)***.

So, we enter **340** for ***VSTART*** and **720** for ***VSIZE***.

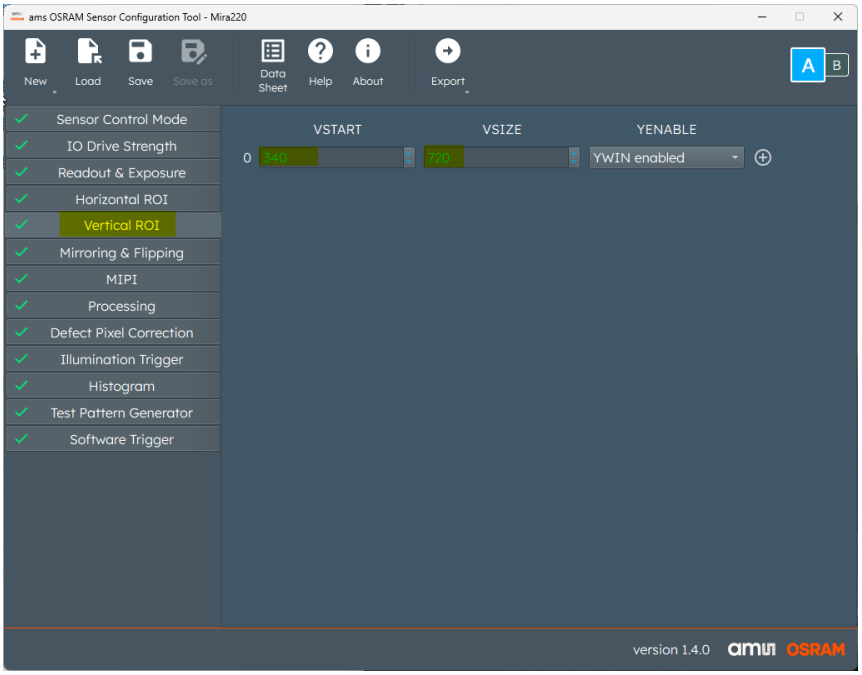


Figure 3-5 : Vertical ROI Screen Settings

3.1.3 MIPI Screen

This screen comes up with the default 1500Mbps, 12-bit data format, and 2-lanes. Change the ***Data Rate [Mbps]*** to **1200**. For the lower resolution that we are using, we don't need to run the data rate at its maximum 1500 Mbps, instead we are slowing it down a bit, to 1200 Mbps. This causes the "B" page (alternate settings) to have errors, which can be ignored. If you want to fix the "B" page errors, click on "B" page and update the failing values to match those in page "A".

Scroll down a bit and ensure that the ***Data Enable, HSYNC, VSYNC Polarity*** are set to **Active Low**. Set the ***CLK_MODE*** to **Continuous PHY clocking**. Set ***Line Count RAW8*** to **Disabled**, Set ***Line Count RAW10*** to **Disabled**. Scroll down and ensure that ***INIT_SKEW_EN*** is set to **Enabled**.

3.1.4 Readout & Exposure Screen

Keep the Input clock period at 26ns, since this is what is driving the sensor on the Mira 220 dev kit.

Set ***VBLANK*** (vertical blanking interval) to **2480** and ***ROW_LENGTH*** to **400**.

The effective frames per second is now set to ***30.048***. Effective frames per second are the actual throughput not the theoretical frames per second.

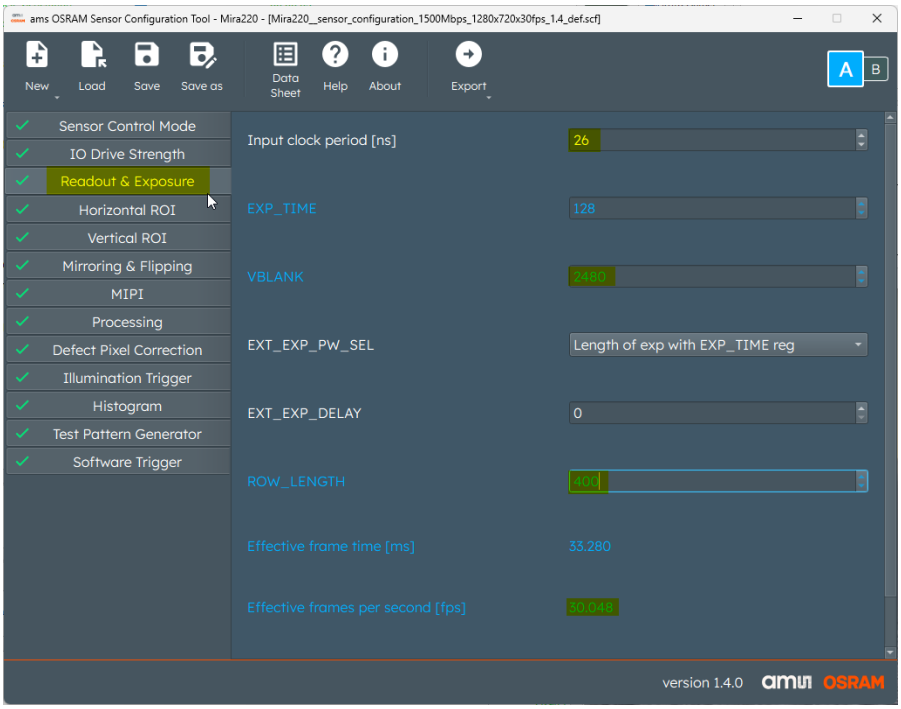


Figure 3-6 : Readout & Exposure Screen Settings

3.1.5 Save and Generate settings

On the menu bar, **click** the ***Save*** icon and navigate to the project folder to save the tool settings in case they need to be reloaded for modification.

To generate a settings file (.csv), **click** the ***Export*** icon and **select *Default [.csv]***. This action prompts you for a location to save the file in. Any location will do because you will be editing it there.

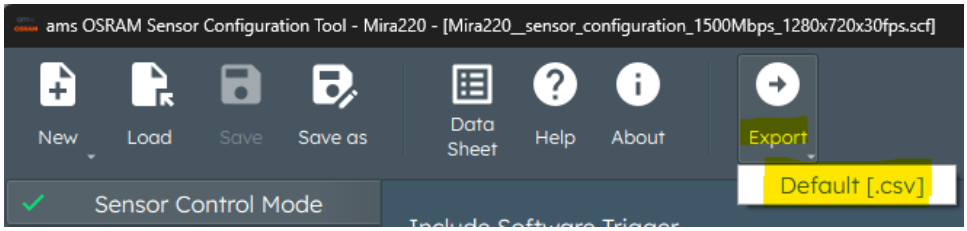


Figure 3-7 : Export Settings File

When you see a ***Warning*** window pop up, click **OK**.

3.1.6 Format the Settings File

The exported file is a text file in comma separated format (.csv) which needs to be edited to fit C format for an array.

Open the file in a text editor and make the format changes (using Find and Replace) to all lines, as shown in the example line below.

Ex: from 0x1003,0x2,Initial Upload to {0x1003,0x2}, //Initial Upload, as seen in the figure below.

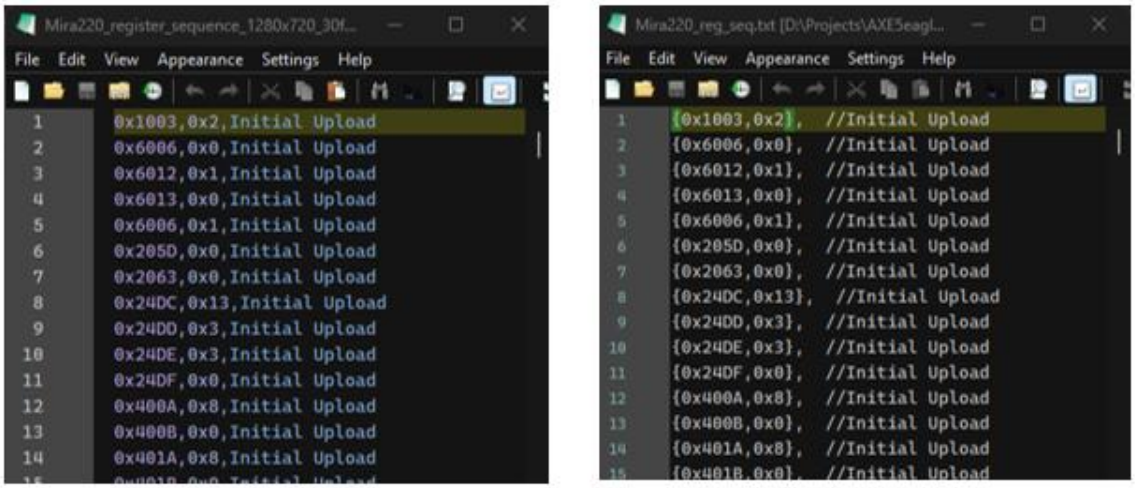


Figure 3-8 : Re-formatted settings file example

Now, this file is ready to be used in the C program.

4 Getting Started

There are a few hardware items required to use this reference design successfully:

- AXC3000 Evaluation Board with a USB-C cable.
- Trenz Camera and Display CRUVI Adapter (Trenz TEC0278-01).
- ams OSRAM Mira220_RGB_mini_SB kit (imager board, 15-pin flat ribbon cable, lens+lens holder), tripod bracket, and tripod. Otherwise, MIRA220_DEM_KT_RGB.
- Quartus Prime Pro 25.1.1, which is used for hardware development.

Note: If you want to acquire your own camera, you will need these 2 part numbers:

1. Imager board Mira220_RGB_mini_SB, which comes with the flat ribbon cable
2. Lens assembly AB04620MG from https://www.alaudoptics.com/AB04620MG.html

5 Design Flow

Overall, the flow for creating a MIPI CSI-2 design is:

- Use Quartus Prime Pro and Platform Designer
- Create & Configure the Clock sub-system
- Create & Configure the MIPI sub-system
- Create & Configure the NIOS-V sub-system
- Create a top-level Platform Designer block to contain the clock, mipi and nios-v subsystems
- Create the top-level Verilog design and assign pins.
- Compile the design.
- Create the Software Project to drive the demo.
- Validate the MIPI path with the camera

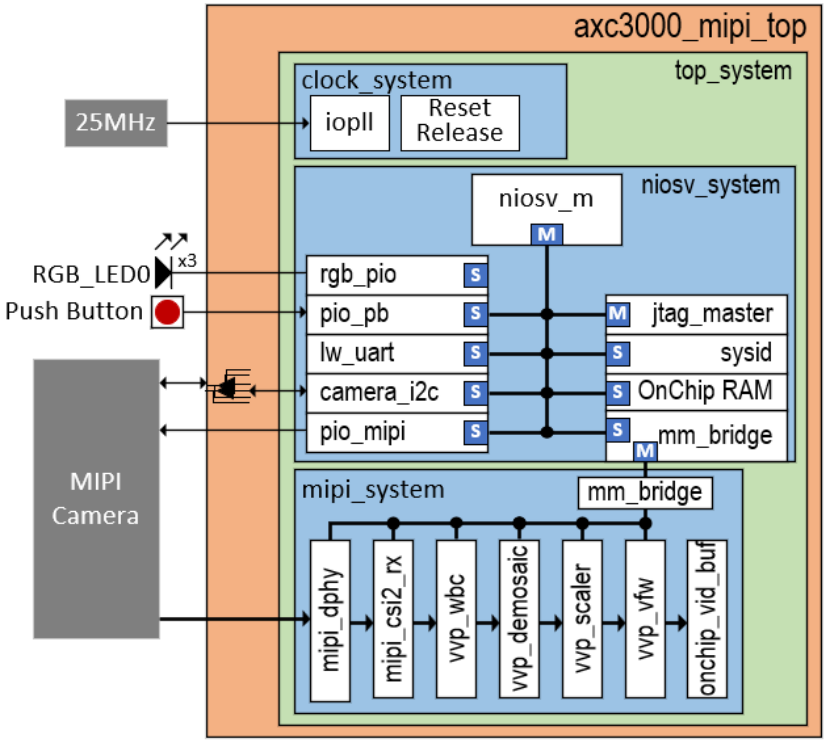


Figure 5-1: Project Block Diagram

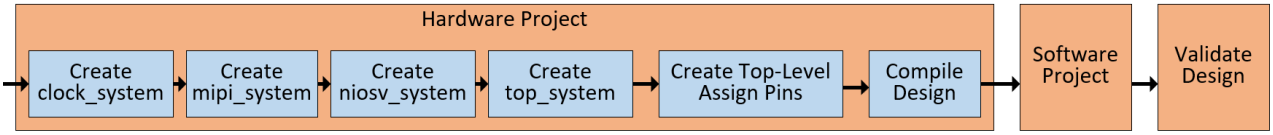


Figure 5-2 : MIPI Project Design Flow

6 Hardware Project with AXC3000 board

6.1 Elements of the design

The entire design is built with components obtained from the Quartus IP catalog. The Video and Vision Processing IP components require a license (provided for the workshop).

The Top-Level Verilog file (***axc3000_top_mipi.v***) holds the instantiation of ***top_system.qsys***, which is the Platform Designer component holding all other subsystems, and the I2C tri-state buffers.

top_system.qsys holds 3 subsystems, the ***niosv_system.qsys***, ***clock_system.qsys***, and ***mipi_system.qsys***, as seen in Figure 5-1.

Clock system:

- A PLL which generates a 150 MHz clock for the system clock (and video input path). It resides in the top-level design.
- Reset Release IP (necessary for Agilex FPGAs)

Niosv system:

- NIOS-V /m 32-bit RISC-V soft processor.
- 96KB (98,304 bytes) OnChip RAM for code storage and execution.
- PIO for RGB LED and camera enable.
- JTAG Master for debugging.
- UART for ***stdio*** operations.
- System ID.

mipi_system.qsys:

- MIPI DPHY hard IP
- CSI-2 soft IP for MIPI-CSI2
- White Balance Correction IP
- Demosaic IP
- Scaler IP
- Video Frame Writer IP
- Video Frame Reader IP
- 250KB (260,336 Bytes) OnChip RAM Video Frame Buffer.

6.2 New Quartus Prime Project

The hardware implementation starts with a Quartus® Prime project. A Quartus project defines the location for the HDL and IP files used, target family, target device, and initial

settings. After the initial creation of the project, changes can be made. Quartus uses the project location for the generated files.

6.2.1 New project creation

Before we begin, please open the **Command prompt** (cmd.exe) application and type the following command to download the lab files:

cd c:\

git clone https://github.com/ArrowElectronics/altera_workshops.git

Inside of the project's home folder (c:\altera_workshops\axc3000\MIPI_CSI2_lab), you will see the following sub-folders:

- **quartus** – Quartus working folder
- **sources** – Source Files folder
- **software** – Nios V software and Board Support Package
  - **mipi_app** – Application sub-folder
- **scripts** – scripts used when viewing the cideo frame buffer contents
- **completed_lab** – pre-compiled lab for reference

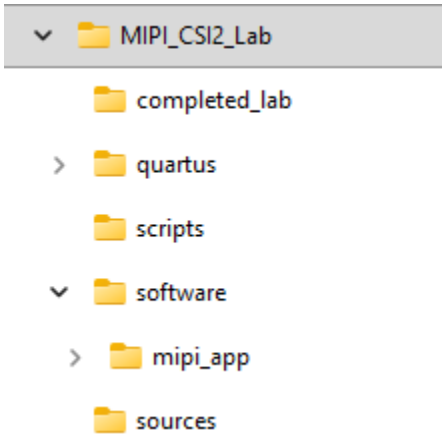


Figure 6-1 : Project folder structure

6.2.1.1 Open Quartus Pro 25.1.1

If not already open, from the Start menu or the Desktop, open the Quartus Prime Pro 25.1.1 IDE.

6.2.1.2 Create a new project

Using the New Project Wizard: **File → New Project Wizard**. This screen shows an introduction window. Click **Next.**

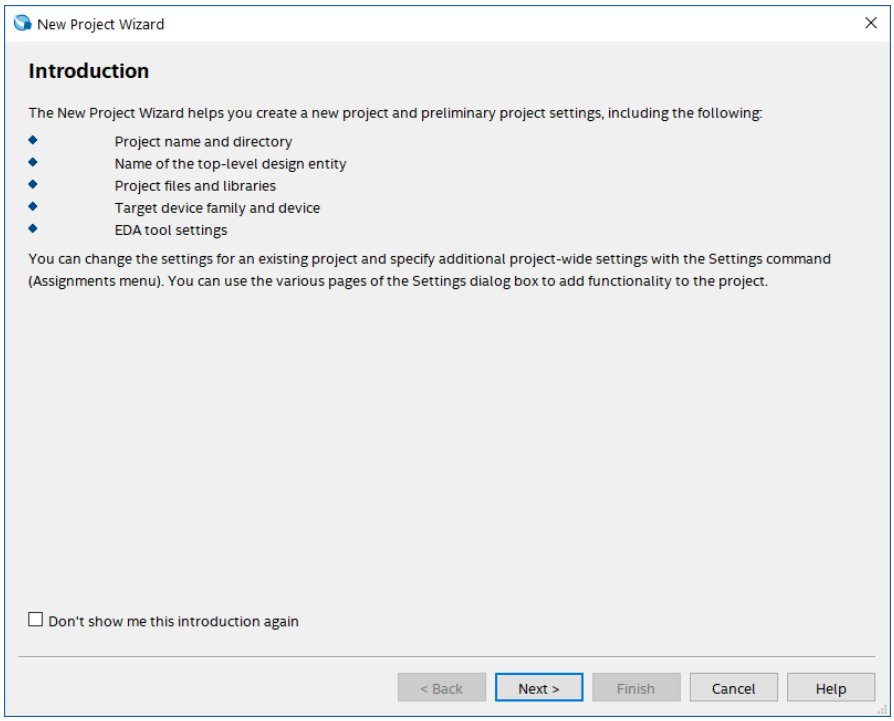


Figure 6-2 : New Project Wizard Introduction

6.2.1.3 Configure the New Project Wizard

Specify project directory, name, and top-level entity information:

Note: No spaces allowed in path or filenames.

- Ensure that the Empty Project radio button is selected on the left side of the window.
- Enter ***c:/altera_workshops/axc3000/MIPI_CSI2_lab*** for the working directory.
- Specify the name of the project: ***axc3000_mipi_top***
- Specify the name of the top-level entity: ***axc3000_mipi_top***
- Click **Next**.

If you are asked to allow creation of a non-existent folder, click **Yes**.

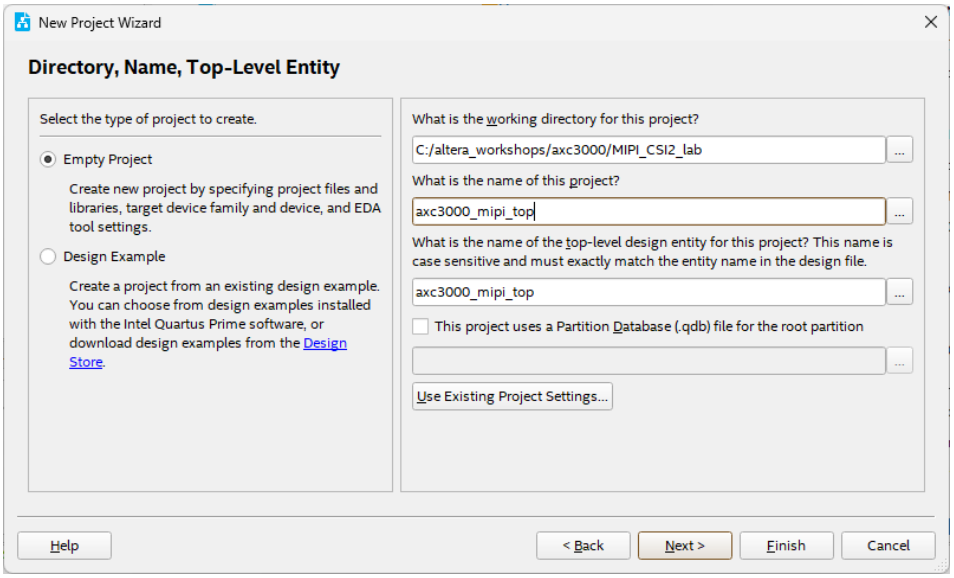


Figure 6-3 : New Project Wizard Folder/Project Names

6.2.1.4 Specify Family and Device Settings

In the Family field, select ***Agilex 3 (C-Series)*** and enter the part number (**A3CY100BM16AE7S**) in the Name Filter text box and select it. Click **Next**.

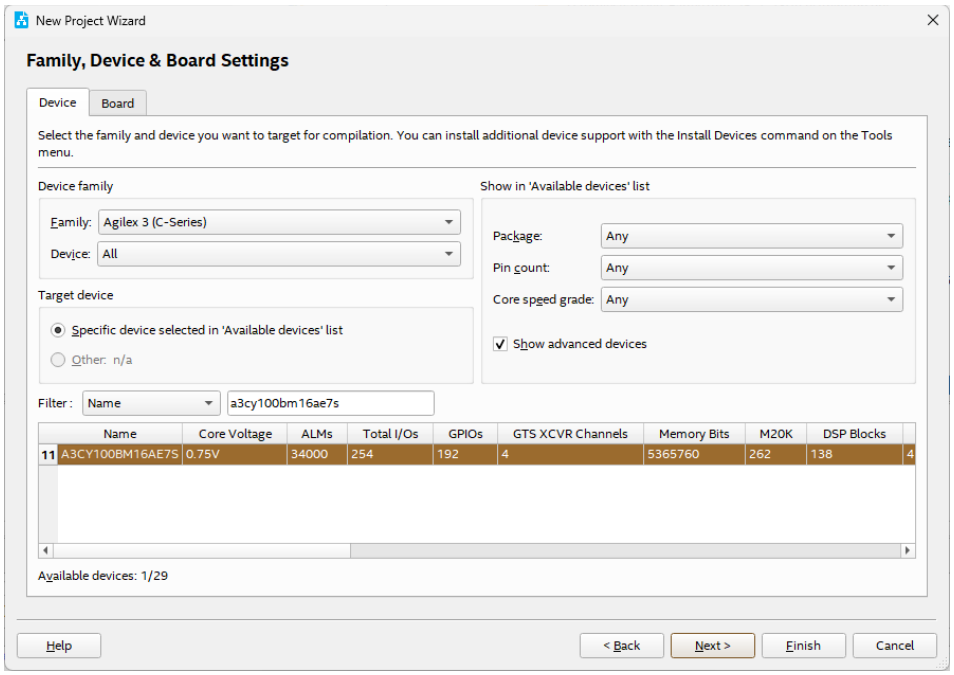


Figure 6-4 : New Project Wizard Device Selection

6.2.1.5 Add Files

On the Add Files page, click the "**...**" to navigate to the **sources** folder.

Change the "**File name**" category to **All Files (*.*)**. This allows the .sdc and .tcl files to also be visible. Select the 3 files, ***axc3000_mipi_top.v***, ***axc3000_mipi_pins.tcl*** and ***axc3000_mipi.sdc*** files, then click **Open**.

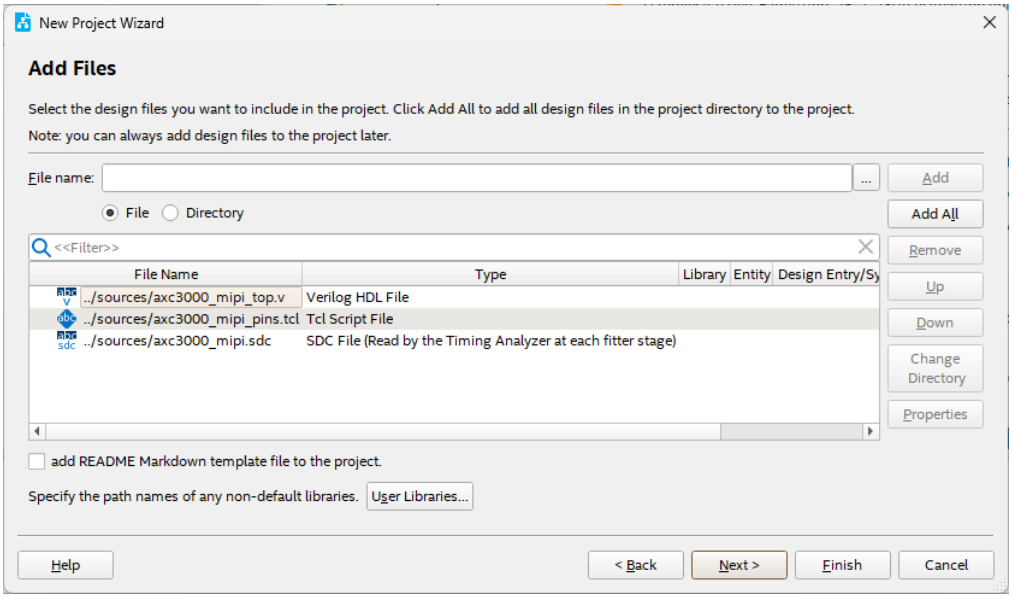


Figure 6-5 : New Project Wizard Add Script Files

Click **Next**.

6.2.1.6 EDA Tools Settings

On the EDA Tools Settings page, select **Next**.

6.2.1.7 Summary Screen

The Summary screen will show the project location and the device name. These settings can be changed if needed, later.

Click **Finish**.

6.2.2 Project Top Level

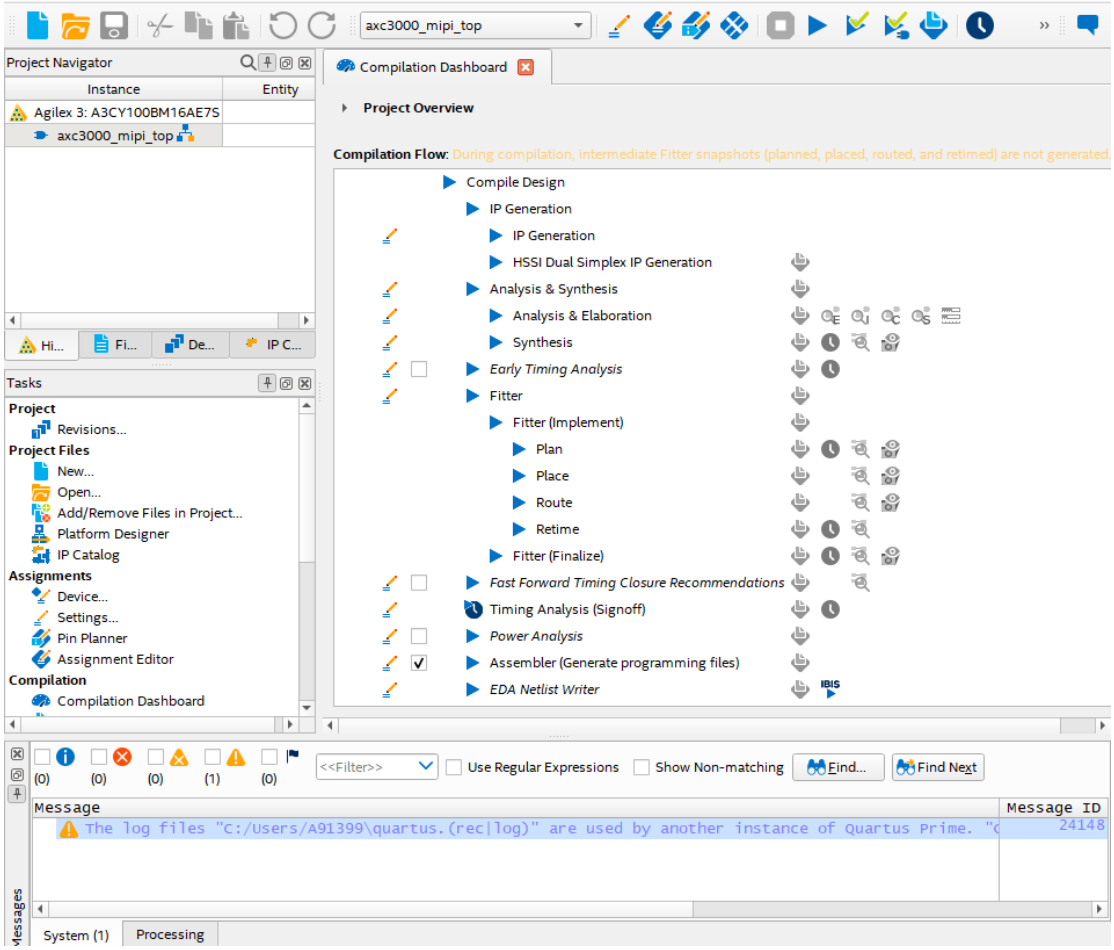


Figure 6-6 : Quartus Project View

The top-level file is seen in the top left corner. The middle section (compilation section) shows the FPGA steps which will be taken by the compiler. These steps include analysis and synthesis, fitter, etc.

The Agilex-3 requires an external clock source (OSC_CLK_1) which is used by the Secure Device Manager (SDM) for configuration. This needs to be set properly in Quartus. To do so, select the menu **Assignments -> Device** then click the **Device and Pin Options** button. When a window opens:

Select **General** in the ***Category*** pane and **Click** the ***Configuration clock source*** option bar and select ***25 MHz OSC_CLK_1 pin***.

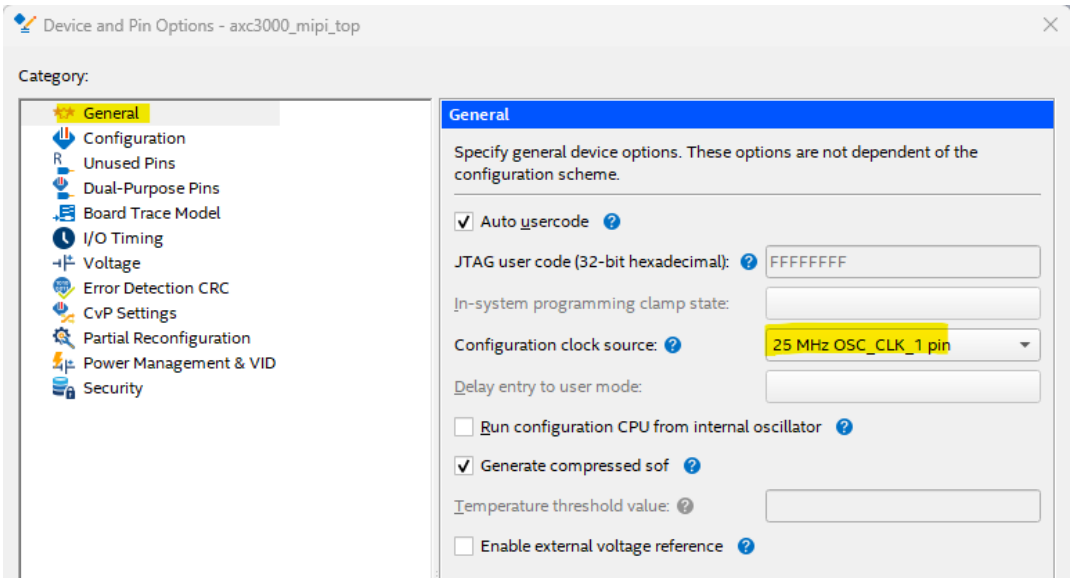


Figure 6-7 : Configuration Clock Source

Select **Configuration** in the **Category** pane and select **25 MHz Clock Frequency** for ***Active serial clock sources***.

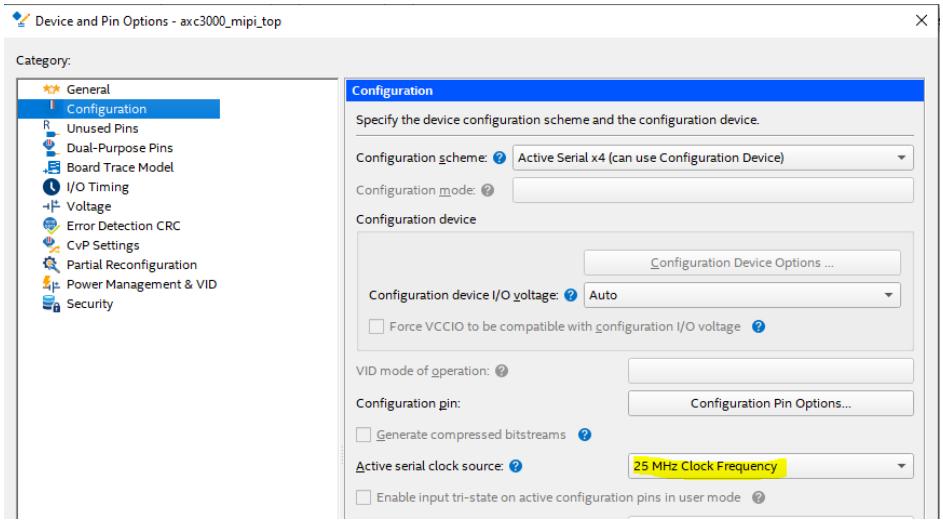


Figure 6-8 : Active Serial Clock Source

Click **OK** in each of the Assignment windows to apply the setting and close them.

6.3 Design Entry

In this module you will create the Platform Designer subsystems used in the design and compile the design.

There are different ways to create a design in Quartus including Verilog, System Verilog, VHDL, and IP (intellectual property) blocks. Quartus will accept a mixture of these file formats. Additionally, the IP catalog in Quartus contains many parameterizable functions.

In this lab, you will create 3 Platform Designer sub-systems. One is for the MIPI path, one for the NIOS-V and its peripherals, and one for the clock (PLL) and Reset elements. A 4th subsystem ties all 3 blocks together, which gets instantiated in the top-level Verilog file.

Clock subsystem

- PLL to multiply the 25MHz input clock
- Reset Release IP (necessary for Agilex FPGAs)

MIPI subsystem:

- MIPI DPHY hard IP
- MIPI CSI-2 soft IP
- White Balance Correction IP
- Demosaic IP
- Scaler IP
- Video Frame Writer IP
- 192KB (196608-byte) Onchip RAM for Video Frame Buffer

NIOS-V subsystem:

- NIOS-V /m 32-bit RISC-V soft processor
- 256KB (262144-byte) Onchip RAM for processor code storage and execution
- I^2C for Camera Interface
- PIO for push-button, LEDs, and camera enable
- System ID
- JTAG Master for debugging
- Memory-mapped Avalon Bridge for accessing registers in the mipi_system

6.3.1 Create Clock subsystem

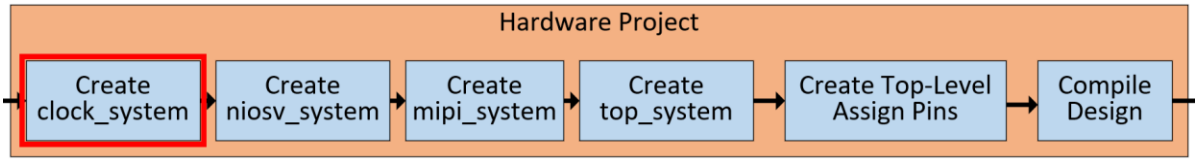


Figure 6-9: Create Clock/Reset subsystem

This subsystem adds the PLL, and Agilex-3/5 Reset Release.

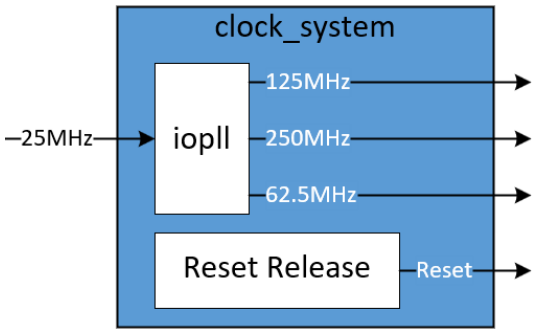


Figure 6-10: Clock/Reset block diagram

6.3.1.1 Creating clock_system.qsys for this lab.

In Quartus, select ***Tools -> Platform Designer*** to open ***Platform Designer*** and click the  to create a new subsystem. Enter **clock_system** in the ***File Name*** box and click **Create**. Click **Create** again.

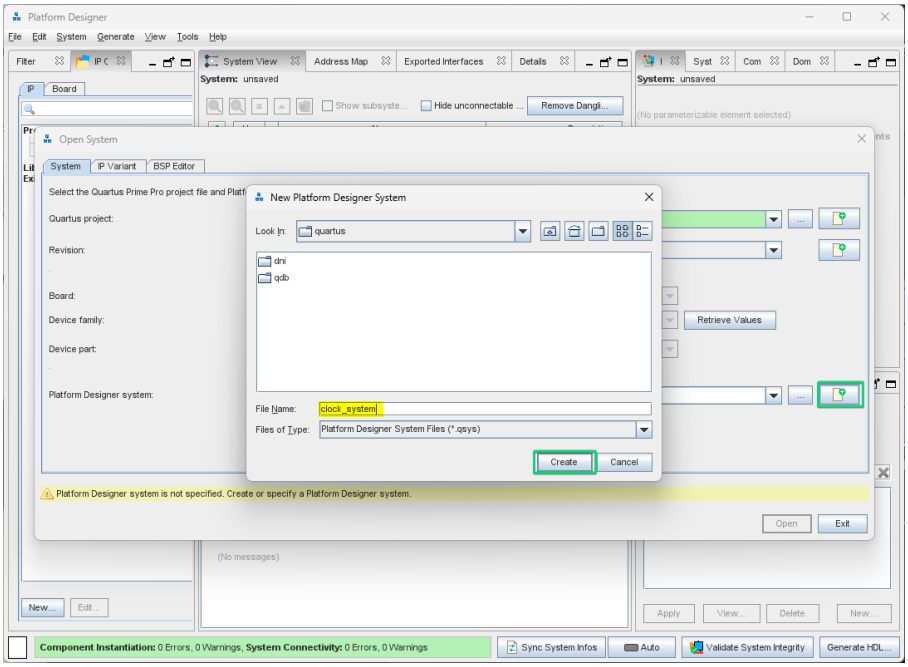


Figure 6-11: Create Platform Designer subsystem

When the ***Create New System Completed*** window is done, click **Close**.

The PD **System View** comes up with a **clock_in** and **reset_in** bridges already inserted. This is the means for passing a clock and a reset into the PD subsystem.

Select the **clock_in** name and set its frequency to **25000000** (25MHz) in the parameters pane (on the right). Double-Click its ***Clock Input*** signal name (in the ***Export*** column) and rename it **clk25m**. This is the external oscillator clock frequency.

Right-Click the **reset_in** name and select ***Rename***. Change its name to **reset_bridge**. Double-Click its ***Reset Input*** signal name (in the ***Export*** column) and erase it.

Double-Click the ***Reset Output*** signal name (in the ***Export*** column) and rename it **reset_out**.

In the IP Catalog, filter on "**release**". Double-click the **Reset Release IP**.

6.3.1.2 Add Reset Release IP

In the IP Catalog search field, filter on ***release***. Double-click the ***Reset Release IP***.

Select the radio button for ***Reset Interface*** and change its ***HDL entity name*** to ***reset_release***. Click **Finish**.

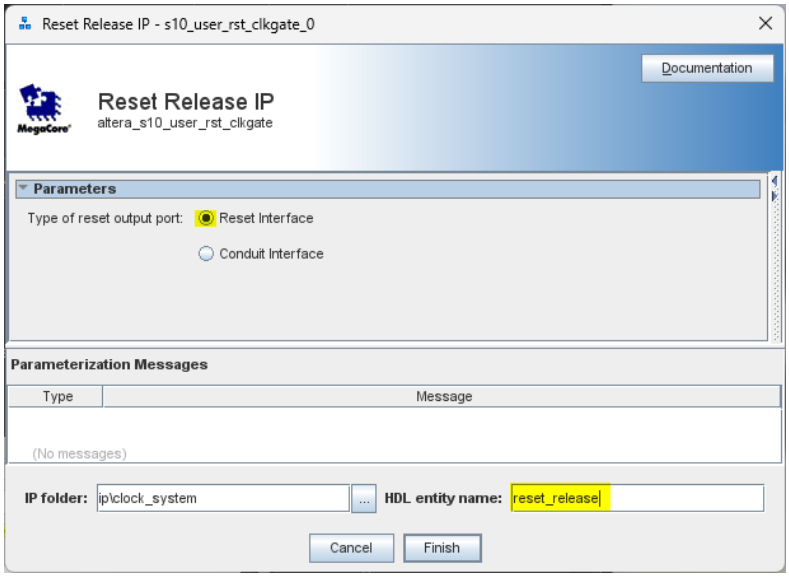


Figure 6-12: Add Reset Release IP

Connect the ***reset_release/ninit_done*** pin to the ***reset_bridge/in_reset*** input.

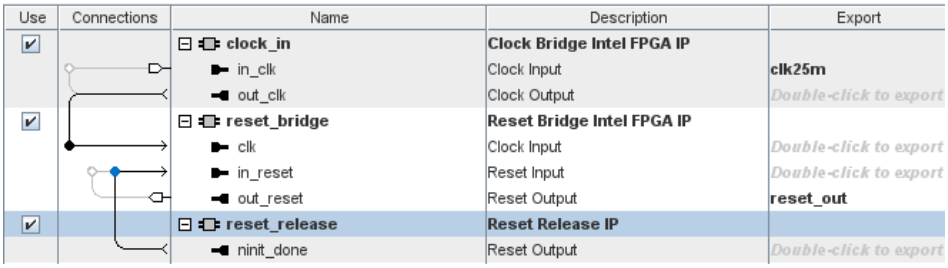


Figure 6-13: Reset connections

6.3.1.3 Add IOPLL IP

In the IP Catalog search field, filter on ***pll***. Double-click the ***IOPLL FPGA IP***.

Change its ***Reference Clock Frequency*** to **25.0** MHz.

Set the ***Number of Clocks*** to 1.

Change the ***outclk0***'s ***Desired Frequency*** to **150** MHz. This becomes the system clock.

Change the ***HDL entity name*** to **iopll**, then click **Finish**.

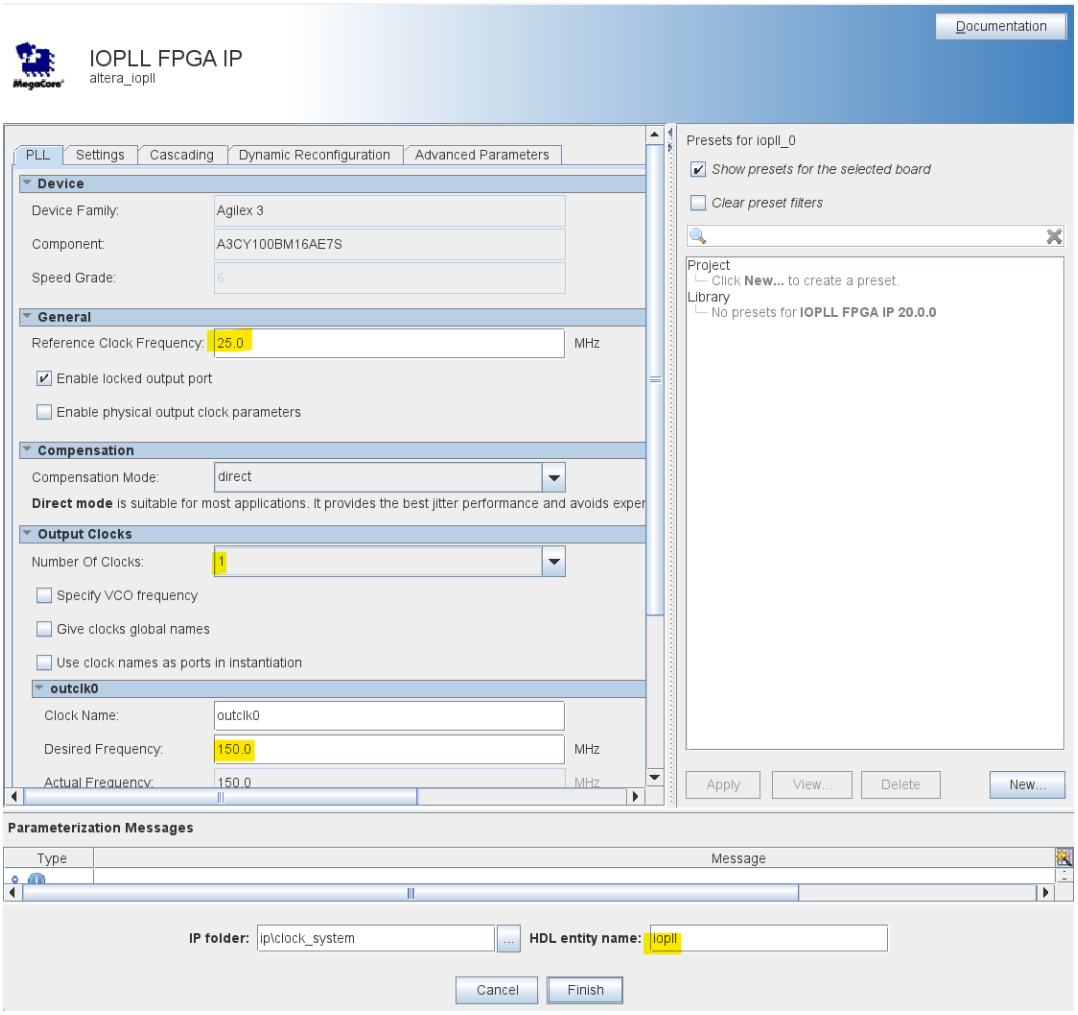


Figure 6-14: iopll Set up

Connect its ***refclk*** input to the ***clock_in/out_clk*** pin.

Connect its ***reset*** input to the ***reset_release/ninit_done***.

Export the ***outclk0*** as **sys_clk**.

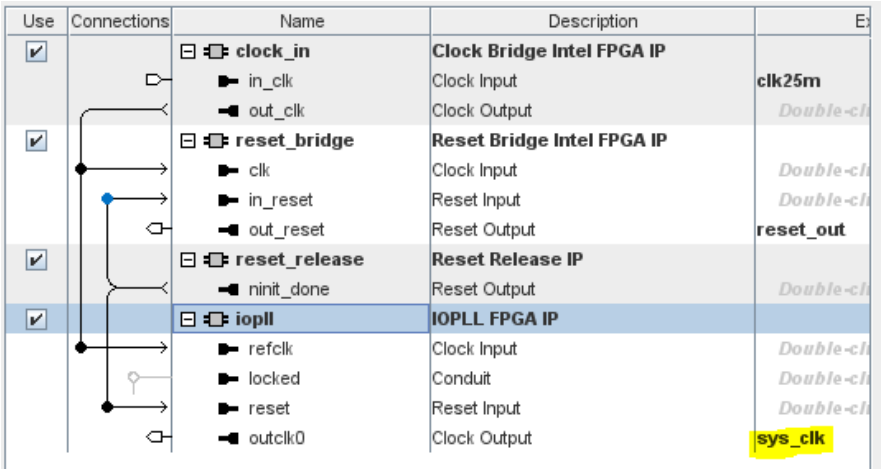


Figure 6-15: Add and connect IOPLL IP

6.3.1.4 Generate HDL

Now that the creation and connectivity are done, we need to extract HDL code from the IP blocks.

Click on the **Generate HDL** button in the lower right-hand corner, then click **Generate** in the pop-up window.

When prompted to ***Save changes***, click **Yes**.

Once the ***Generate*** is done without errors, click **Close**.

Save the design and exit the Platform Designer session

6.3.2 Create MIPI subsystem

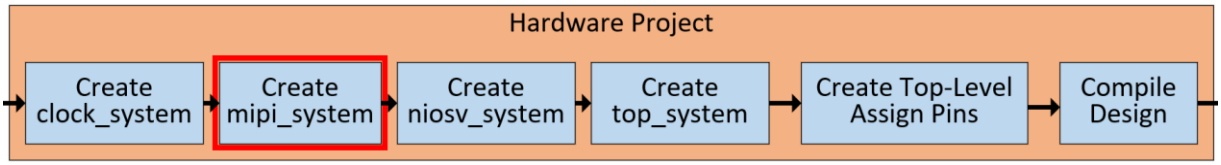


Figure 6-16: Add Video Path Modules

The figure below shows the video data flow between the external MIPI camera/display and the frame buffer.

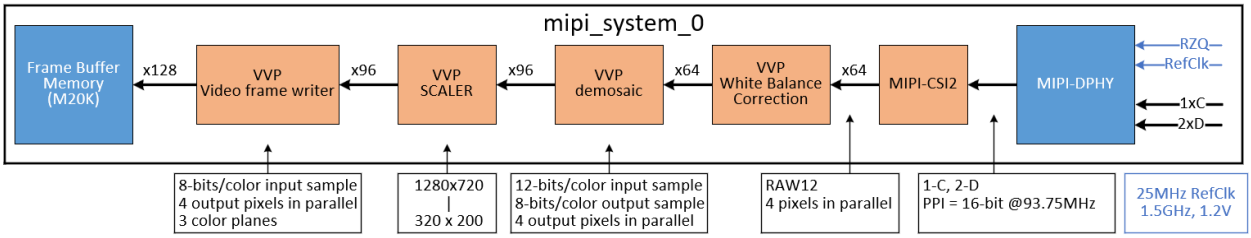


Figure 6-17: MIPI subsystem Data Flow Diagram

6.3.2.1 Using Platform Designer (PD)

The MIPI video path modules will be in their own subordinate (**mipi_system**) which gets instantiated with the other modules in the main Platform Designer, **top_system.qsys**.

PD can be opened by clicking the  symbol on the Quartus menu bar or selecting **Tools -> Platform Designer** from the drop-down menus.

PD will ask you to either select an existing .qsys design or create a new one.

On the **Platform Designer system:** line, click  to open an existing file, or  to create a new one. If creating a new one, enter a name for it in the **File Name:** box and click **Create** in each of the 2 windows.

Creating **mipi_system.qsys** for this lab.

Open Platform Designer.

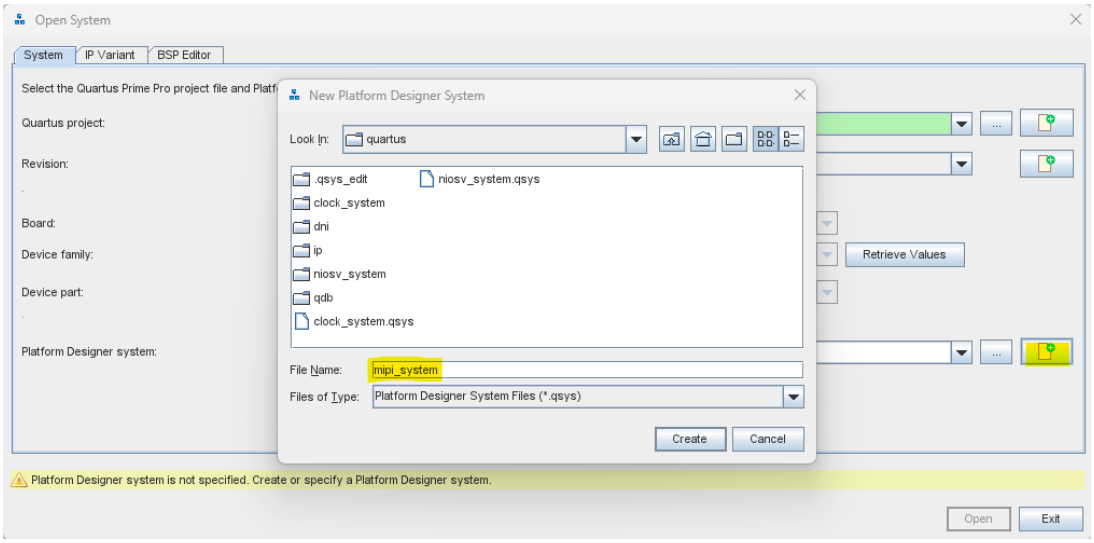


Figure 6-18: Create new PD design

There are a few modules that need to be added, as seen in the diagram below.

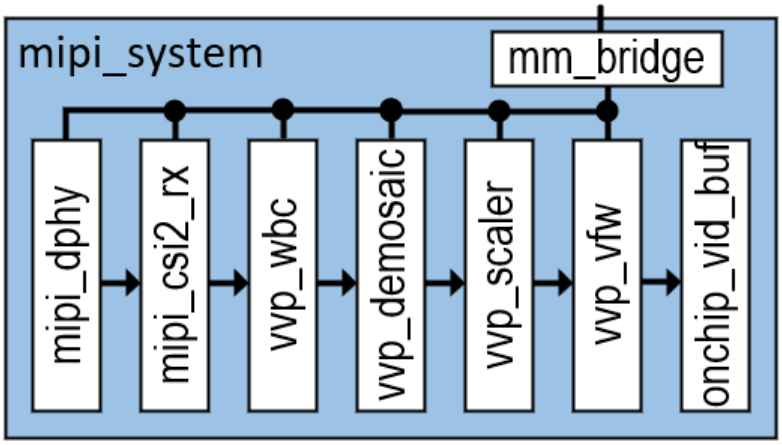


Figure 6-19 : mipi_system modules

The PD **System View** comes up with a **clock_in** and **reset_in** bridges already inserted. This is the means for passing a clock and a reset into the PD subsystem.

Right-Click the **clock_in** name and select ***Rename***. Change its name to **sys_clock_bridge**, and set its frequency to **125000000** (125 MHz) in the parameters pane (on the right). Double-Click its ***Clock Input*** signal name (in the *Export* column) and rename it **sys_clk_in**. This is the CPU bus frequency.

Right-Click the **reset_in** name and select ***Rename***. Change its name to **sys_rst_bridge**. Double-Click its ***Reset Input*** signal name (in the *Export* column) and rename it **sys_rst_in**.

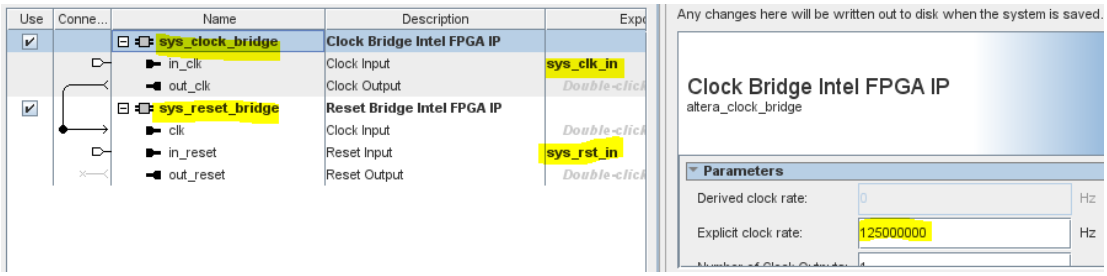


Figure 6-20 : Rename Clock and Reset Bridges

6.3.2.2 Add new Bridges

We need to connect the CPU bus into this subsystem. This is done via another bridge. In the IP Catalog, filter on "**bridge**". Double-click the **Avalon Memory Mapped Pipeline Bridge Intel FPGA IP**. **Check** the box for ***Use automatically-determined address width*** and change its ***HDL entity name*** to **vid_mm_bridge**. Click **Finish**.

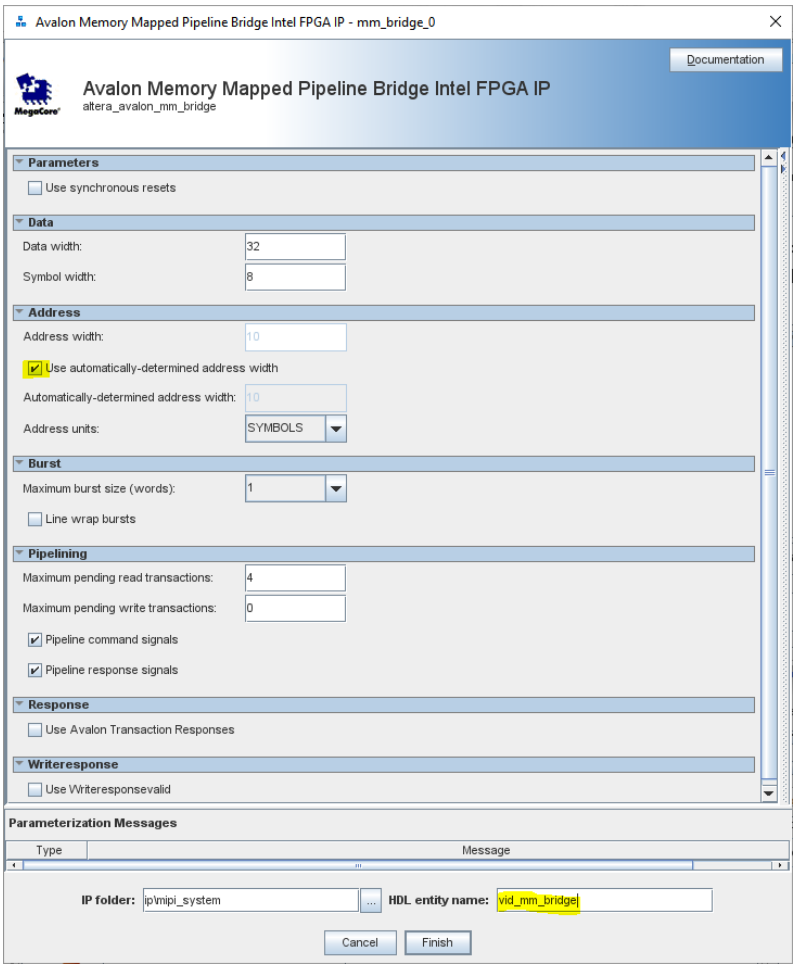


Figure 6-21: Add Memory-Mapped Pipeline Bridge

Export the **vid_mm_bridge/s0** bus as **vid_mm_bus** by double-clicking in the ***Export*** Column and adding the signal name.

Connect the **sys_clock_bridge/out_clk** pin to its **clk** input.

Connect the **sys_reset_bridge/out_reset** pin to its **reset** input.

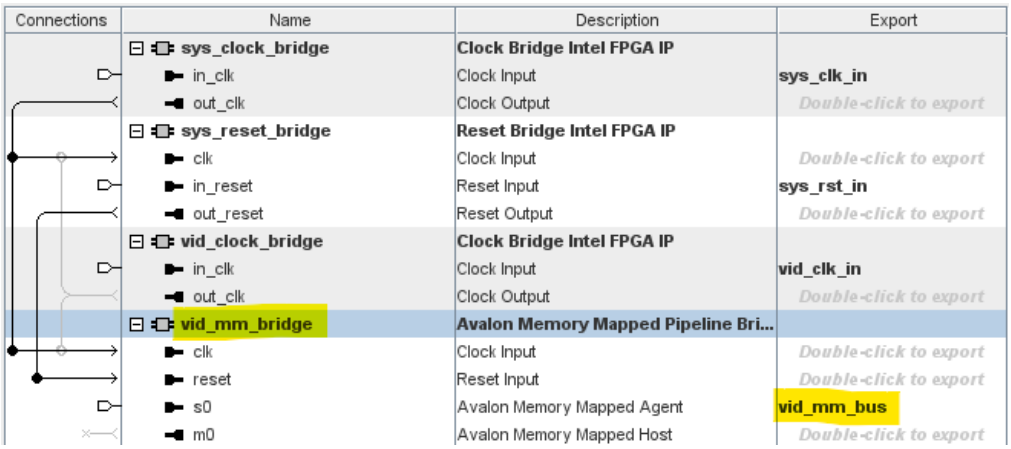


Figure 6-22: Connect Clock and Reset to MM Bridge

6.3.2.3 Add MIPI DPHY IP

In the **IP Catalog** search field, type ***mipi***. This filters the D-PHY and CSI-2 IP blocks. Double-click the **MIPI DPHY IP**.

Click the ***Link 0*** tab. Set the ***DPHY Role*** to **Rx**. Set the ***Bitrate*** to **1500.0**. Set the ***Number of Lanes*** to **1 Clk & 2 Data**. Set the ***Continuous Clock*** to **True**. Set the ***Byte Location*** to **6**, which is what lines up with the CRUVI_HS interface for a MIPI CSI-2.

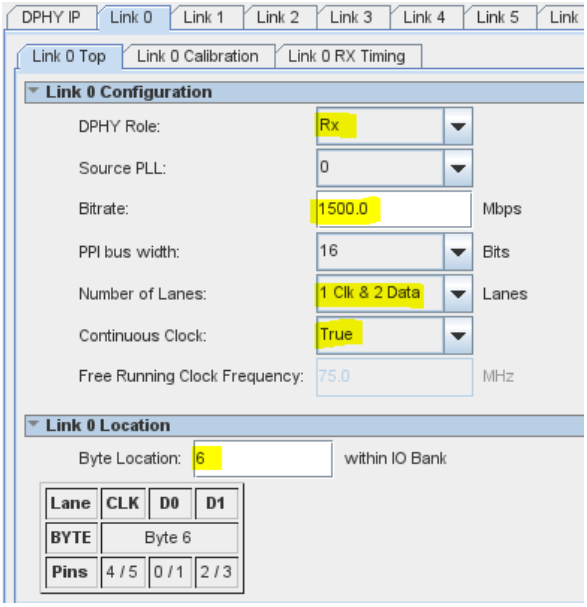


Figure 6-23: DPHY Link0 top-level settings

Select ***Link 0*** and click the ***Link0 Calibration*** sub-tab. Set the ***Skew calibration enable*** to **False** and the ***Rx Equalization mode*** to **Disable**. The speeds we are using do not require enhanced equalization.

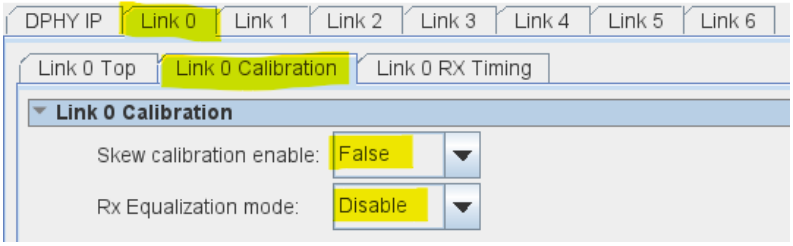


Figure 6-24: DPHY Link0 RX Calibration settings

Select Link 0 and click the Link0 RX Timing sub-tab. Set the RX Data lane deskew delay 0 to 40 and the RX Data lane deskew delay 1 to 40.

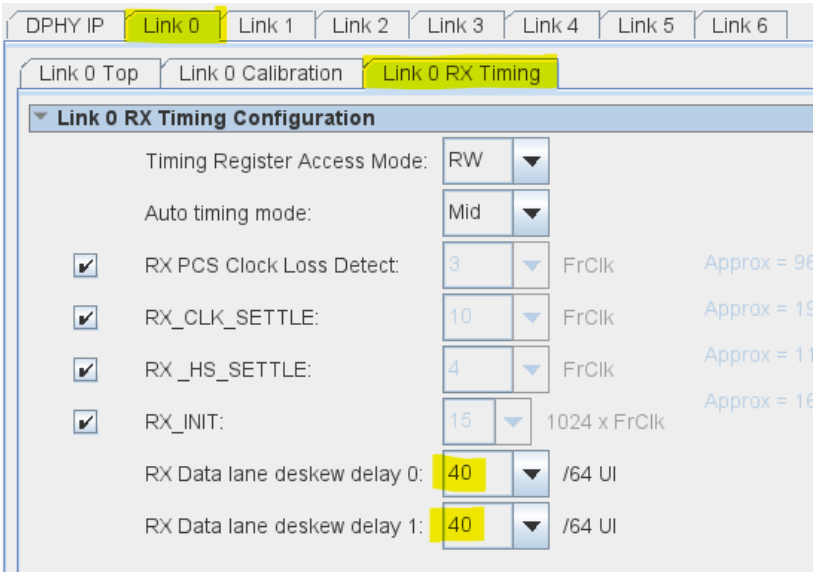


Figure 6-25: DPHY Link0 RX Calibration settings

Click the **DPHY IP** tab and select the **PLL** sub-tab. Set the ***Reference Clock Frequency*** to **25.0,** and the ***Reference Clock IO Type*** to **Single-ended 1.2V**. This is due to the AXC3000 board having a single-ended 25 MHz input reference clock. Set the ***VCO Clock Frequency*** to **750.0** MHz in order to get the 1500 MHz bit rate.

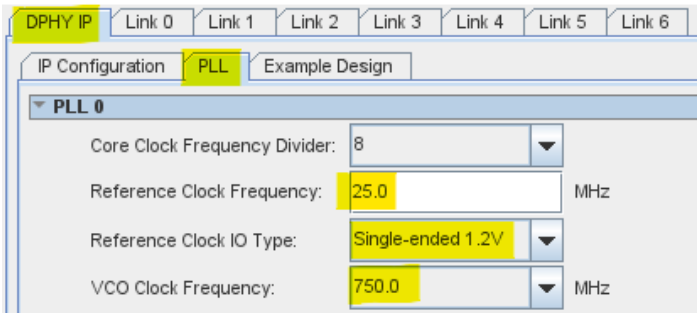


Figure 6-26: DPHY PLL settings

Change the ***HDL entity name*** to **mipi_dphy**. Click **Finish**.

Export the ***rzq*** pin as **mipi_dphy_rzq**.
Export the ***ref_clk_0*** pin as **mipi_dphy_refclk**.
Connect the ***arst*** and ***reg_srst*** inputs to the ***sys_reset_bridge/out_reset*** pin.
Connect the ***reg_clk*** input to the ***sys_clock_bridge/out_clk*** pin.
Connect the ***axi_lite*** bus to the ***vid_mm_bridge /m0*** bus.
Export the ***LINK0_dphy_io*** bus as **mipi_csi2_io**.

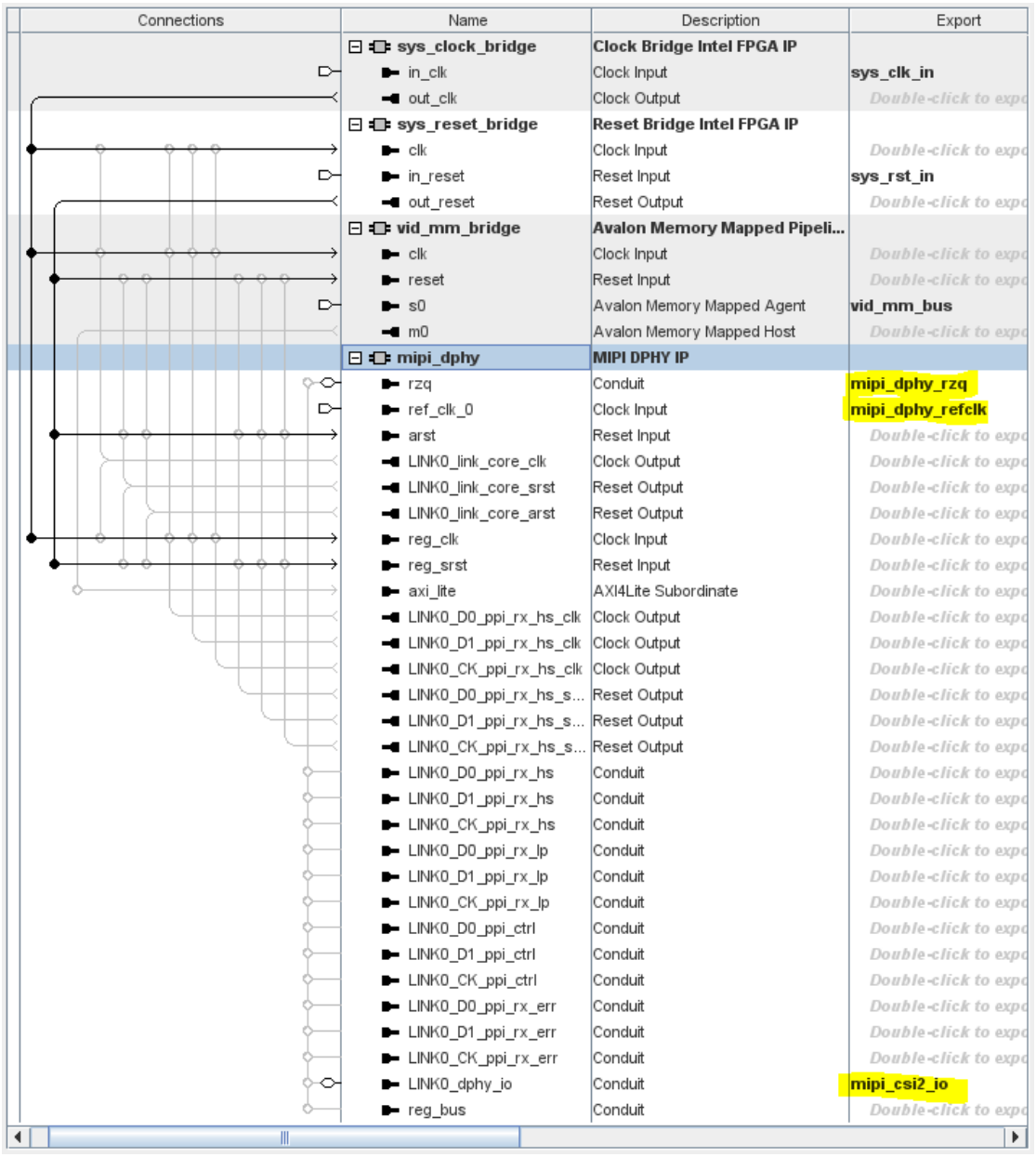


Figure 6-27: DPHY external connections

6.3.2.4 Add MIPI CSI-2 IP

The **MIPI-CSI2** IP module take the data from the D-PHY, buffers it and outputs 4 pixels in parallel in RAW12 format.

Double-click the ***MIPI CSI-2 IP***. Set the ***Direction*** to **Rx**. Set the ***Buffer Depth*** to **4096**. Set the ***Pixels in Parallel*** to **4**. **Unselect** the ***Support RGB888*** and **Select** the ***Support RAW12 Video Packing***.

Change the ***HDL entity name*** to **mipi_csi2_rx** and click **Finish**.

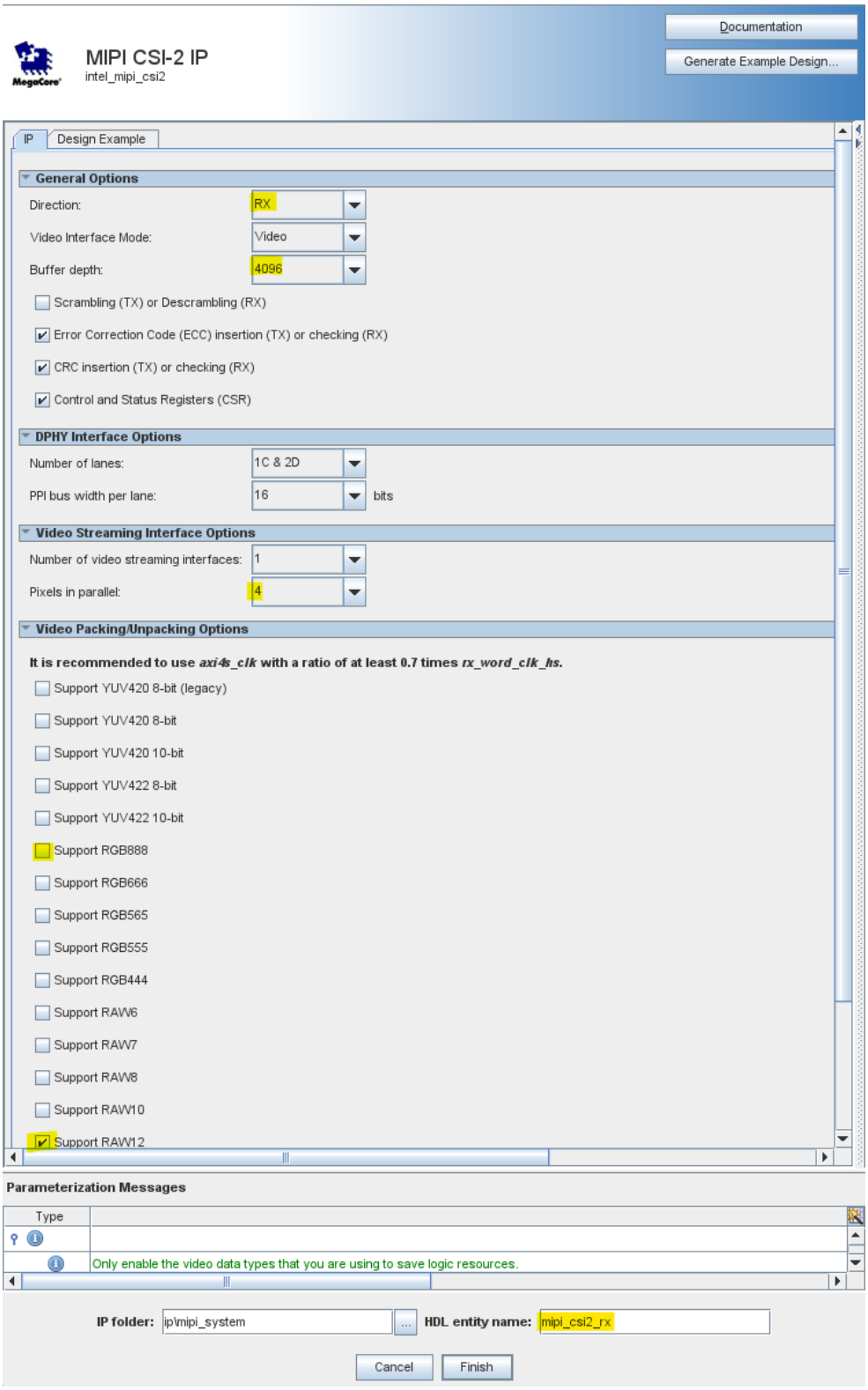


Figure 6-28: CSI-2 RX settings

Connect the ***axi4s_clk*** input to the ***sys_clock_bridge/out_clk*** pin.

Connect the ***axi4s_rst*** to the ***sys_reset_bridge/out_reset*** pin.

Connect the ***avalon_mm_control_interface*** bus to the ***vid_mm_bridge/m0*** bus.

Connect the ***d0_ppi_hs_clk*** pin to the ***mipi_dphy/LINK0_D0_ppi_rx_hs_clk*** pin.

Connect the ***d0_ppi_rx_hs_srst*** pin to the ***mipi_dphy/LINK0_D0_ppi_rx_hs_srst*** pin.

Connect the ***d0_ppi_hs*** pin to the ***mipi_dphy/LINK0_D0_ppi_rx_hs*** pin by clicking on 2 dots near those pins.

Connect the ***d0_ppi_lp*** pin to the ***mipi_dphy/LINK0_D0_ppi_rx_lp*** pin by clicking on 2 dots near those pins.

Connect the ***d0_ppi_ctrl*** pin to the ***mipi_dphy/LINK0_D0_ppi_ctrl*** pin by clicking on 2 dots near those pins.

Connect the ***i_d0_ppi_rx_err*** pin to the ***mipi_dphy/LINK0_D0_ppi_rx_err*** pin by clicking on 2 dots near those pins.

Make the same connections for the ***LINK0_D1*** and **LINK0_CK** pin groups.

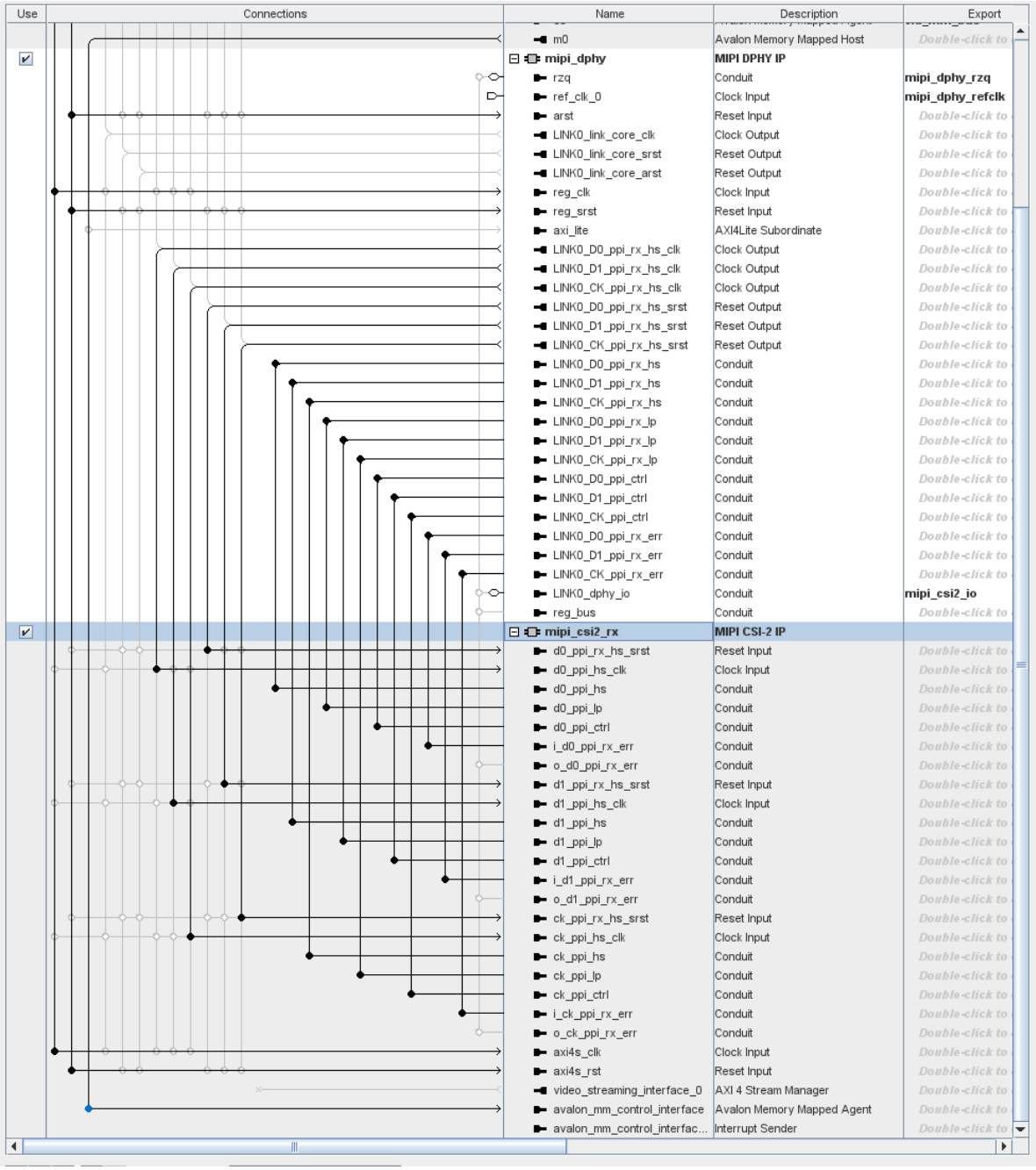


Figure 6-29: CSI-2 Input connections

6.3.2.5 Add White Balance Correction IP

The White Balance Correction IP scales individual color channels of a 2x2 color filter array to correct color distortion.

In the IP Catalog search field, enter ***white***. Double-click the **White Balance Correction**.

Set the ***Input bits per color sample*** to **12**.

Set the ***Output bits per color sample*** to **12**.

Set the **Number of pixels in parallel** to **4**.

Keep the ***Maximum field height*** field set to **1024**.

Keep the ***Maximum field width*** field set to **2048**.

Check the boxes for Pipeline ready signals, Separate clock for control interface and Debug features.

Change the ***HDL entity name*** to **vvp_wbc** and click **Finish**.

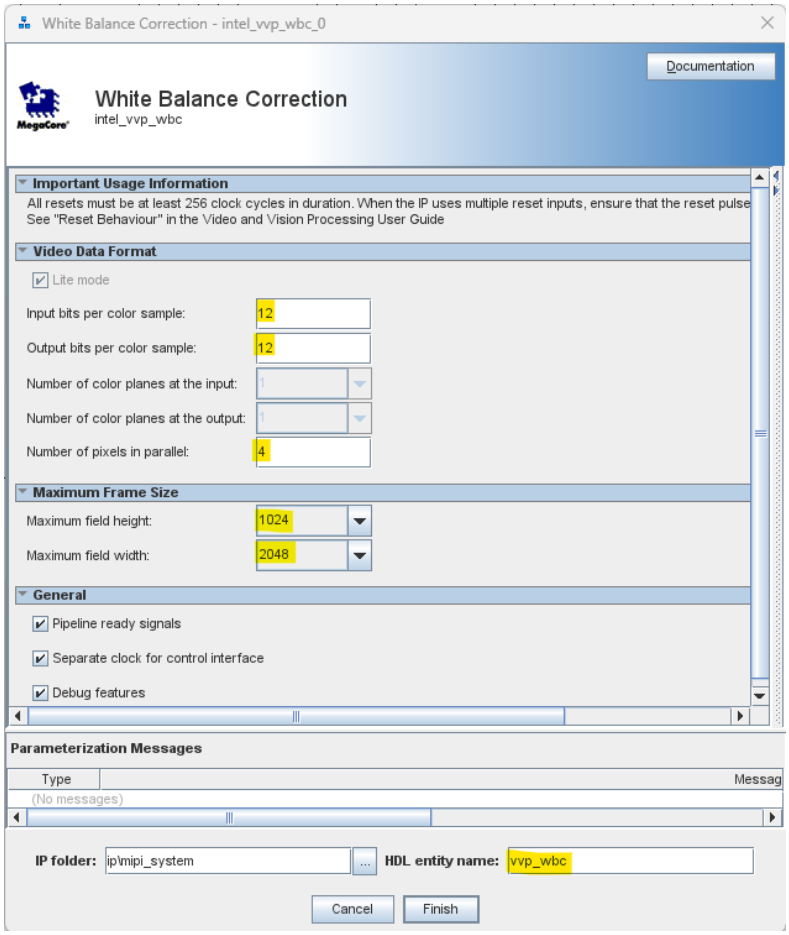


Figure 6-30: White Balance Correction settings

Connect the ***axi4s_vid_in*** to the ***mipi_csi2_rx/video_streaming_interface_0***.

Connect the ***main_clock*** to the ***sys_clock_bridge/out_clk***.

Connect the ***main_reset*** to the ***sys_reset_bridge/out_reset***.

Connect the ***agent_clock*** to the ***sys_clock_bridge/out_clk***.

Connect the ***agent_reset*** to the ***sys_reset_bridge/out_reset***.

Connect the ***av_mm_control_agent*** to ***vid_mm_bridge/m0***.

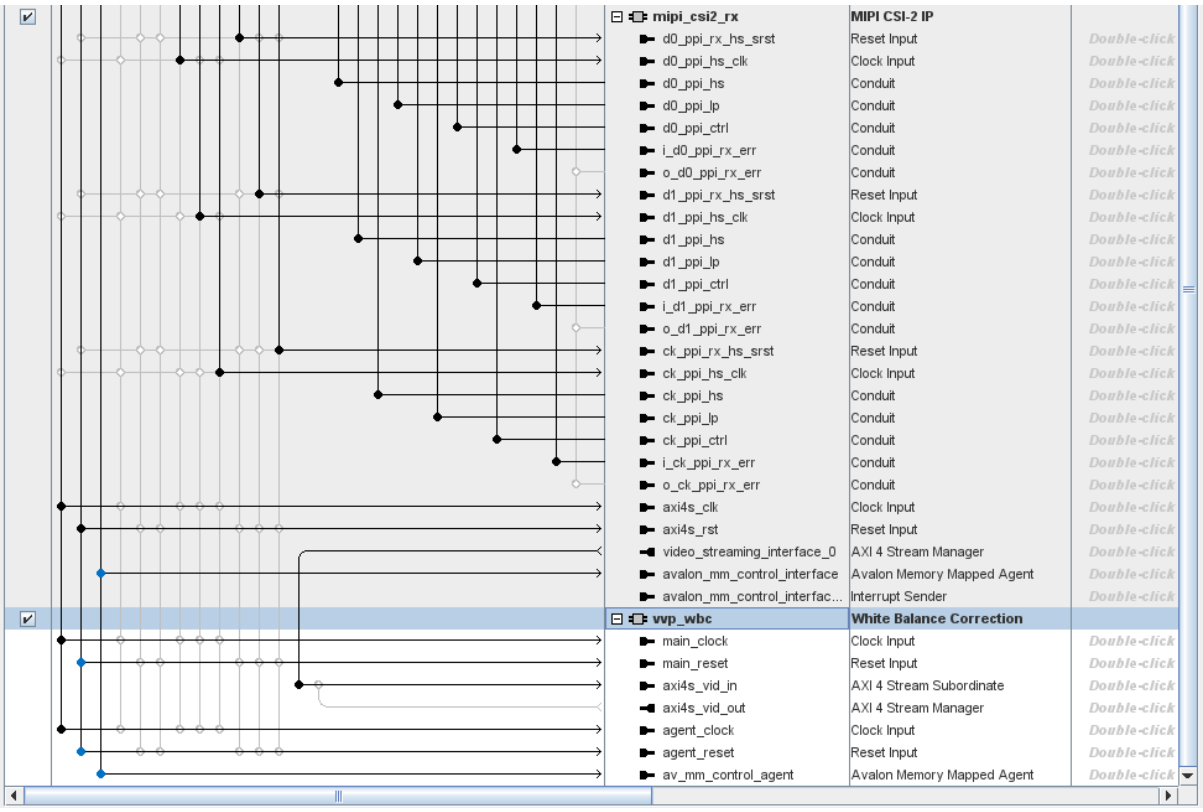


Figure 6-31: White Balance Correction connections

6.3.2.6 Add Demosaic IP

The ***Demosaic*** IP converts the data from a 2x2 Bayer filter array video stream into an RGB video stream.

In the IP Catalog search field, enter ***demosaic***. Double-click the **Demosaic**.

Set the ***Input bits per color sample*** to **12**.

Set the ***Output bits per color sample*** to **8**.

Set the ***Number of pixels in parallel*** to **4**.

Change the ***Maximum field height*** field to **1024**.

Change the ***Maximum field width*** field to **2048**.

Check the boxes for ***Pipeline ready signals***, ***Separate clock for control interface*** and ***Debug features***.

Change the ***HDL entity name*** to **vvp_demosaic**, then click **Finish**.

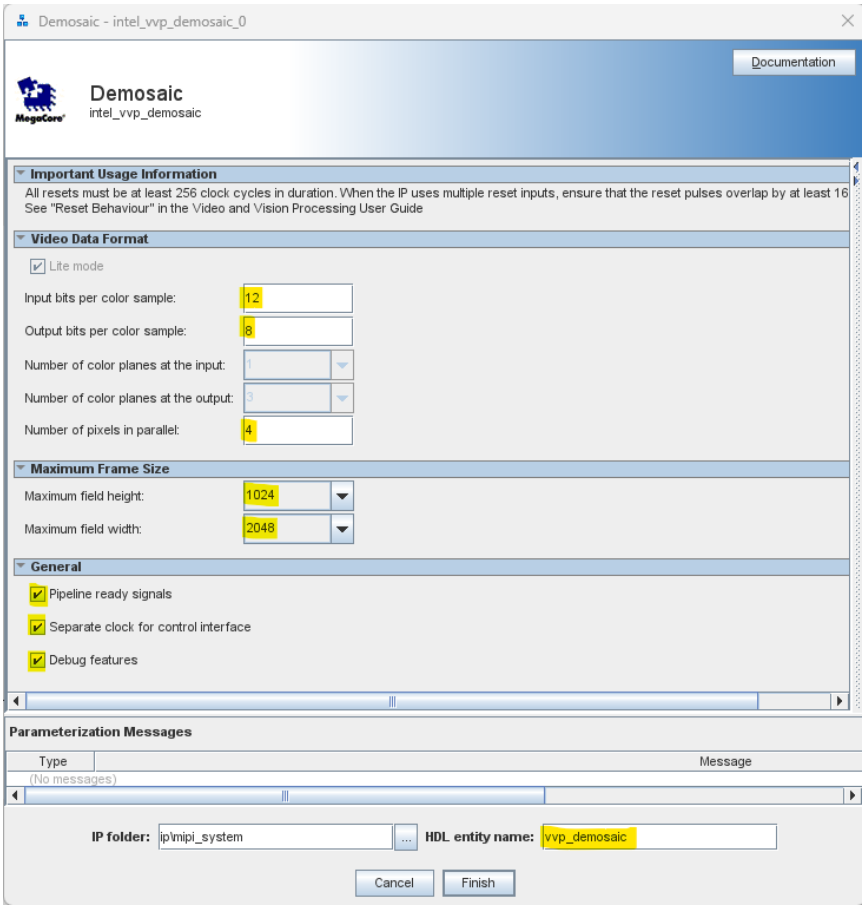


Figure 6-32: Add Demosaic IP

Connect the ***axi4s_vid_in*** to the ***vvp_wbc/axi4s_vid_out***.

Connect the ***main_clock*** to the ***sys_clock_bridge/out_clk***.

Connect the ***main_reset*** to the ***sys_reset_bridge/out_reset***.

Connect the ***agent_clock*** to the ***sys_clock_bridge/out_clk***.

Connect the ***agent_reset*** to the ***sys_reset_bridge/out_reset***.

Connect the **av_mm_control_agent** to ***vid_mm_bridge/m0***.

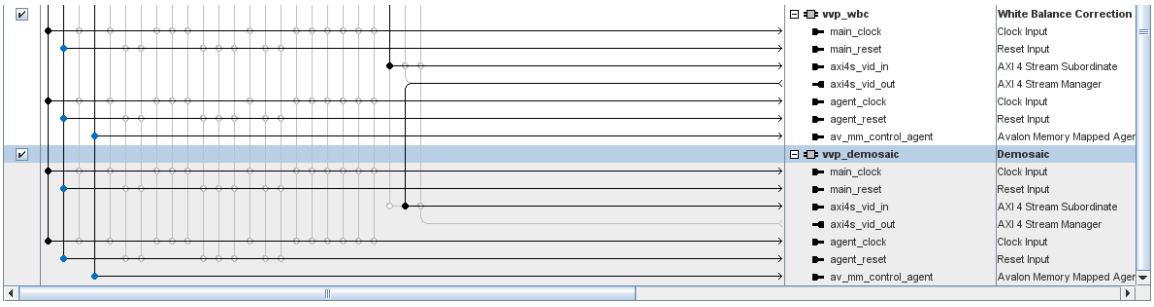


Figure 6-33: Demosaic Input connections

6.3.2.7 Add Scaler IP

Since the camera initialization is being done for a 1600x900 frame size, the Scaler IP is used to convert the horizontal resolution from 1600 to 320 and the vertical resolution from 900 to 200.

The Scaler module takes the 1600x900 input image and scales it to 320x200 RGB format.

In the IP Catalog search field, enter ***scaler***. Double-click the **Video and Vision Processing/Scaler**.

Check the ***Lite mode*** box.

Set the ***Bits per color sample*** to **8**, ***Number of color planes*** to **3**, and ***Number of pixels in parallel*** to **4**.

Set the ***Maximum input field width*** to **1280**, and ***Maximum output field width*** to **512**.

Set the ***Scaling algorithm*** to **Nearest neighbor**.

Check the box for ***Vertical scaling***.

Check the box for ***Horizontal scaling***.

Check the box for ***Separate clock for control interface***.

Change the ***HDL entity name*** to **vvp_scaler**.

Click **Finish**.

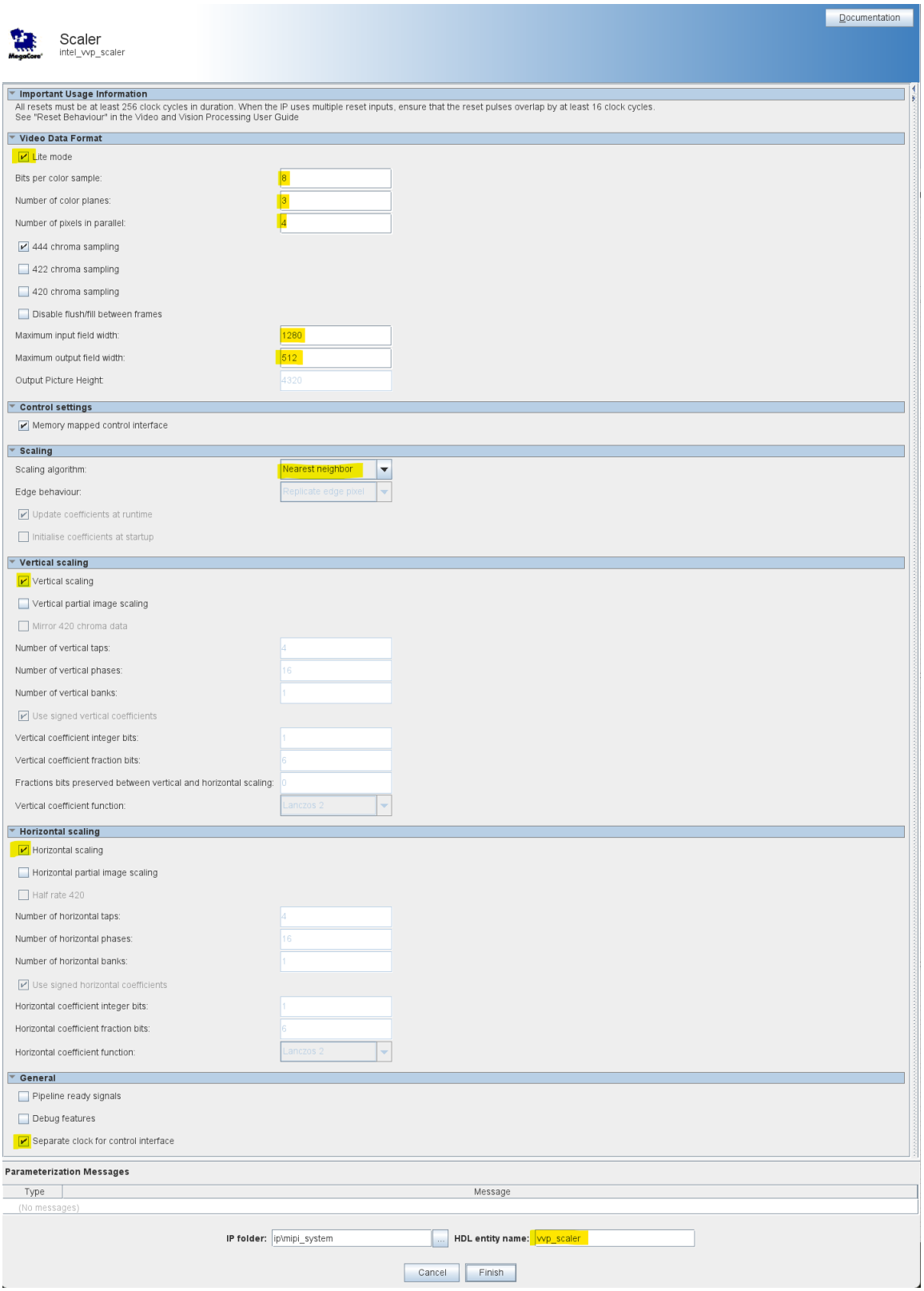


Figure 6-34: Add Scaler IP

Connect the ***axi4s_vid_in*** to the ***vvp_demosaic/axi4s_vid_out***.

Connect the ***main_clock*** to the ***sys_clock_bridge/out_clk***.

Connect the ***main_reset*** to the ***sys_reset_bridge/out_reset***.

Connect the ***agent_clock*** to the ***sys_clock_bridge/out_clk***.

Connect the ***agent_reset*** to the ***sys_reset_bridge/out_reset***.

Connect the ***av_mm_control_agent*** to ***vid_mm_bridge/m0***.

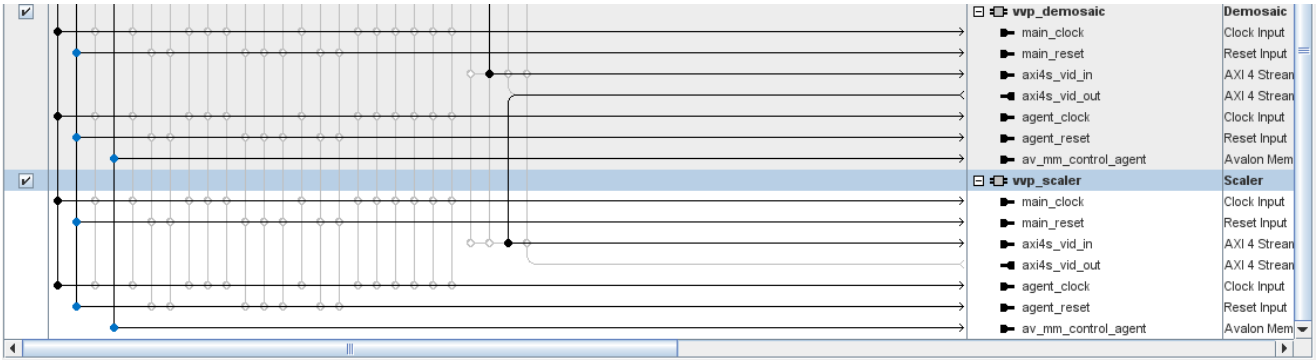


Figure 6-35: Scaler Input connections

6.3.2.8 Add Video Frame Writer IP

The video frame writer is the last IP in the video input path. It writes video data in 128-bit width into 1 frame buffer located in the on-chip SRAM. The 320x200 24-bit color frame needs 1,536,000 bits (192,000 bytes).

In the IP Catalog search field, enter ***vfw***. Double-click the **Video Frame Writer**.

Set the Bits per color sample to 8, Number of color planes to 3, and Number of pixels in parallel to 4.

Set the ***Maximum field height*** to **1080**. This allows for larger images in the future.

Set the ***Maximum field width*** to **1920**. This allows for larger images in the future.

Check the boxes for ***Lite mode*** and ***Separate clock for control interface***.

Set the ***Avalon memory-mapped local ports width*** to **128** (due to having 4 Pixels-in-Parallel), The ***depth of the write FIFO*** to **512**, the ***Avalon memory-mapped write burst target*** to **32**, and the ***Packing method*** to **PERFECT**.

Check the box for ***Separate clock for the Avalon memory-mapped host interface(s)***.

Change the ***HDL entity name*** to **vvp_vfw**.

Click **Finish**.

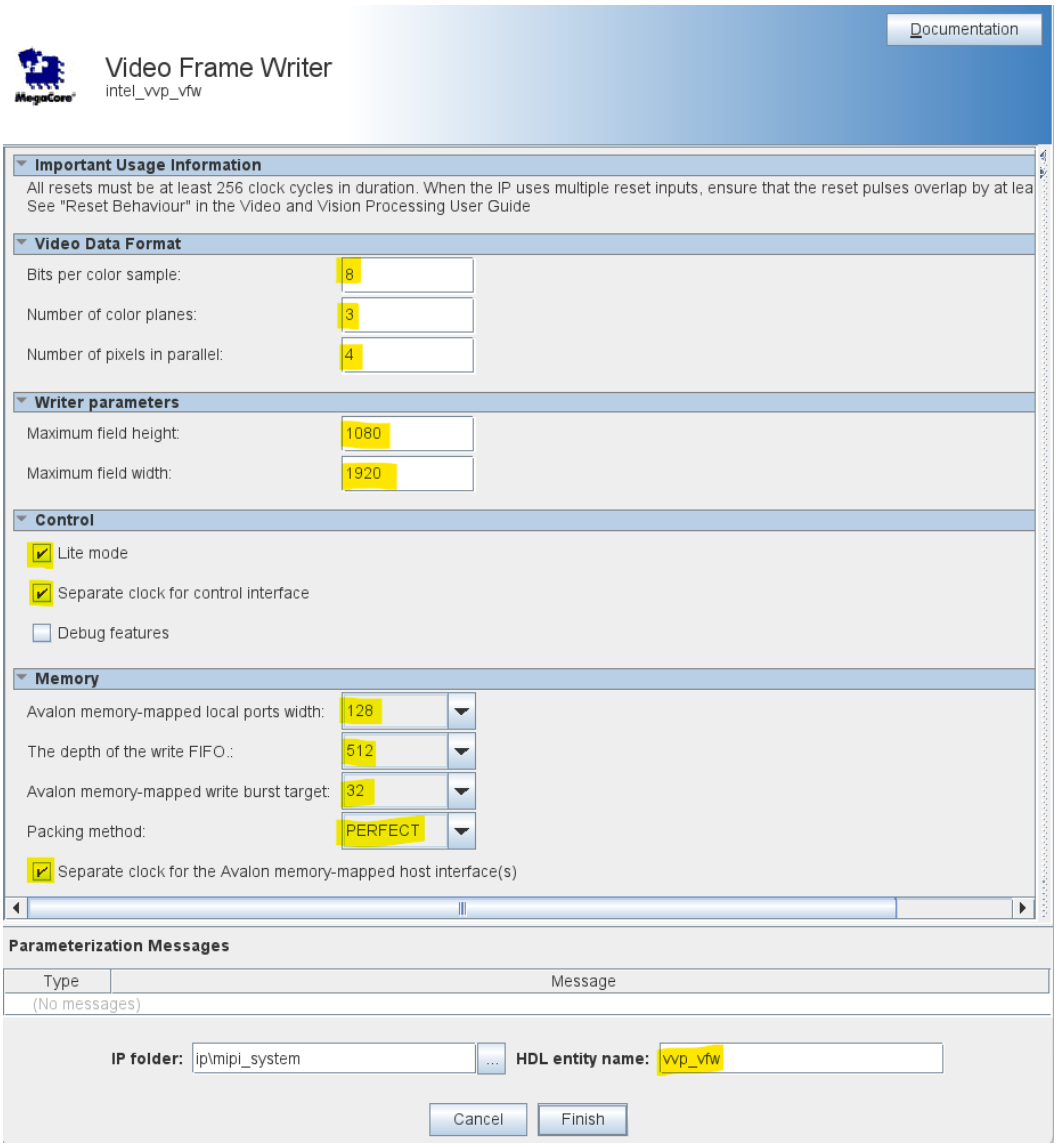


Figure 6-36: Add Video Frame Writer IP

Connect the ***main_clock*** to the ***sys_clock_bridge/out_clk***.

Connect the ***main_reset*** to the ***sys_reset_bridge/out_reset***.

Connect the ***mem_clock*** to the ***sys_clock_bridge/out_clk***.

Connect the ***mem_reset*** to the ***sys_reset_bridge/out_reset***.

Connect the ***control_clock*** to the ***sys_clock_bridge/out_clk***.

Connect the ***control_reset*** to the ***sys_reset_bridge/out_reset***.

Connect the ***av_mm_control_agent*** to ***vid_mm_bridge/m0***.

Connect the ***axi4s_vid_in*** to the ***vvp_scaler/axi4s_vid_out***.

Export the ***frame_witer_int*** interrupt signal to be connected to the NIOS-V processor later as **vvp_vfw_frame_writer_int**.

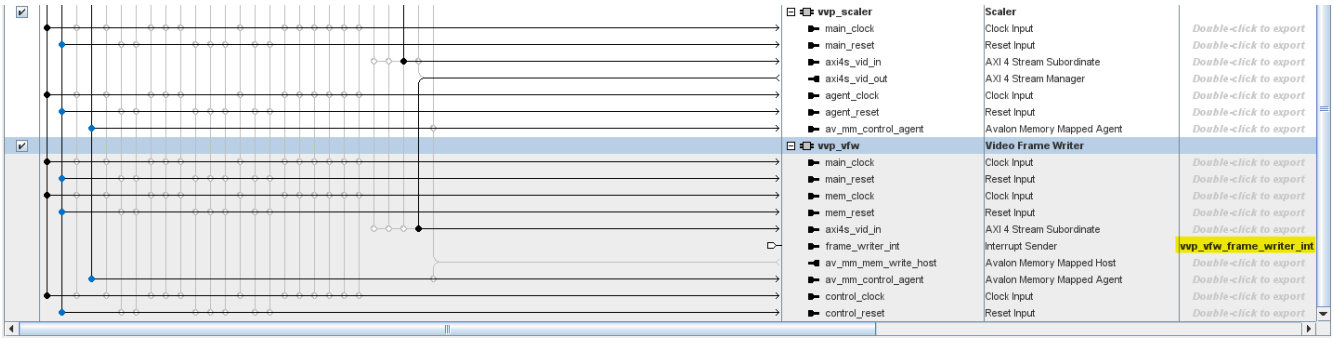


Figure 6-37: Video Frame Writer connections

6.3.2.9 Add Video Frame Buffer

The Video Frame buffer is created from M20K onchip SRAM blocks to hold 1 192000-byte frame (320-200x3). In order to maximize the M20K usage, the size will be set to 196608 bytes (12xM20K).

In the IP Catalog search field, filter on ***onchip***. Double-click on ***On-Chip Memory II (RAM or ROM) IP***.

Change the ***Total memory size*** to **196608** bytes.

Uncheck the box for ***Initialize memory content***.

Change the ***HDL entity name*** to **vid_frame_buf**. Click **Finish**.

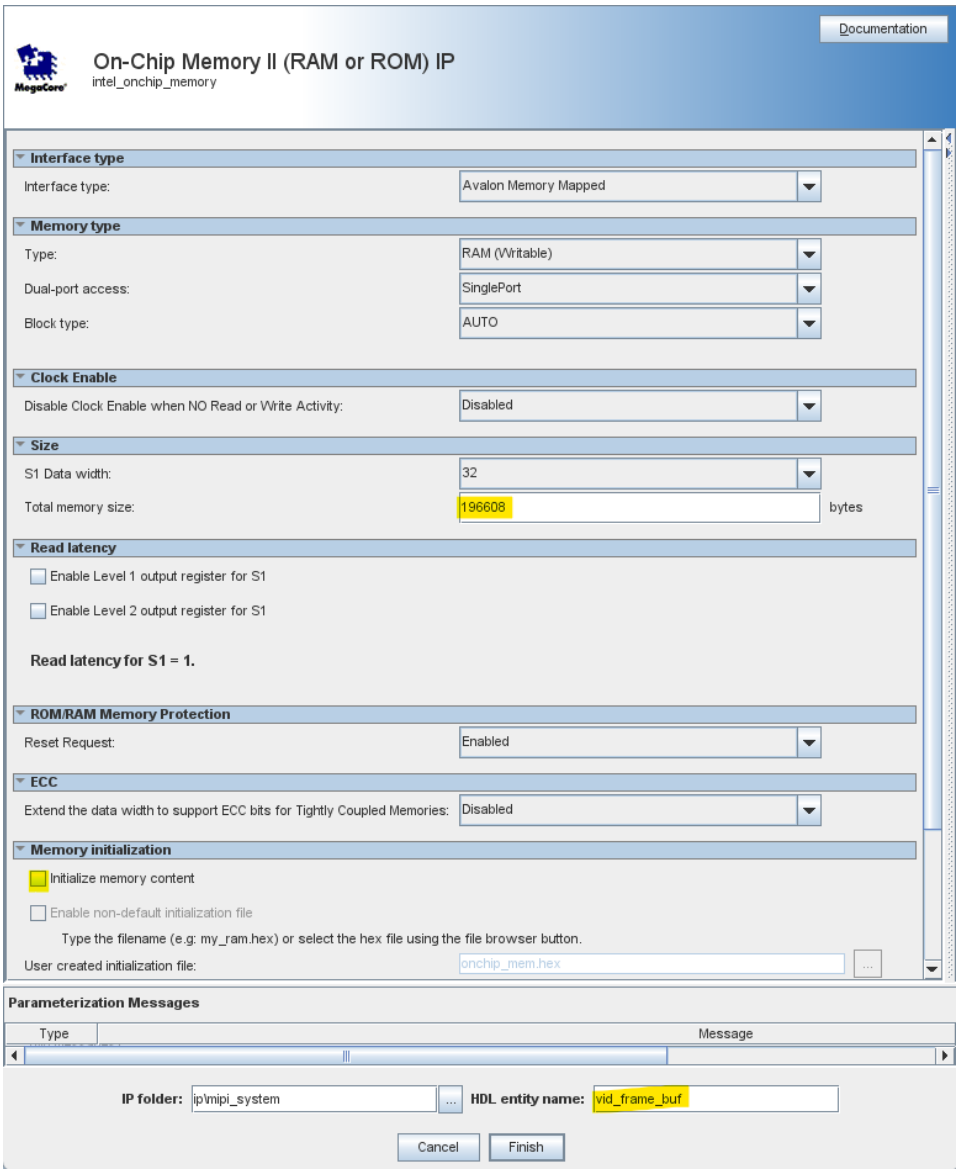


Figure 6-38: Add Onchip Memory IP for Video Frame Buffer

Connect its ***clk1*** input to the ***sys_clock_bridge/out_clk*** pin.

Connect its ***reset1*** input to the ***sys_reset_bridge/out_reset*** pin.

Connect its ***s1*** bus to both ***vvp_vfw/av_mm_mem_write_host*** and to ***vid_mm_bridge/m0*** busses.

6.3.2.10 Add JTAG Avalon Master IP

The JTAG Master allows for access to the Video Frame Buffer and mipi_system registers bus via the JTAG port driven by a debug tool called system Console.

In the IP Catalog search field, filer on ***JTAG***. Double-click on the ***JTAG to Avalon Master Bridge Intel FPGA IP***.

Change its ***HDL entity name*** to **mipi_jtag_master** and click **Finish**.

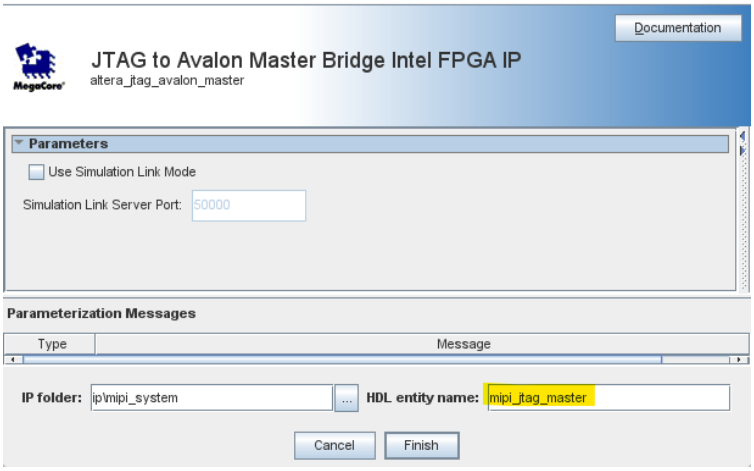


Figure 6-39: Add mipi_system JTAG Master IP

Connect its ***clk*** input to ***sys_clock_bridge/out_clk*** pin.

Connect its ***reset*** input to ***sys_reset_bridge/out_reset*** pin.

Connect its ***master*** bus to the ***vid_frame_buf/s1*** bus, and all ***av_mm_control_agent***, ***av_mm_control_interface*** and ***axi_lite*** busses.

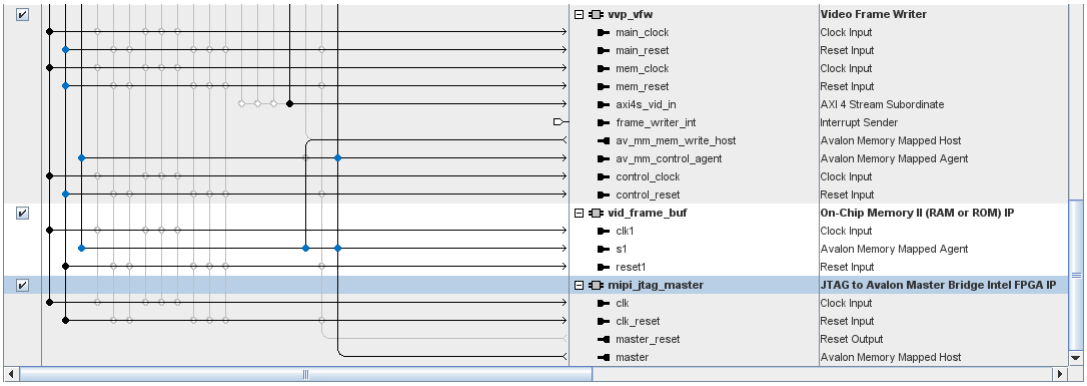


Figure 6-40: Add JTAG Avalon Master IP and connections

6.3.2.11 Assign localized base addresses

On the menu bar, select **System -> Assign Base Addresses**. This will automatically assign all peripheral addresses.

Your memory map may differ.

Subordinate	vid_mm_bridge.m0	vvp_vfw.av_mm_mem_write_host	mipi_jtag_master.master
mipi_dphy.axi_lite	**0x0004_0000** - 0x0004_0fff		**0x0004_0000** - 0x0004_0fff
vid_mm_bridge.s0			
mipi_csi2_rx.avalon_mm_control_interface	**0x0004_1000** - 0x0004_1fff		**0x0004_1000** - 0x0004_1fff
vvp_wbc.av_mm_control_agent	**0x0004_2600** - 0x0004_27ff		**0x0004_2600** - 0x0004_27ff
vvp_demosaic.av_mm_control_agent	**0x0004_2400** - 0x0004_25ff		**0x0004_2400** - 0x0004_25ff
vvp_scaler.av_mm_control_agent	**0x0004_2200** - 0x0004_23ff		**0x0004_2200** - 0x0004_23ff
vvp_vfw.av_mm_control_agent	**0x0004_2000** - 0x0004_21ff		**0x0004_2000** - 0x0004_21ff
vid_frame_buf.s1	**0x0000_0000** - 0x0002_ffff	**0x0000_0000** - 0x0002_ffff	**0x0000_0000** - 0x0002_ffff

Figure 6-41: mipi_system address map

6.3.2.12 Generate HDL

Now that the creation and connectivity are done, we need to extract HDL code from the IP blocks.

Click on the **Generate HDL** button in the lower right-hand corner, then click **Generate** in the pop-up window.

When prompted to ***Save changes***, click **Yes**.

Once the ***Generate*** is done without errors, click **Close**.

Exit the Platform Designer session.

6.3.3 Create NIOS-V system

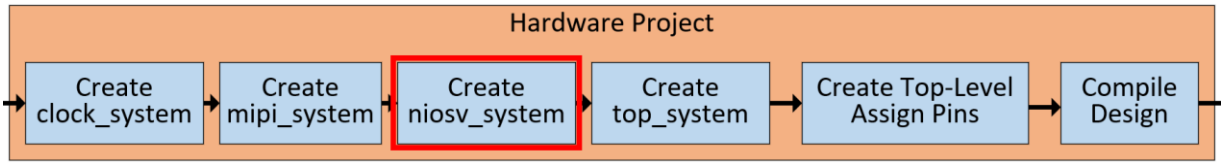


Figure 6-42: Create NIOS-V System

Open a new PD session. It can be opened by clicking the symbol on the Quartus menu bar or selecting **Tools -> Platform Designer** from the drop-down menus.

PD will ask you to either select an existing .qsys design or create a new one.

On the **Platform Designer system:** line, click ... to open an existing file, or to create a new one. If creating a new one, enter a name for it in the **File Name:** box and click **Create**.

Creating **niosv_system.qsys** for this step.

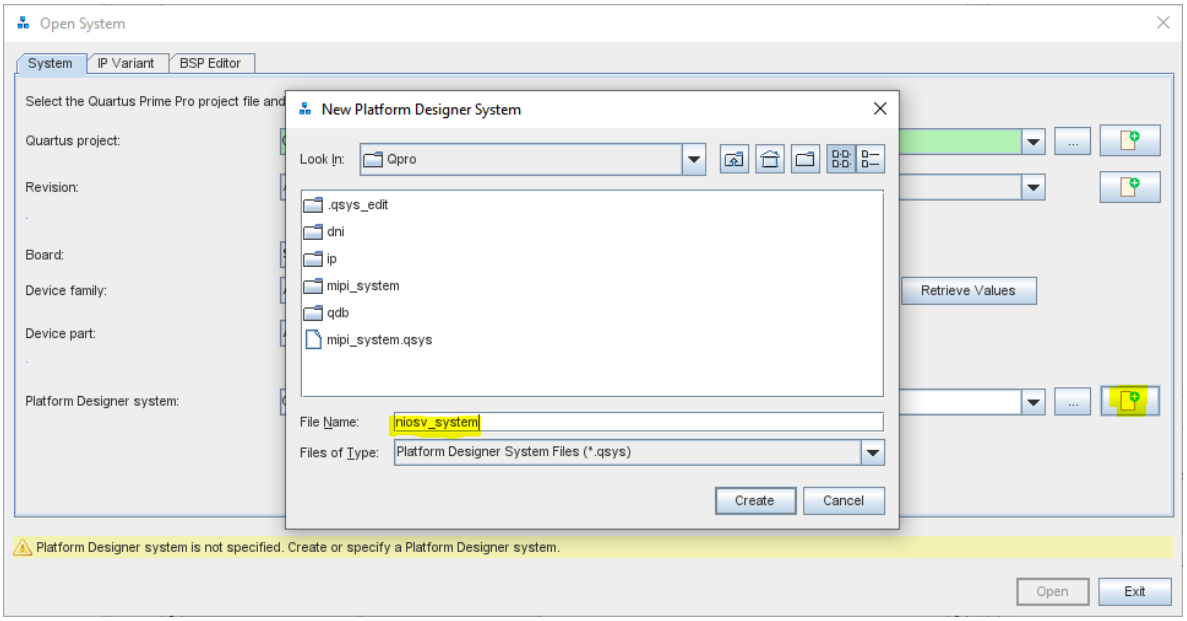


Figure 6-43: Create NIOS-V System

When the ***Create New System Completed*** window is done, click **Close**.

We now begin creating the NIOS-V system shown in the diagram below:

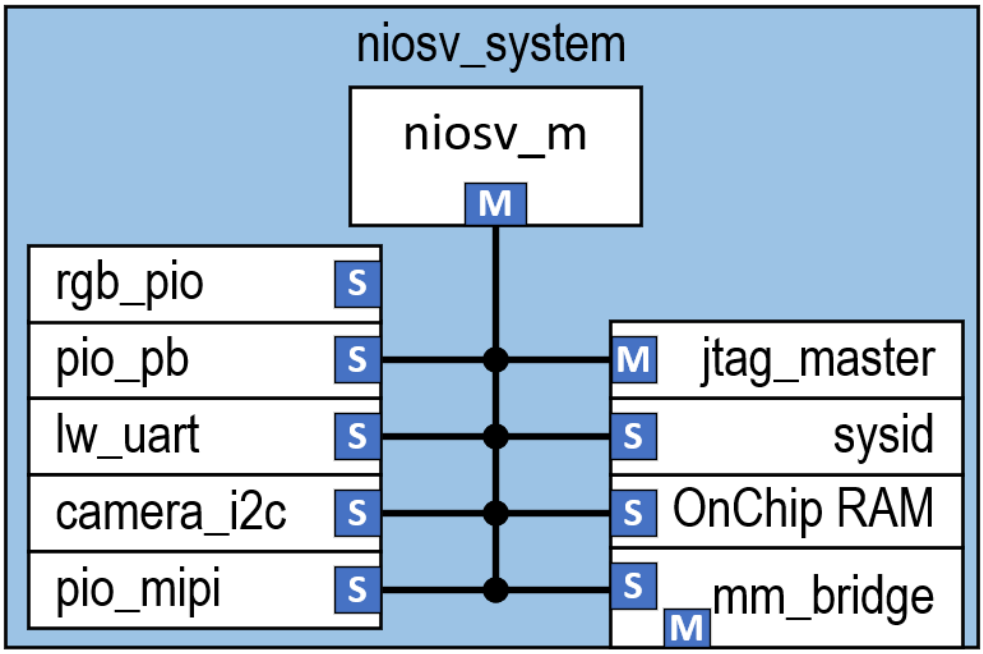


Figure 6-44: NIOS-V System Block Diagram

The PD **System View** comes up with a **clock_in** and **reset_in** bridges already inserted. This is the means for passing a clock and a reset into the PD subsystem.

Select the **clock_in** name and set its frequency to **12500000** (125MHz) in the parameters pane (on the right). This is the oscillator input frequency. Rename its exported input signal name ***clk*** to **sys_clk**.

Select the **reset_in** name and rename its exported input signal name ***reset*** to **sys_reset**.

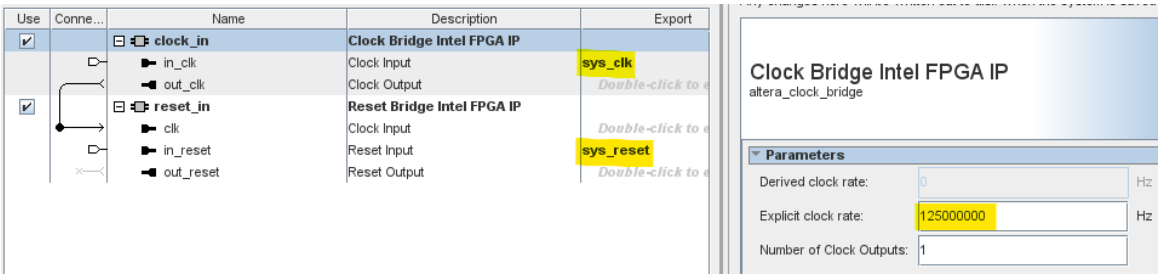


Figure 6-45: Clock and Reset bridge settings

6.3.3.1 Add Nios V /m IP

In the IP Catalog search field, filter on ***nios***.Double-click on the ***Nios V/m Microcontroller Intel FPGA IP***.

Keep all defaults.

Change its ***HDL entity name*** to **niosv_m**, then click **Finish**.

Connect its ***clk*** input to ***the clock_in/out_clk*** pin

Connect its ***reset*** input to ***reset_in/out_reset*** pin.

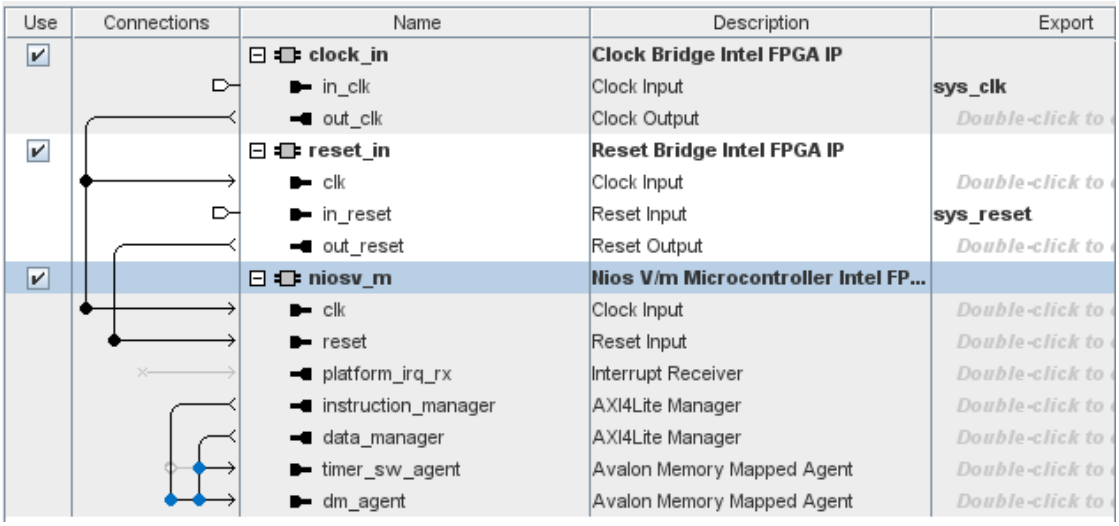


Figure 6-46: Add NIOS-V/m IP

The Reset vector will be set later, after the ***onchip_memory*** is added.

6.3.3.2 Add OnChip Memory IP

The onchip memory is internal M20K RAM which is used for the NIOSV to execute code from. We will create a 64KB block.

In the IP Catalog search field, filter on ***onchip***. Double-click on ***On-Chip Memory II (RAM or ROM) IP***.

Change the ***Total memory size*** to **65536** bytes.

Check the box for Initialize memory content.

Change the path for the User created initialization file to ***../software/mipi_app/build/Debug/niosv_system_0_onchip_memory.hex***. This file will be loaded into memory during the ***Assembler*** compile step.

Change the ***HDL entity name*** to **onchip_memory**. Click **Finish**.

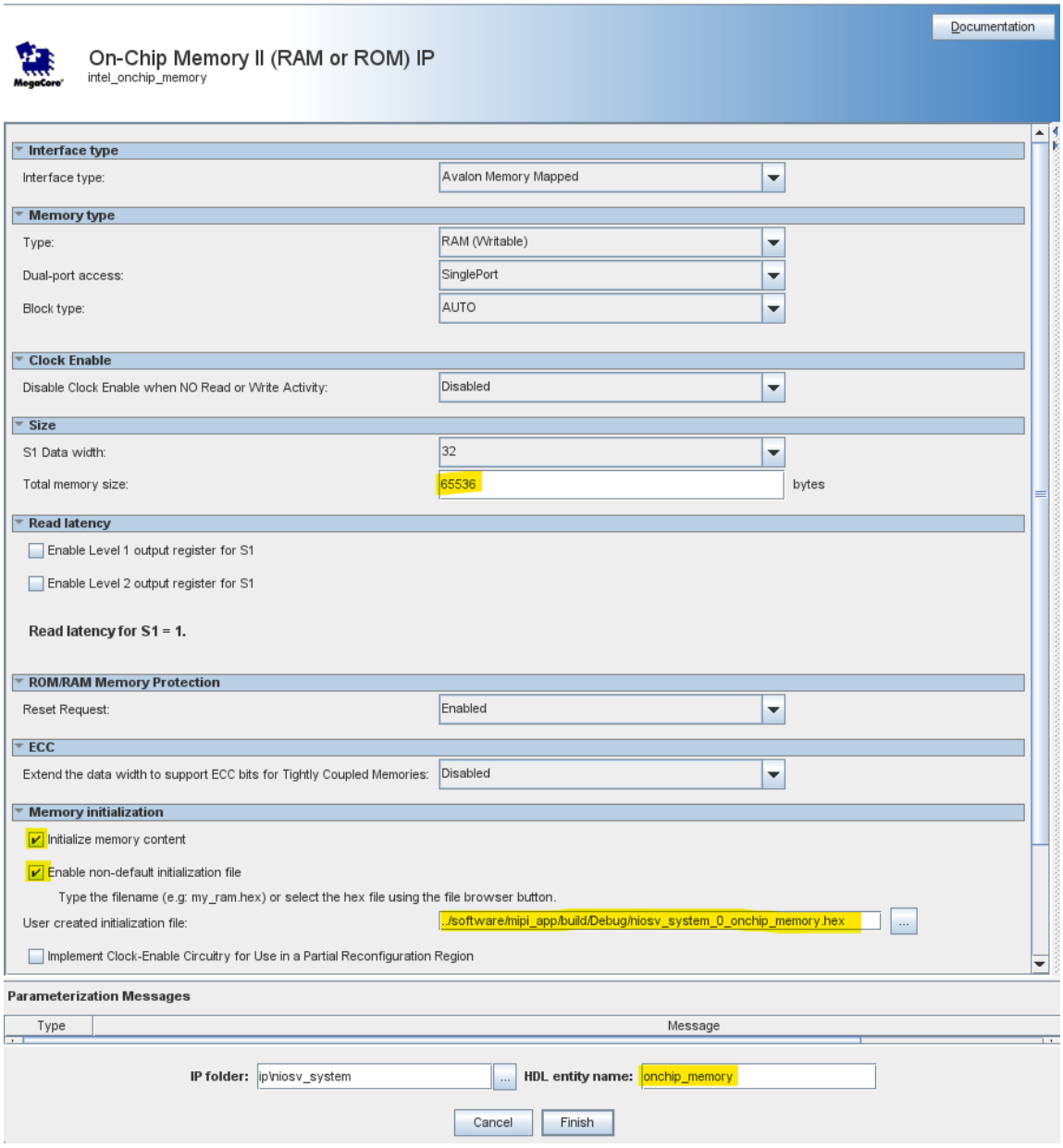


Figure 6-47: Add Onchip Memory IP

Connect its ***clk1*** input to the ***clock_in/out_clk*** pin.

Connect its ***reset1*** input to the ***reset_in/out_reset*** pin.

Connect its ***s1*** bus to both ***niosv_m/instruction_manager*** and ***niosv_m/data_manager*** buses. This allows for instructions to be executed out of this RAM, as well as data storage.

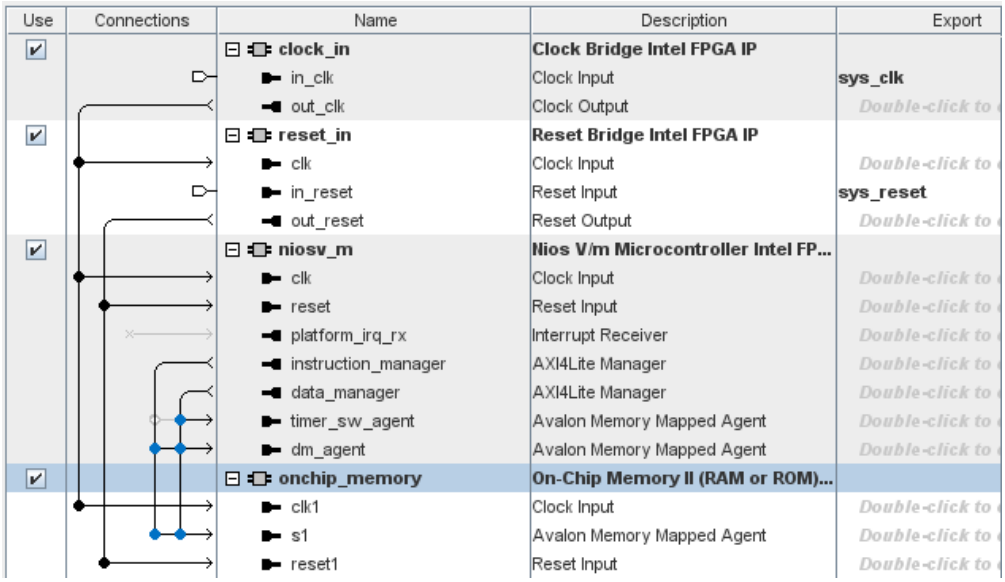


Figure 6-48: Onchip Memory connections

Notice that the ***onchip_memory*** has been assigned an address space that starts at **0x00000000**. **Lock it** by clicking the lock icon to the left of the number. This keeps it from being changed by the system. This is important because the Nios-V reset vector is set to address 0x00000000.

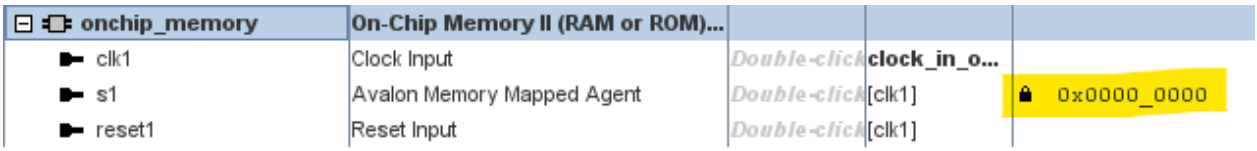


Figure 6-49: Lock the Onchip Memory start address

6.3.3.3 Add JTAG Avalon Master IP

The JTAG Master allows for access to the CPU bus via the JTAG port driven by a debug tool called system Console.

In the IP Catalog search field, filer on ***JTAG***. Double-click on the ***JTAG to Avalon Master Bridge Intel FPGA IP***.

Change its ***HDL entity name*** to **jtag_master** and click **Finish**.

Connect its ***clk*** input to ***clock_in/out_clk*** pin.

Connect its ***reset*** input to ***reset_in/out_reset*** pin.

Connect its ***master*** bus to the ***onchip_memory/s1*** bus.

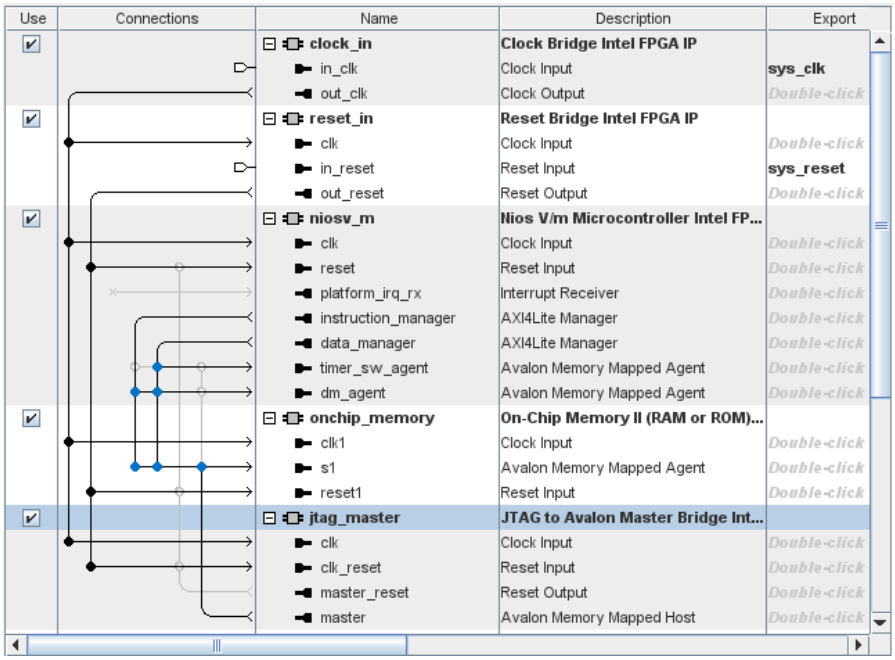


Figure 6-50: Add JTAG Avalon Master IP and connections

6.3.3.4 Add System ID IP

In the IP Catalog search field, filter on ***System ID***. Double-click on the ***System ID Peripheral IP*** and set its ***32-bit System ID*** to a Hex value of **0xAC300025**. This is an arbitrary number that you recognize as belonging to this design. Later when running the demo, this value will be read and displayed.

Change the ***HDL entity name*** to **sysid**. Click **Finish**.

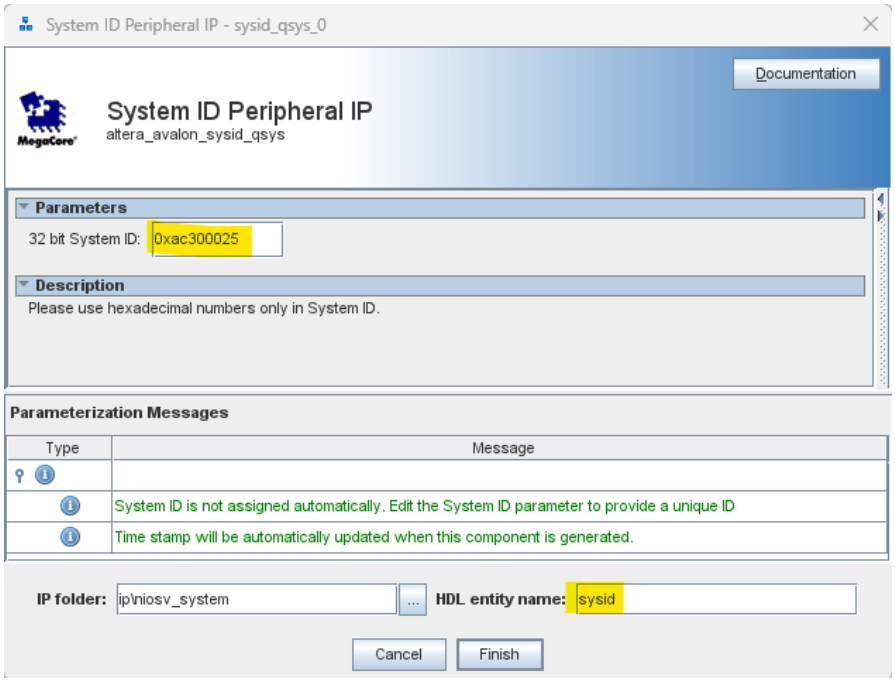


Figure 6-51: Add System ID IP

Connect its ***clk*** input to ***clock_in/out_clk*** pin.

Connect its ***reset*** input to ***reset_in/out_reset*** pin.

Connect its ***control_slave*** bus to the ***niosv_m/data_manager*** and ***jtag_master/master*** busses.

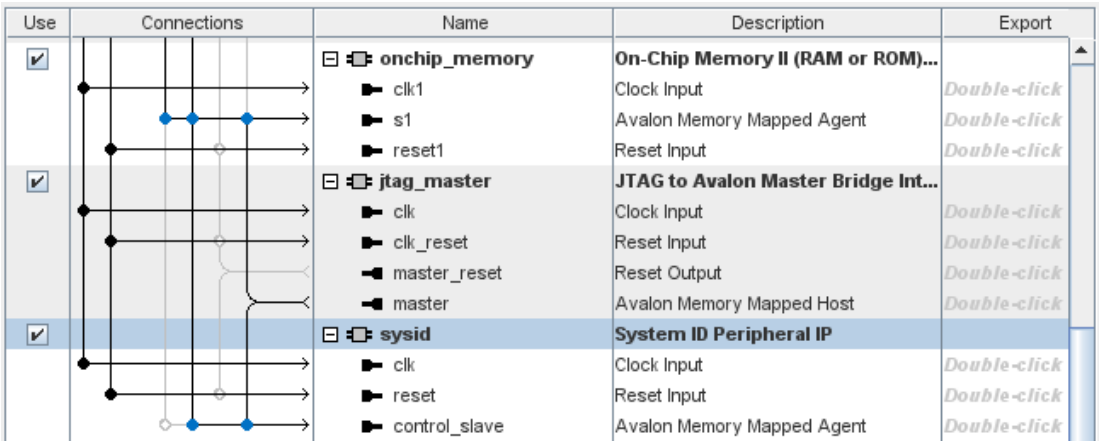


Figure 6-52: System ID connections

6.3.3.5 Add I2C Master IP

In the IP Catalog search field, filter on ***i2c***. Double-click on the ***Avalon I2C (Master) IP***.

Change its ***HDL entity name*** to **camera_i2c** and click **Finish**.

Connect its ***clock*** input to ***clock_in/out_clk*** pin.

Connect its ***reset_sink*** input to ***reset_in/out_reset*** pin.

Connect its ***interrupt_sender*** output to the ***niosv_m/platform_irq_rx*** input.

Connect its ***csr*** bus to the ***niosv_m/data_manager*** and ***jtag_master/master*** busses.

Export its ***i2c_serial*** conduit as **camera_i2c**.

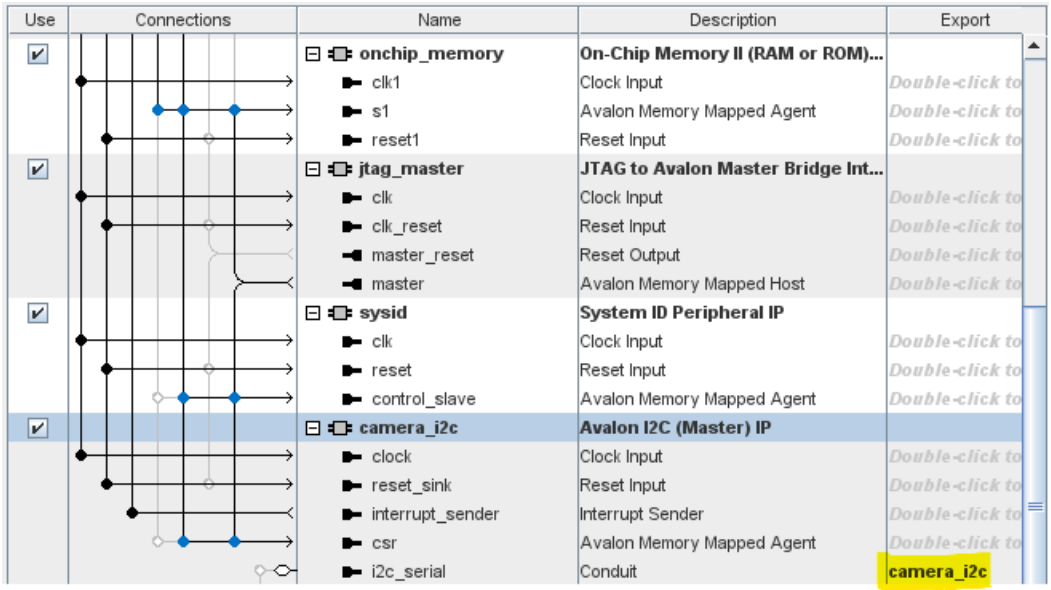


Figure 6-53: Add I2C IP and connections

6.3.3.6 Add PIO IP for LED

The design uses a 3-bit RGB LED with an active-low energizing.

In the IP Catalog search field, filter on ***pio***. Double-click on the ***PIO (Parallel I/O) IP***.

Set its ***Width*** to **3**, the ***Direction*** to **Output**, and the ***Output Port Reset Value*** to **0x0007** (LEDs off).

Change the ***HDL entity name*** to **pio_led** and click **Finish**.

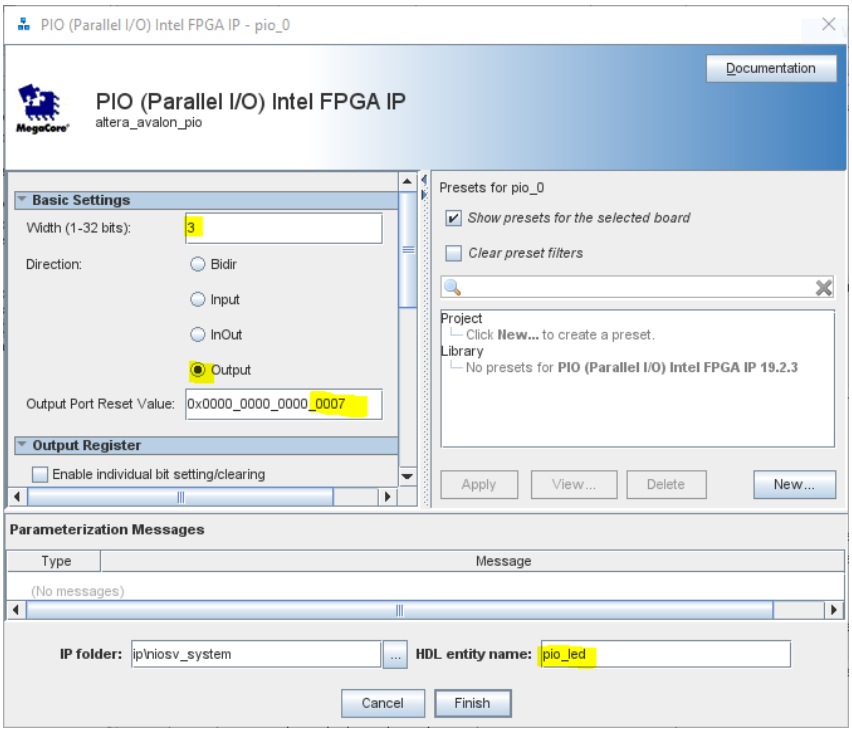


Figure 6-54: Add 3-bit PIO for RGB LED

Connect its ***clk*** input to ***clock_in/out_clk*** pin.

Connect its ***reset*** input to ***reset_in/out_reset*** pin.

Connect its **s1** bus to the ***niosv_m/data_manager*** and ***jtag_master/master*** busses.

Export its ***external_connection*** conduit as ***pio_led***.

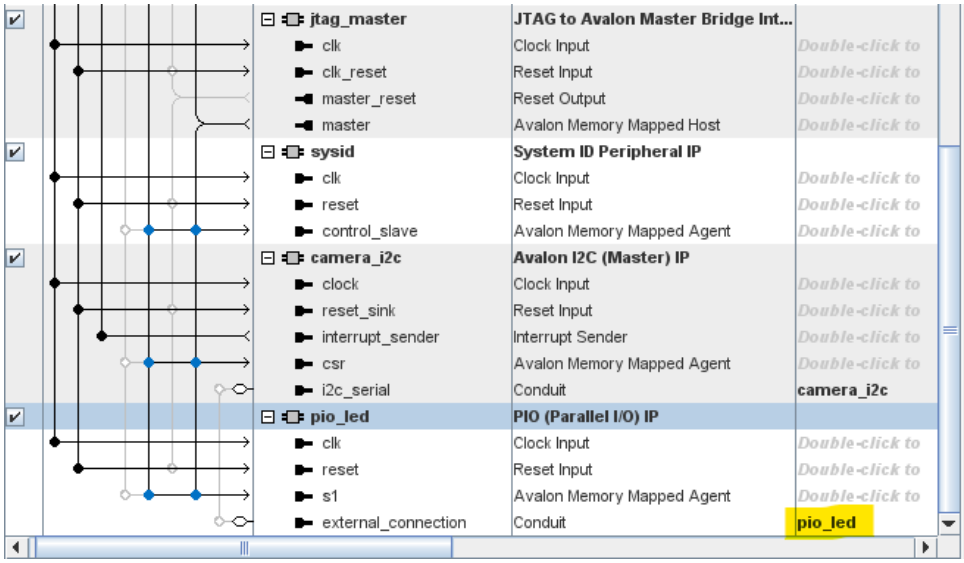


Figure 6-55: LED PIO connections

6.3.3.7 Add PIO IP for MIPI control

The design uses a 1-bit output PIO to control the camera Enable pin (active-high).

In the IP Catalog search field, filter on ***pio***. Double-click on the ***PIO (Parallel I/O) IP***.

Set its ***Width*** to **1**, the ***Direction*** to **Output**, and the ***Output Port Reset Value*** to **0x0000** (Camera off – bit 0=0).

Change the ***HDL entity name*** to **pio_mipi** and click **Finish**.

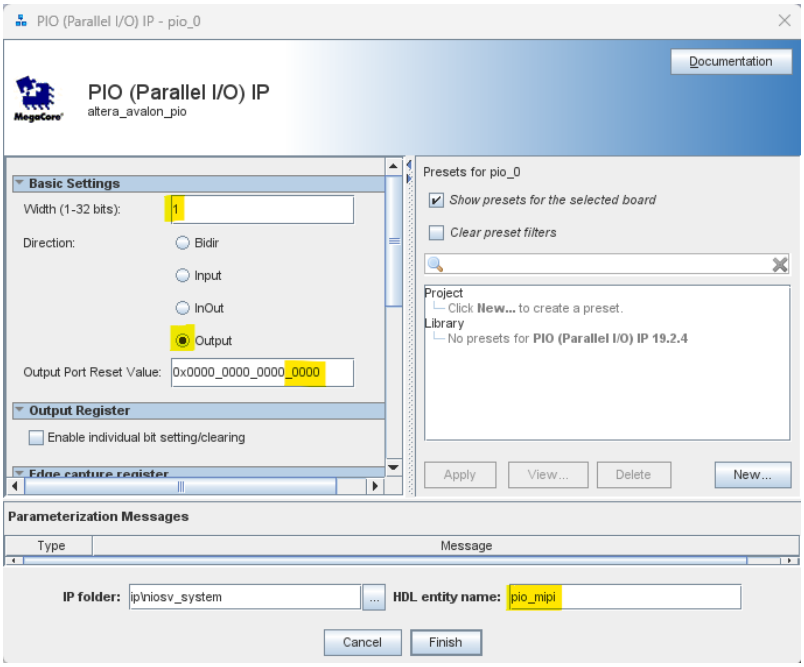


Figure 6-56: Add 2-bit PIO for Video subsystem

Connect its ***clk*** input to ***clock_in/out_clk*** pin.

Connect its ***reset*** input to ***reset_in/out_reset*** pin.

Connect its ***s1*** bus to the ***niosv_m/data_manager*** and ***jtag_master/master*** busses.

Export its ***external_connection*** conduit as ***pio_mipi***.

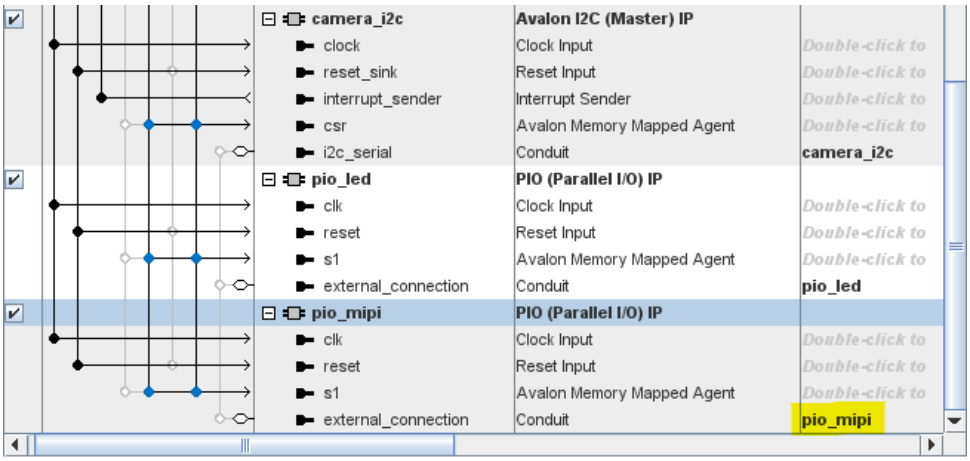


Figure 6-57: MIPI PIO connections

6.3.3.8 Add PIO IP for Push-Button

The design uses an active-low push-button for interacting with the software.

In the IP Catalog search field, filter on ***pio***. Double-click on the ***PIO (Parallel I/O) IP***.

Set its ***Width*** to **1**, the ***Direction*** to **Input**, and **check** the box for ***Synchronous capture***.

Select **FALLING** for ***Edge Type*** and **check** the box for ***Enable bit-clearing for edge capture register.***

Check the box for ***Generate IRQ*** and select **EDGE** for the ***IRQ Type***.

Change the ***HDL entity name*** to **pio_pb** and click **Finish**.

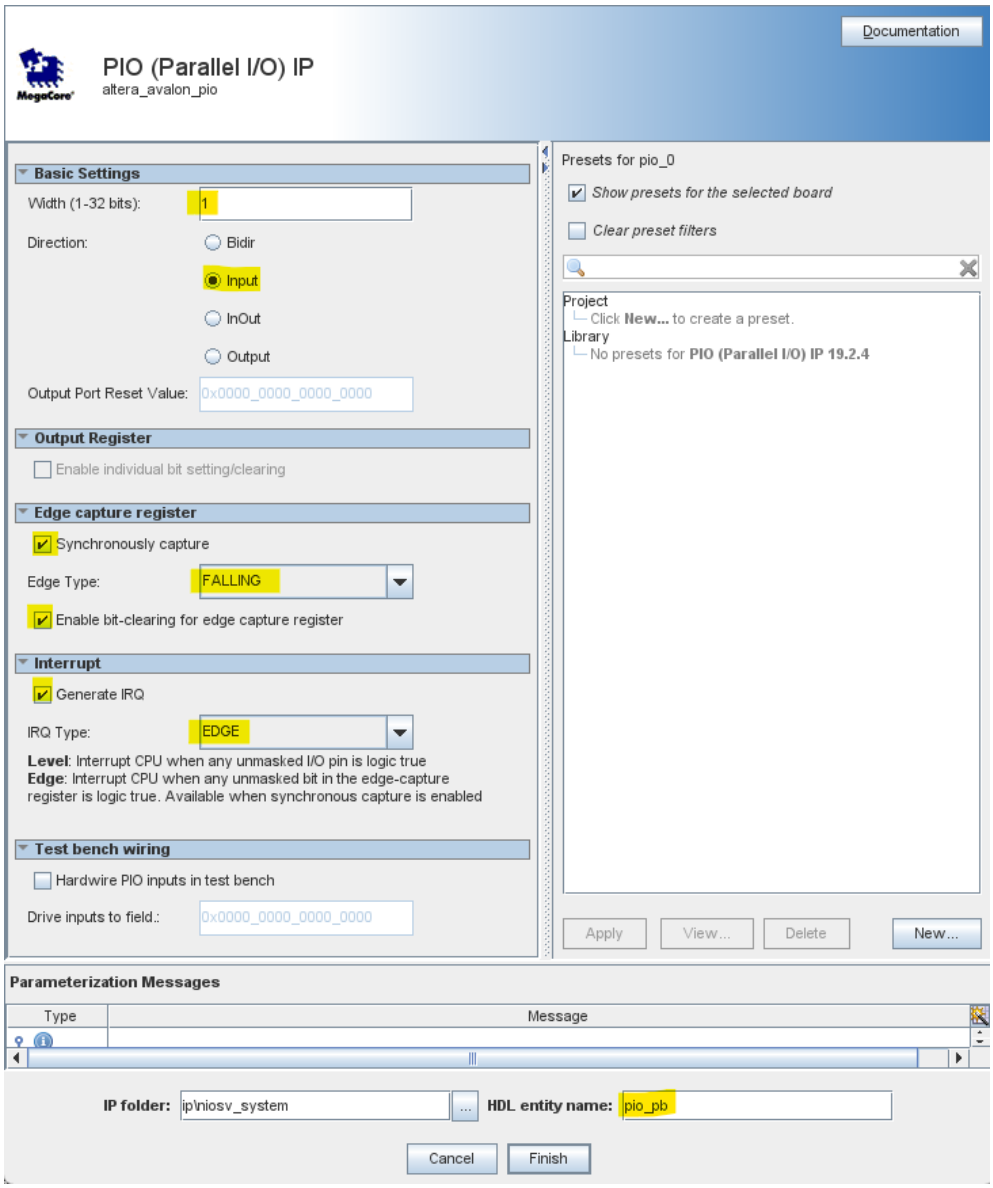


Figure 6-58: Add 1-bit PIO for push-button

Connect its ***clk*** input to ***clock_in/out_clk*** pin.

Connect its ***reset*** input to ***reset_in/out_reset*** pin.

Connect its **s1** bus to the ***niosv_m/data_manager*** and ***jtag_master/master*** busses.

Connect its ***irq*** pin to the ***niosv_m/platform_irq_rx*** pin.

Export its ***external_connection*** conduit as ***pio_pb***.

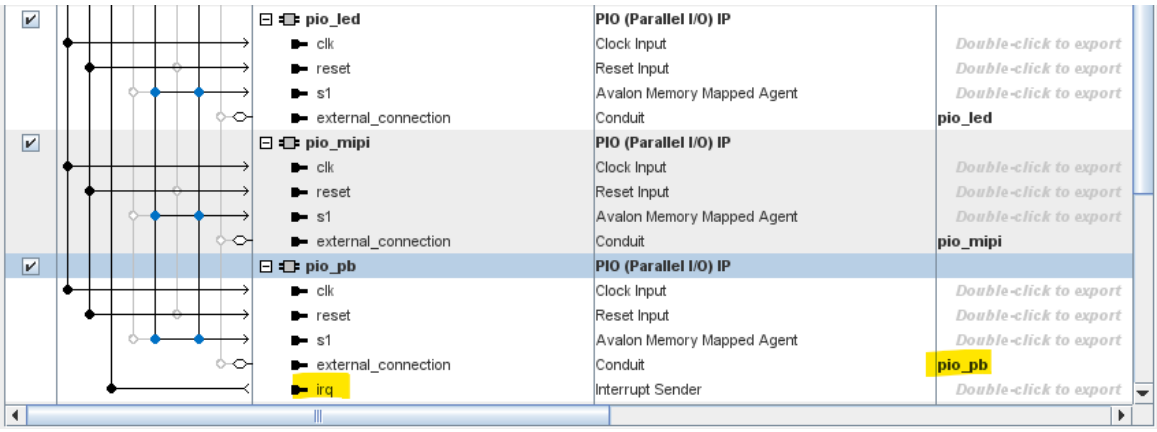


Figure 6-59: Push-Button PIO connections

6.3.3.9 Add Light Weight UART IP

The design uses a UART at 115200 baud to communicate status to and accept commands from the User.

In the IP Catalog search field, filter on **uart**. Double-click on the ***Lightweight UART (RS-232 Serial port)*** IP.

Keep all its ***default*** settings, making sure the ***Baud rate*** is set to **115200**.

Change the ***HDL entity name*** to **lw_uart** and click **Finish**.

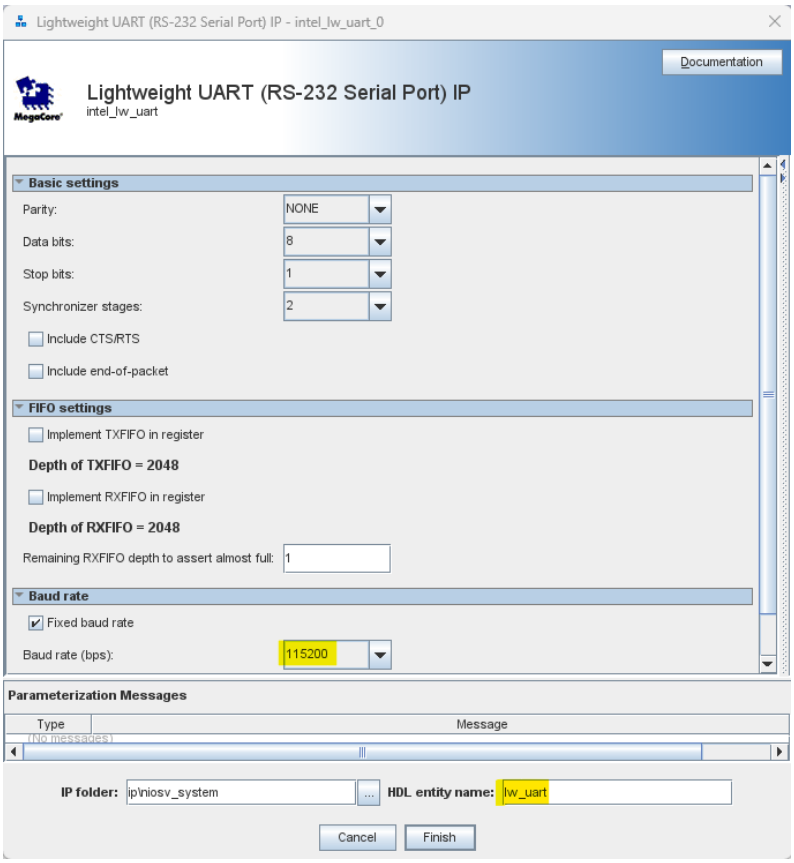


Figure 6-60: Add Lightweight UART IP

Connect its ***clk*** input to ***clock_in/out_clk*** pin.

Connect its ***reset*** input to ***reset_in/out_reset*** pin.

Connect its ***s1*** bus to the ***niosv_m/data_manager*** and ***jtag_master/master*** busses.

Connect its ***irq*** to the ***niosv_m/platform_irq_rx*** pin. Software ***stdio*** relies on it being connected.

Export its ***external_connection*** conduit as ***dbg_uart***.

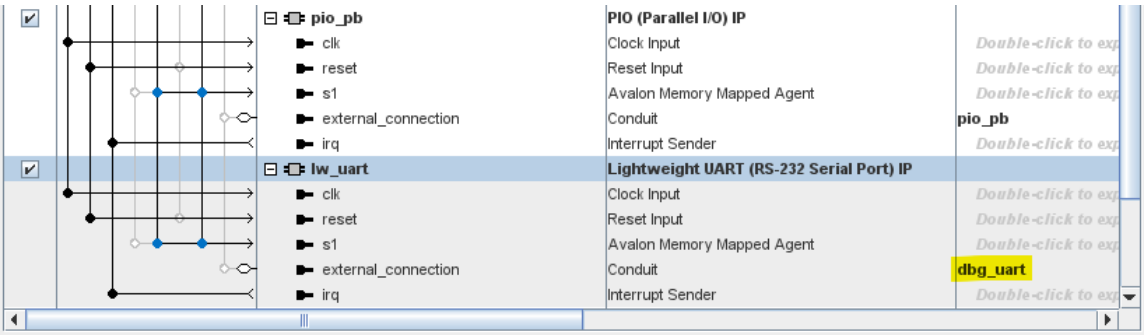


Figure 6-61: UART connections

6.3.3.10 Add memory-mapped Avalon Bridge IP

The NIOS-V needs to access memory-mapped registers in the ***mipi_system***. In order for the NIOS-V bus to transfer outside of this system, it needs to be carried over a memory-mapped Avalon bridge.

In the IP Catalog search field, filter on ***bridge***. Double-click on the ***Avalon Memory Mapped Pipeline Bridge Intel FPGA IP***.

Keep all its default settings except the Address width. For that, **set** the ***Address width*** to **19**. This setting allows full access to the ***mipi_system_0*** address space which occupies 0x0000_0000-0x0007_FFFF address range.

Change the ***HDL entity name*** to **niosv_mm_bridge** and click **Finish**.

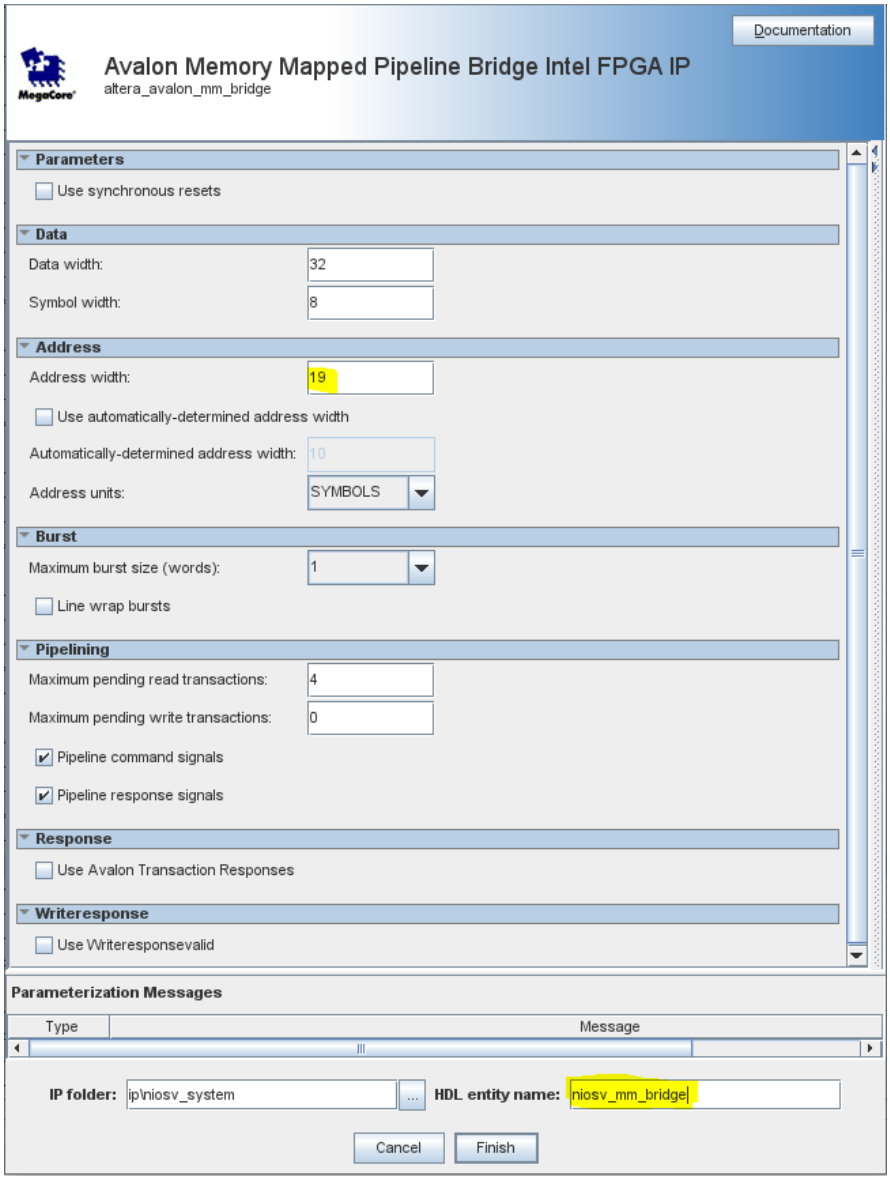


Figure 6-62: Add memory-mapped bridge

Connect its ***clk*** input to ***clock_in/out_clk*** pin.

Connect its ***reset*** input to ***reset_in/out_reset*** pin.

Connect its ***s0*** bus to the ***niosv_m/data_manager*** and ***jtag_master/master*** busses.

Export its ***m0*** Avalon Host bus as **niosv_mm_m0**.

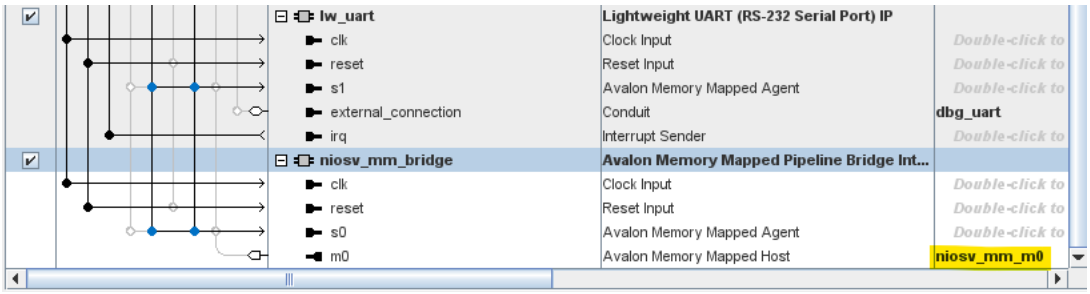


Figure 6-63: Memory-mapped bridge connections

6.3.3.11 Add IRQ Bridge IP

The NIOS-V interrupt pin has a few sources inside the ***niosv_system_0*** driving it. The ***Video Frame Writer*** from the ***mipi_system_0*** needs to also connect to it. In this case, we add an IRQ Bridge IP.

In the IP Catalog search field, filter on ***bridge***. Double-click on the ***IRQ Bridge Intel FPGA IP***.

Set the ***IRQ signal width*** to **1**.

Change the ***HDL entity name*** to **irq_bridge** and click **Finish**.

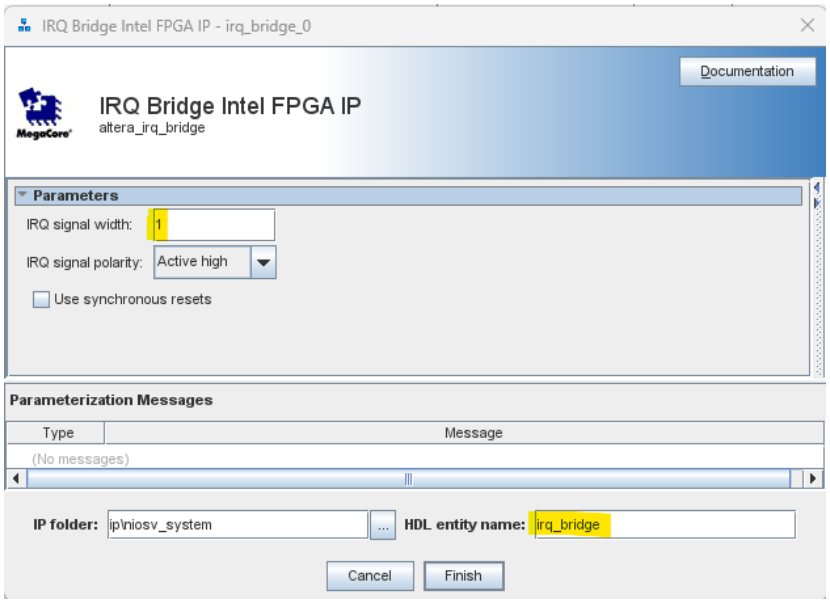


Figure 6-64: Add IRQ Bridge IP

Connect its ***clk*** input to ***clock_in/out_clk*** pin.

Connect its ***clk_reset*** input to ***reset_in/out_reset*** pin.

Connect its ***sender0_irq*** to the ***niosv_m/platform_irq_rx*** pin.

Export its ***receiver_irq*** pin as ***niosv_irq***.

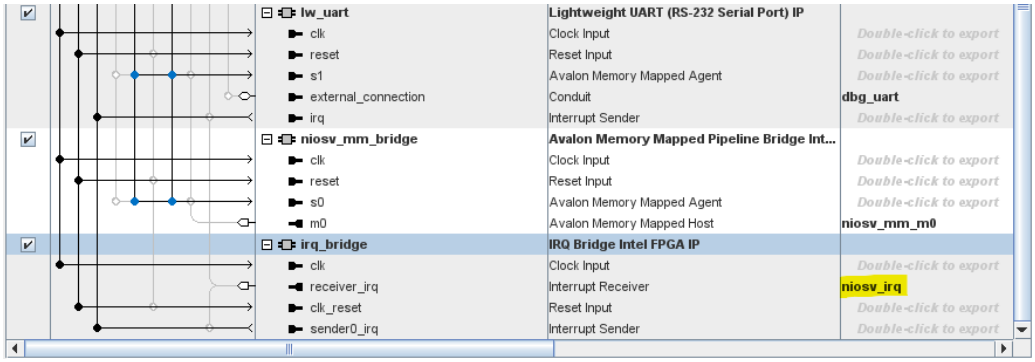


Figure 6-65: IRQ bridge connections

6.3.3.12 Assign localized base addresses

On the menu bar, select **System -> Assign Base Addresses**. This will automatically assign all peripheral addresses.

Your memory map may be different.

Subordinate	jtag_master.master	niosv_m.instruction_manager	niosv_m.data_manager
camera_i2c.csr	**0x0001_0000** - 0x0001_003f		**0x0001_0000** - 0x0001_003f
lw_uart.s1	**0x0001_0040** - 0x0001_005f		**0x0001_0040** - 0x0001_005f
niosv_m.timer_sw_agent			**0x0001_00c0** - 0x0001_00ff
niosv_m.dm_agent		**0x0002_0000** - 0x0002_ffff	**0x0002_0000** - 0x0002_ffff
niosv_mm_bridge.s0	**0x0008_0000** - 0x000f_ffff		**0x0008_0000** - 0x000f_ffff
onchip_memory.s1	0x0000_0000 - 0x0000_ffff	0x0000_0000 - 0x0000_ffff	0x0000_0000 - 0x0000_ffff
pio_led.s1	**0x0001_0080** - 0x0001_008f		**0x0001_0080** - 0x0001_008f
pio_mipi.s1	**0x0001_0070** - 0x0001_007f		**0x0001_0070** - 0x0001_007f
pio_pb.s1	**0x0001_0060** - 0x0001_006f		**0x0001_0060** - 0x0001_006f
sysid.control_slave	**0x0001_0090** - 0x0001_0097		**0x0001_0090** - 0x0001_0097

Figure 6-66: niosv_system base addresses

Click on the **Generate HDL** button in the lower right-hand corner, then click **Generate** in the pop-up window.

When prompted to ***Save changes***, click **Yes**.

Once the ***Generate*** is done without errors, click **Close**.

Exit the Platform Designer session.

6.4 Create top-level system

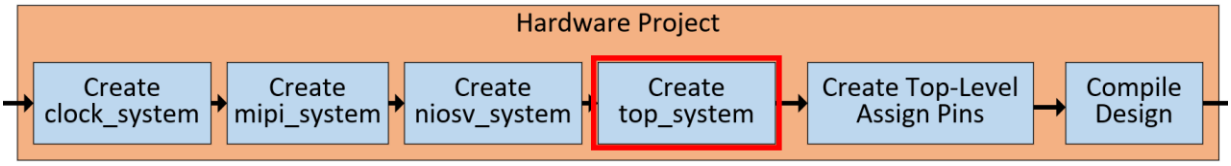


Figure 6-67: Create top-level System (top_system)

Open a new PD session. It can be opened by clicking the symbol on the Quartus menu bar or selecting **Tools -> Platform Designer** from the drop-down menus.

PD will ask you to either select an existing .qsys design or create a new one.

On the **Platform Designer system:** line, click ... to open an existing file, or to create a new one. If creating a new one, enter the **top_system** name for it in the **File Name:** box and click **Create** in both windows.

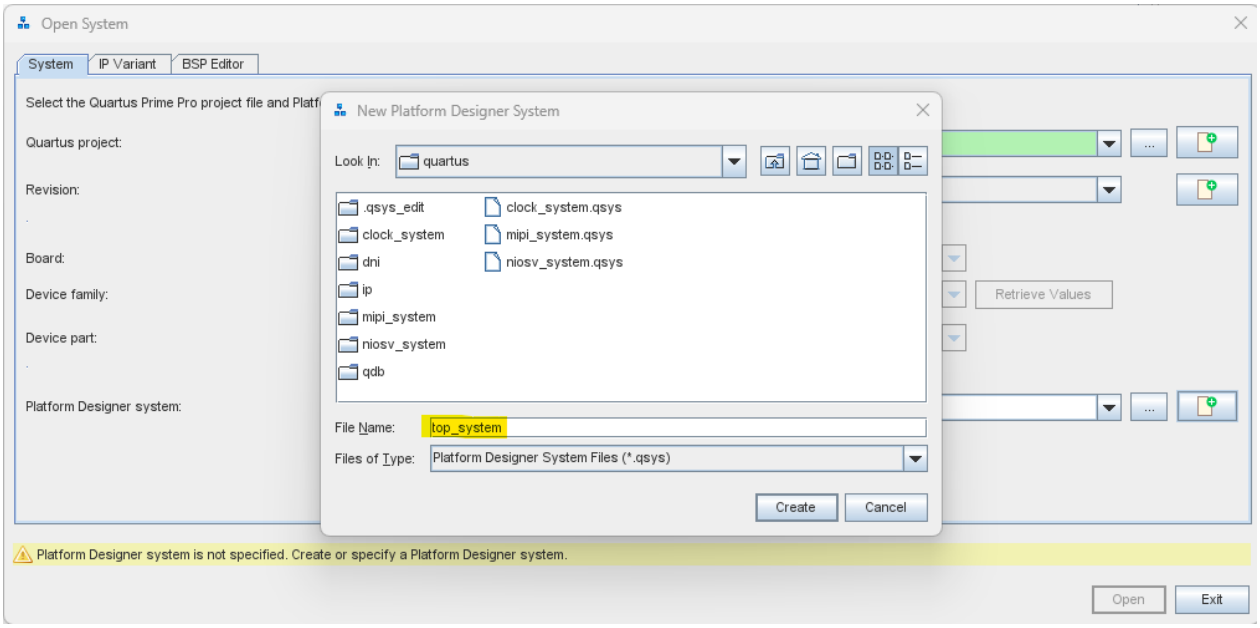


Figure 6-68: Create top_system in PD

6.4.1 Add All sub-system

In the IP Catalog region, under ***Project → Systems*** (expand Systems), you will see the 3 previously created subsystems, ***clock_system***, ***mipi_system***, and ***niosv_system*** listed.

Note: the search field must be empty.

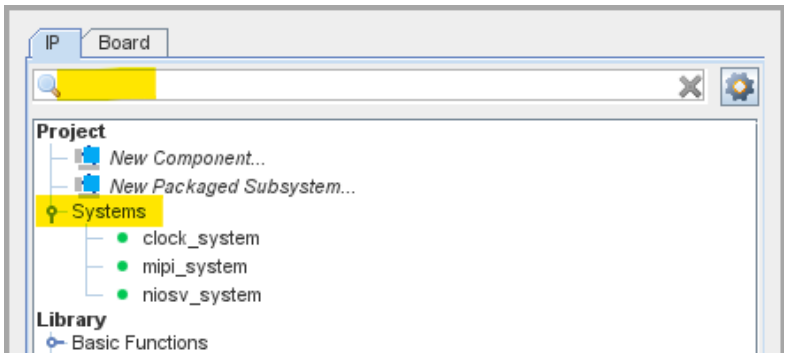


Figure 6-69: Expose all sub-systems

Double-click all 3 to add them to the ***top_system***.

Select the ***clock_in*** name and set its frequency to **25000000** in the parameters pane (on the right). **Double-Click** its Clock Input signal name (in the Export column) and rename it from ***clk*** to ***clk25mhz***. This is the external oscillator clock frequency.

Right-Click the ***reset_in*** name and **select *Remove***, then **click *Yes*** to confirm. There is no external reset pin in this design.

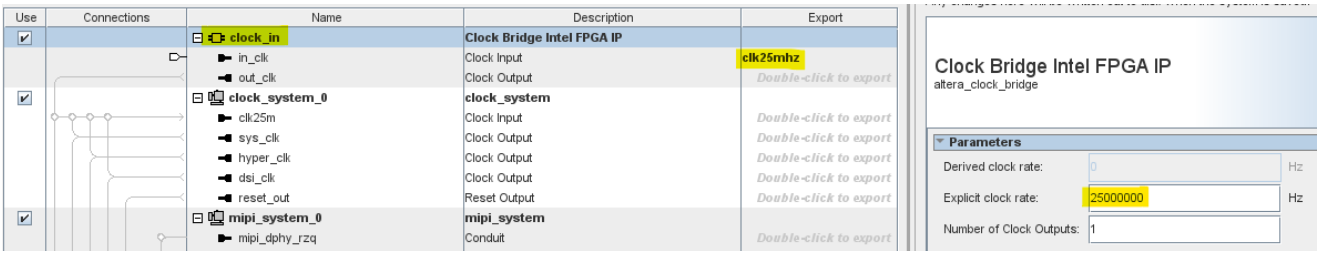


Figure 6-70: Edit the External Clock Input parameters

Connect the ***clock_in/out_clk*** pin to ***clock_system_0/clk25m*** pin.

Connect the ***clock_system_0/reset_out*** pin to ***mipi_system_0/sys_rst_in*** and ***niosv_system_0/sys_reset*** pins.

Connect the ***clock_system_0/sys_clk*** pin to ***mipi_system_0/sys_clk_in*** and ***niosv_system_0/sys_clk*** pins.

Connect ***niosv_system_0/niosv_mm_m0*** bus to ***mipi_system_0/vid_mm_bus*** bus.

Connect ***niosv_system_0/niosv_irq*** pin to ***mipi_system_0/vvp_vfw_frame_writer_int*** pin.

Export ***clock_in/in_clk*** as **clk25mhz**.

Export ***mipi_system_0/mipi_dphy_rzq*** as ***mipi_dphy_rzq***.

Export ***mipi_system_0/mipi_dphy_refclk*** as ***mipi_dphy_refclk***.

Export ***mipi_system_0/mipi_csi2_io*** bus as ***mipi_csi2_io***.

Export ***niosv_system_0/camera_i2c*** as ***camera_i2c***.

Export ***niosv_system_0/dbg_uart*** as ***dbg_uart***.

Export ***niosv_system_0/pio_led*** as ***led***.

Export ***niosv_system_0/pio_mipi*** as ***camera_en***.

Export ***niosv_system_0/pio_pb*** as ***pb***.

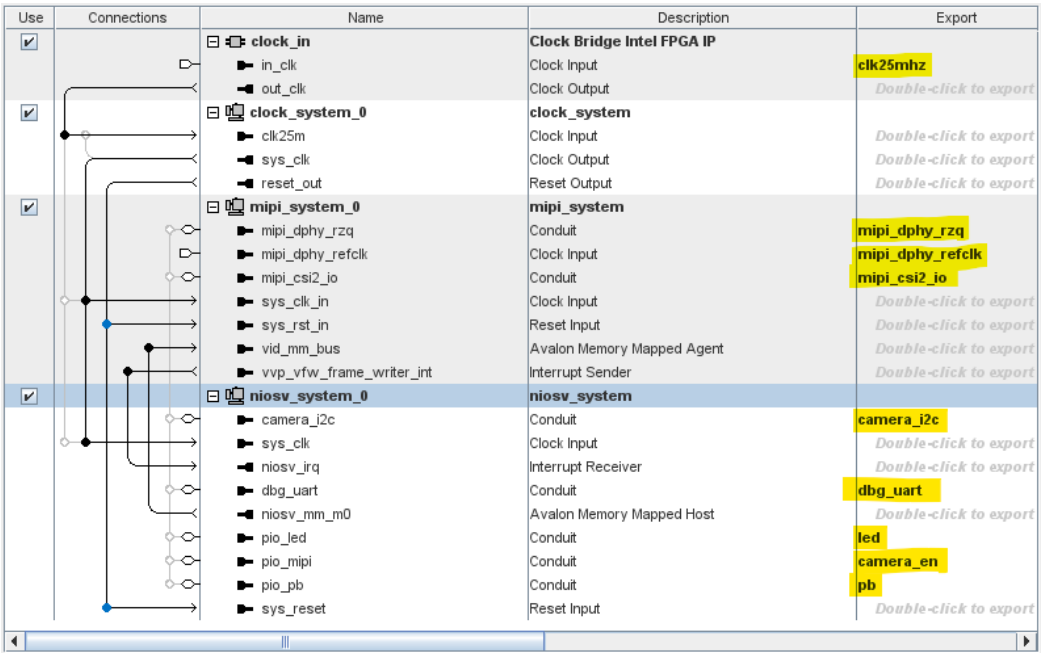


Figure 6-71: top_system connections

6.4.2 Generate HDL

Now that all IP blocks have been added, configured, and their base addresses assigned, we need to compile (Generate HDL) the PD design.

In the lower right-hand corner of the PD window, click on the ***Generate HDL*** button, then ***Generate*** in the pop-up window.

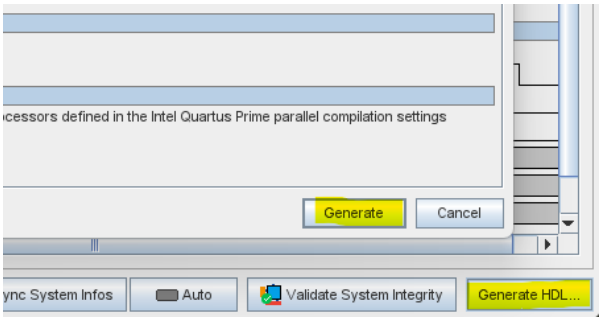
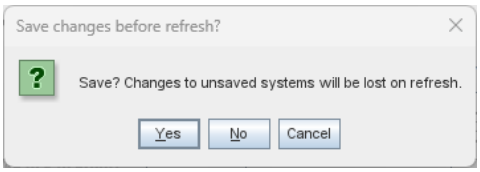


Figure 6-72: Generate HDL in PD

You may be asked to save changes, click **Yes**.



When the ***Generate Completed*** window is done processing, with no errors, click **Close**.

The ***top_system.qsys*** may need to be added to the project files.

From drop-down menu ***Project -> Add/Remove Files in Project…***, click the "**…**" on the File name line to add ***top_system.qsys***.

Double-click ***top_system.qsys***, then click **OK**.

6.4.3 Create the instantiation template

Platform Designer can generate an instantiation template for the ***top_system*** in either Verilog or VHDL. This step is done for you in the top-level Verilog file. If you are starting from a raw top-level, you need this template.

On the menu bar, Click ***Generate -> Show Instantiation Template…***

Copy it and **paste** it into the top-level file. This step is already done for you.

6.4.4 Software consideration

When you start the Software Project later, you will need to return to the Platform Designer to generate a Board Support Package (BSP).

You can perform the BSP generation step while you are still in Platform Designer ***top_system***.

In Platform Designer, **Click *File-> New BSP…***

In the ***New Platform Designer BSP*** window, navigate to the application folder ***../software/mipi_bsp*** and enter the ***File Name*** of **Settings.bsp**, then click **Create**.

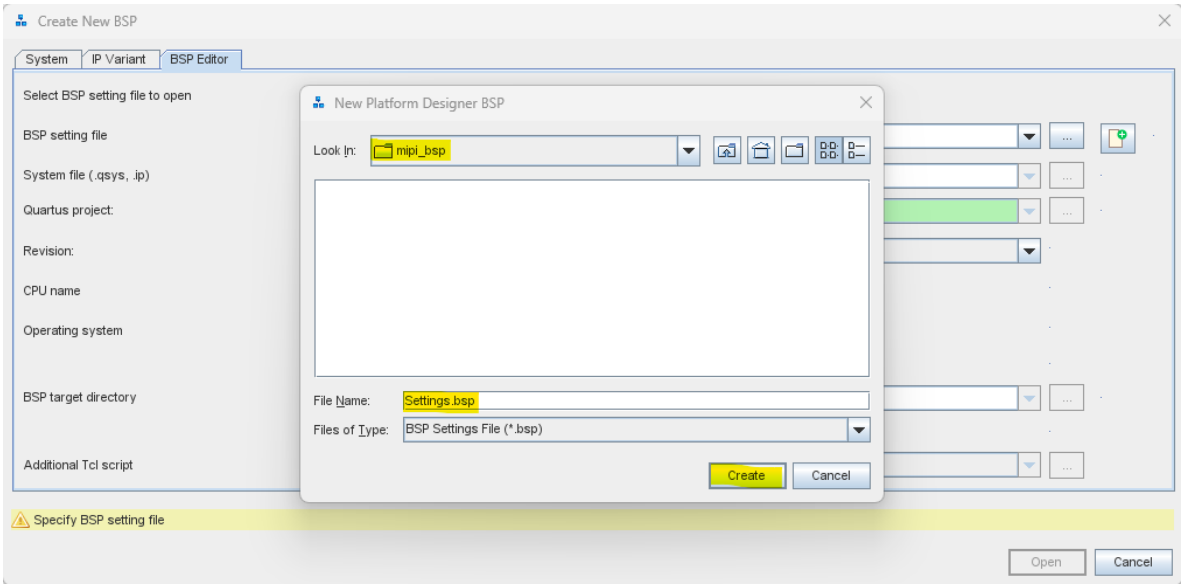


Figure 6-73: Create BSP

In the larger window (***Create New BSP***), click the ... for the ***System file (.qsys, .ip)*** and navigate to the ***top_system.qsys*** and **double-click** on it to select it.

Click **Create**.

Select the ***BSP Linker Script*** tab and change all ***Linker Regions*** to point to the **onchip_memory**, by clicking on the region and changing it.

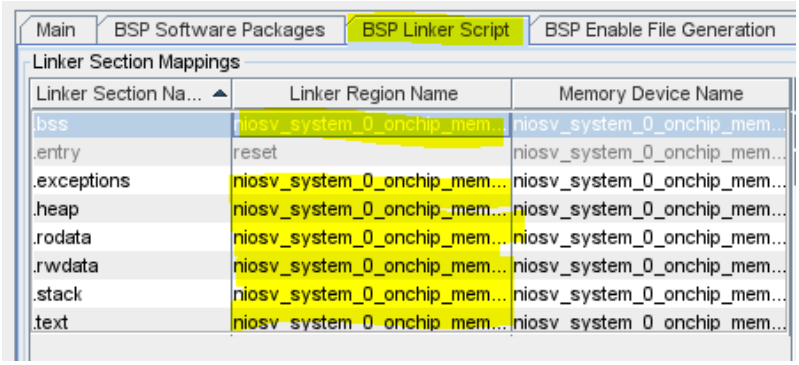


Figure 6-74: BSP Linke Script

6.4.4.1 Generate BSP

Now that we created the BSP, we need to ***Generate*** it. In the lower right corner, you should see a button called Generate BSP. If you don't see it, expand the BSP Editor window until the button is visible.

Click the ***Generate BSP*** button. This action should complete with no errors. It creates the HAL layer files and other files needed by the software tools.

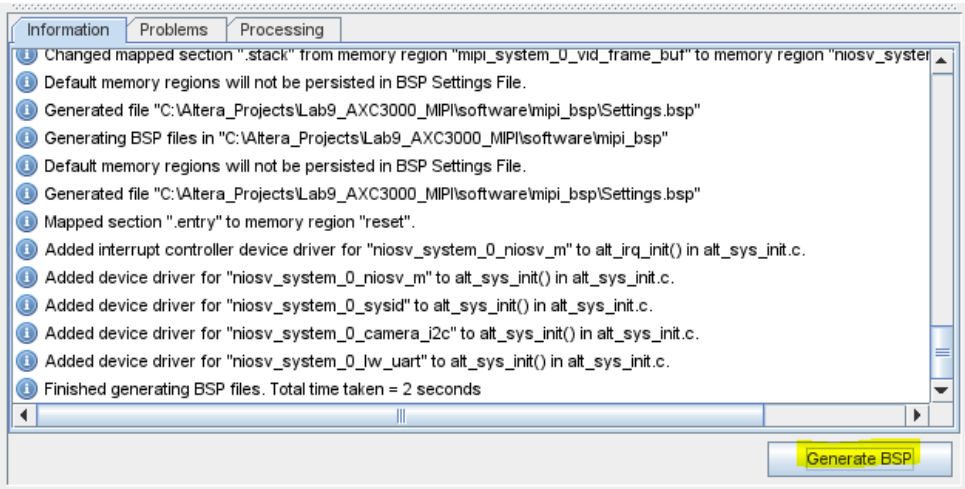


Figure 6-75: Generate BSP

6.5 Create the Top-Level Design

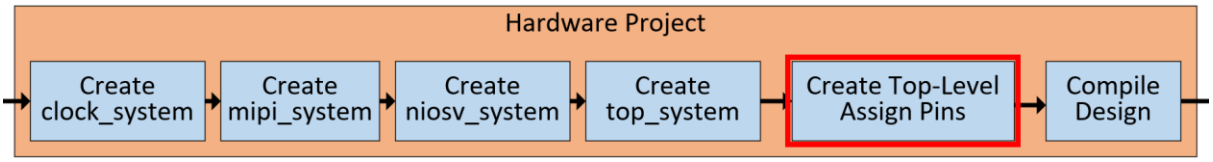


Figure 6-76: Create top-level design

The ***axc3000_mipi_top.v*** file contains the top-level entity called **axc3000_mipi_topXE5_MIPI_CSI2_Lab**. It instantiates the ***niosv_system*** subsystem, has a ***counter*** that blinks the Red LED on LED1, and connects the I2C pins to their tri-state I/O buffers.

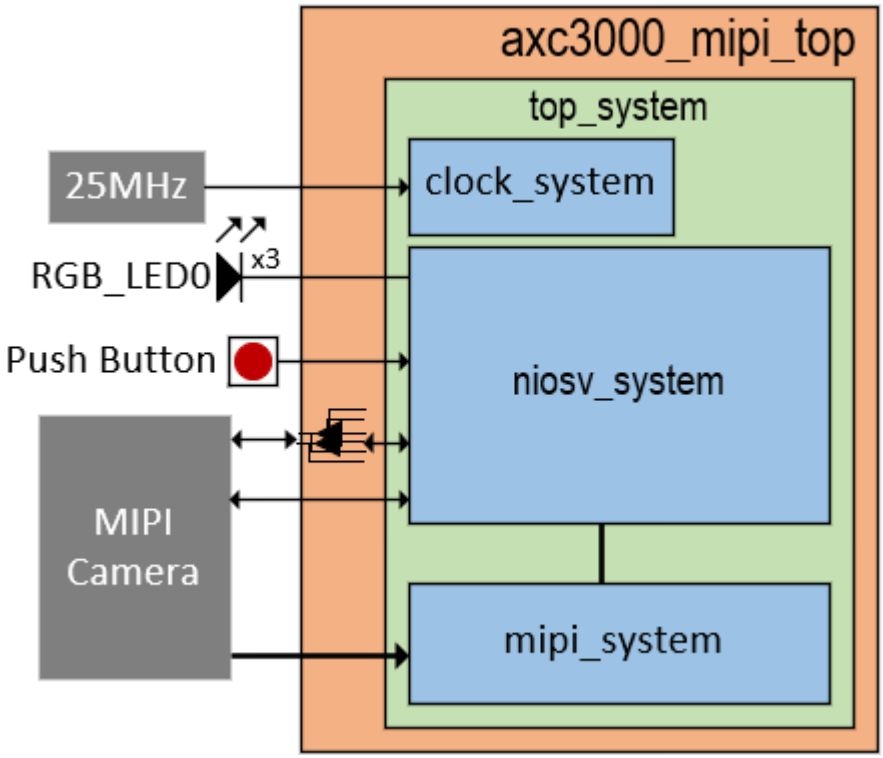


Figure 6-77: Create top-level design

6.5.1 Apply the pin assignments

The pin assignment file, ***axc3000_mipi_pins.tcl***, is in the ***sources*** folder, and needs to be applied to the project.

In Quartus, use the drop-down menu bar to do **View -> Console** to open a Tcl console for sourcing the pin assignment file.

In the Tcl console, **execute** the command, ***pwd*** to see which directory Quartus is pointing to., then source the ***.tcl*** file from the ***sources*** folder:

source ../sources/axc3000_mipi_pins.tcl

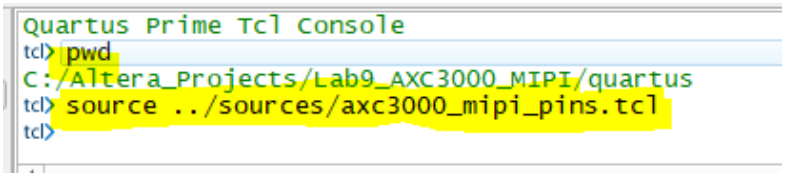


Figure 6-78: Apply pin assignment

6.6 Compile the Design

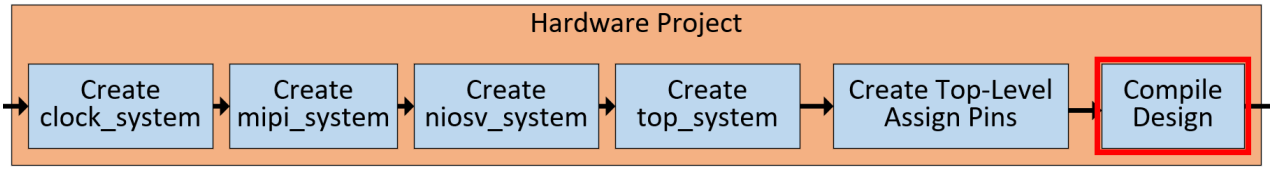


Figure 6-79: Compile the Design

The hardware project needs to be compiled with no errors in order to get a ***.sof*** configuration file. There are multiple steps performed in Quartus. These steps are listed in the flow diagram below. Ensure that the box to the left of the **Assembler** step is checked. This is the step that generates the ***.sof*** configuration file.

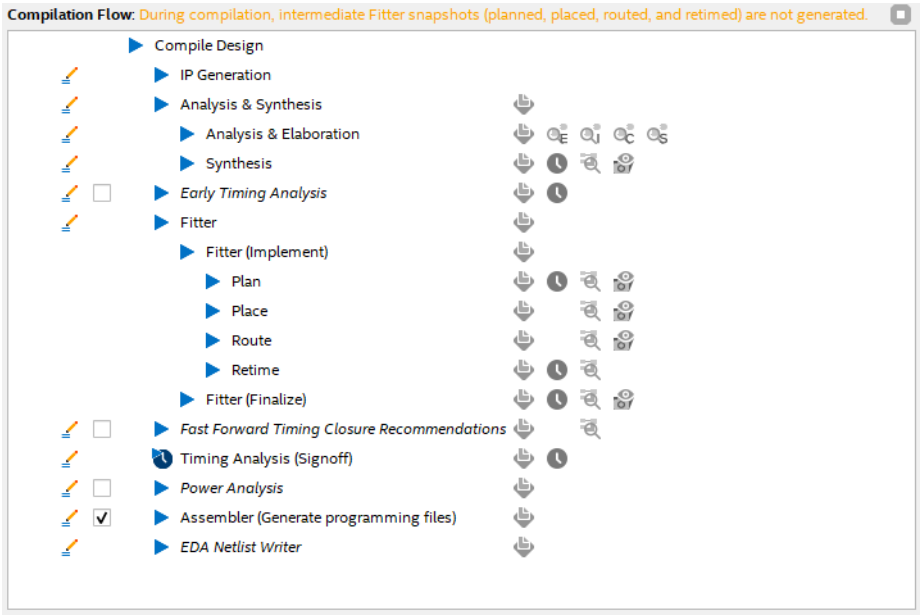
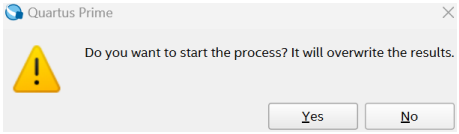


Figure 6-80: Compilation Flow

Start Compilation by clicking on  button on the toolbars, or ***Processing → Start Compilation***.

If a window is seen that states



Select **Yes**.

There should be no errors during compilation. If there are errors, they should be fixed before re-compilating. A green checkmark next to the Compile steps in the **Compilation Flow** window indicates that the compilation was successful.

As Quartus compiles the design, there will be a progress bar that shows the steps that it does and the checkmarks as the steps are completed. Notice the checkmarks as steps are completed. The time used for each step depends on the computer used such as the number of processors, amount of memory, and processor clock speed.

There will be a **Timing Analysis** Window that appears at the end, but the focus is on the compilation.

Once the compilation is complete, the ***Project Overview*** should look like:

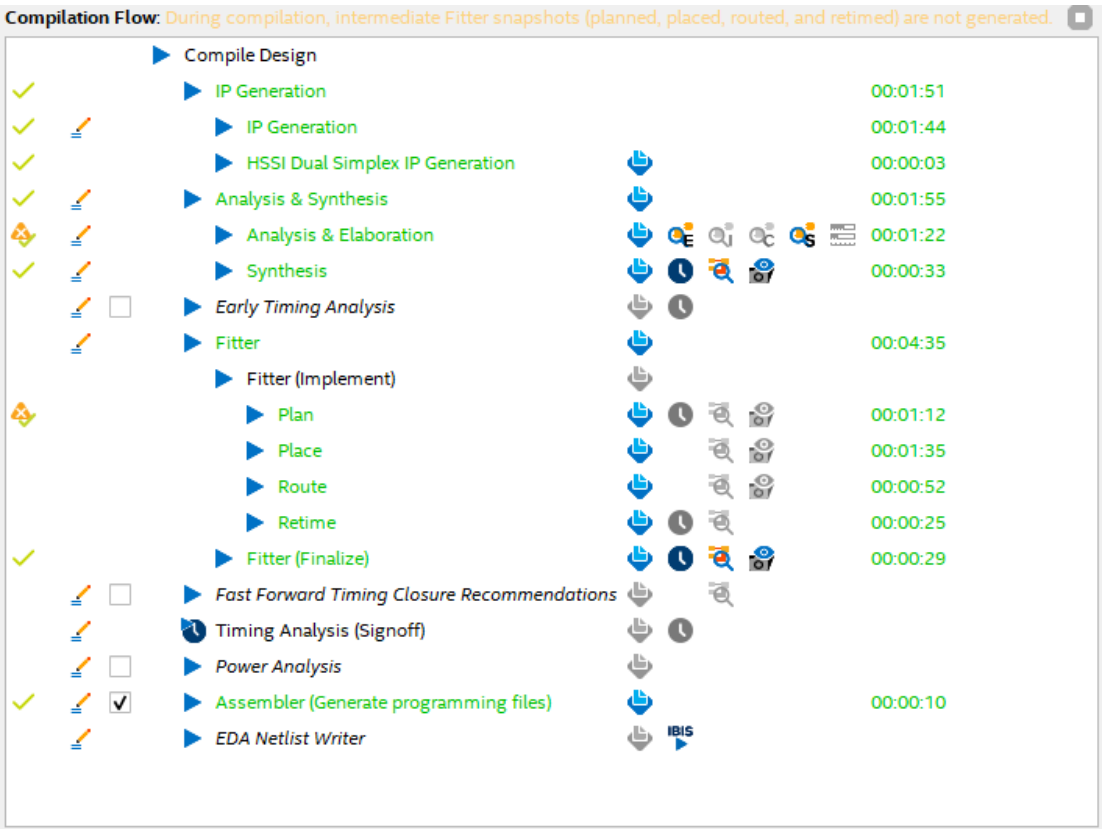


Figure 6-81: Completed compile flow

This completes the Hardware Phase of the Project.

7 Software Project

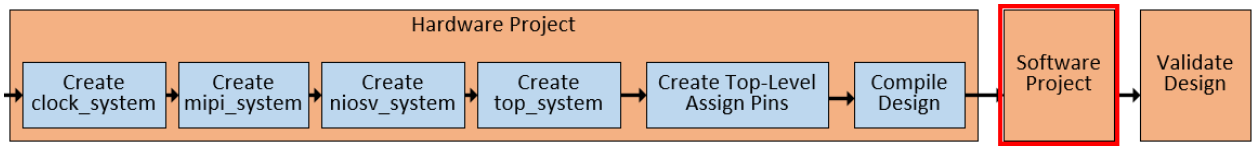


Figure 7-1 : Testing the Design

Every processor-based design has a software project in addition to the hardware project. The software project needs a Board Support Package (BSP) prior to compiling the C code.

If you didn't generate the BSP during the Hardware Project stage, refer to section **6.4.4**.

The software project is in the software folder and has 2 sub-folders, one for the BSP (***mipi_bsp***) and one for the application code (***mipi_app***).

The ***mipi_bsp*** folder contains the BSP file ***settings.bsp***, which is generated in Platform Designer.

The ***mipi_app*** folder contains the NIOS-V C code source ***niosv_mipi_mira220.c*** file which runs the application.

7.1 Building software project using Ashling RiscFree IDE for Intel FPGA

The application C code is provided to you, and you generated the BSP. There are a few steps that need to be taken to open and compile the software project.

7.1.1 Create Application Project

The Ashling Riscfree IDE tool uses CMake flow, and the files for it need to be generated from the BSP file.

Open a **Nios V Command shell (Quartus Prime Pro 25.1.1)** from the Quartus Windows Programs menu and change directory to the project's folder, then execute the command :
niosv-app -b=software/mipi_bsp ***-a=software/***mipi_app ***-s=software/mipi_app/niosv_mipi_mira220.c***
to generate the CMake application file **CMakeLists.txt**.

Figure 7-2 : Generate CMake files

Open the **Ashling RiscFree IDE** either from the Windows Programs menu and point its workspace to the **software** folder, or by typing **riscfree -data software** in the ***Nios V Command Shell***. This points the RiscFree Workspace to the **software** folder.

Click on **Create a project...** in the thumbnails.

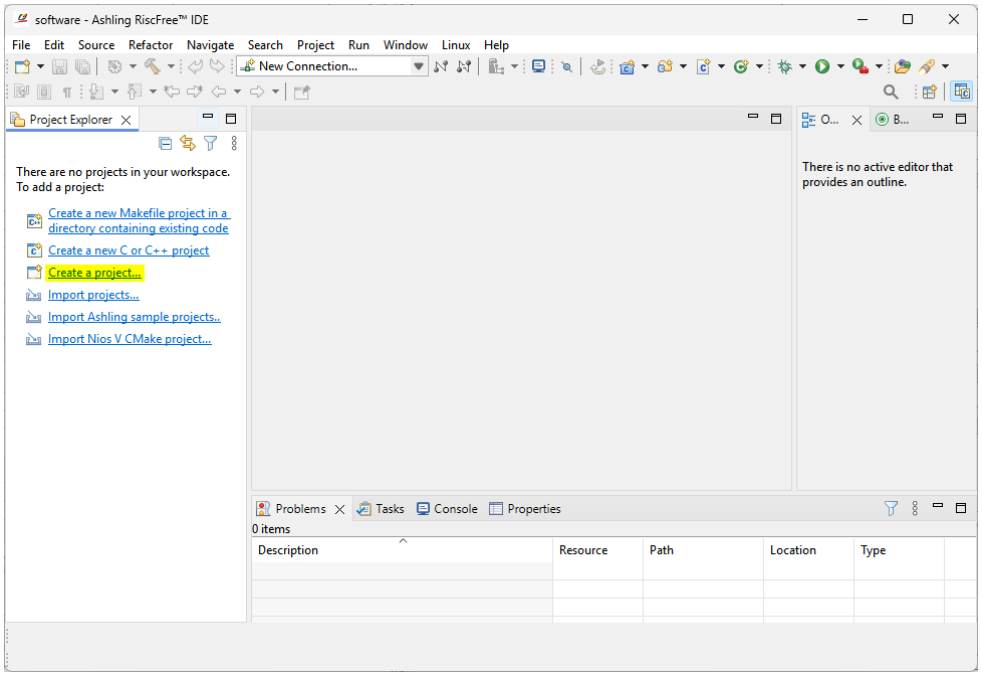


Figure 7-3 : Open Ashling RiscFree IDE

Expand the **C/C++** wizard and select the **C Project** item, then click **Next.**

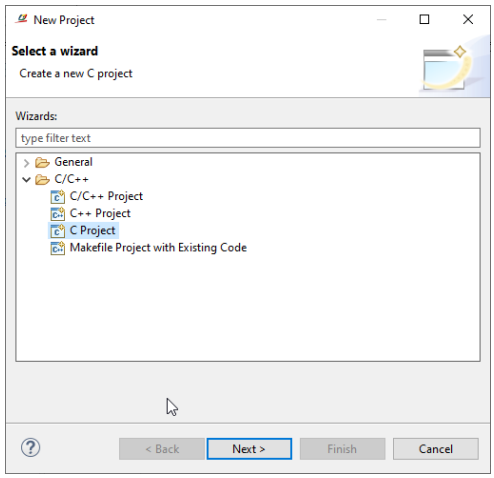


Figure 7-4 : Select C Project type

Expand the **CMake driven** Project type and select **Empty Project**. In the Toolchains pane, select **CMake driven**.

Type **mipi_app** for the Project name and click **Finish**. This matches the pre-existing folder containing the application project.

Click **Finish**.

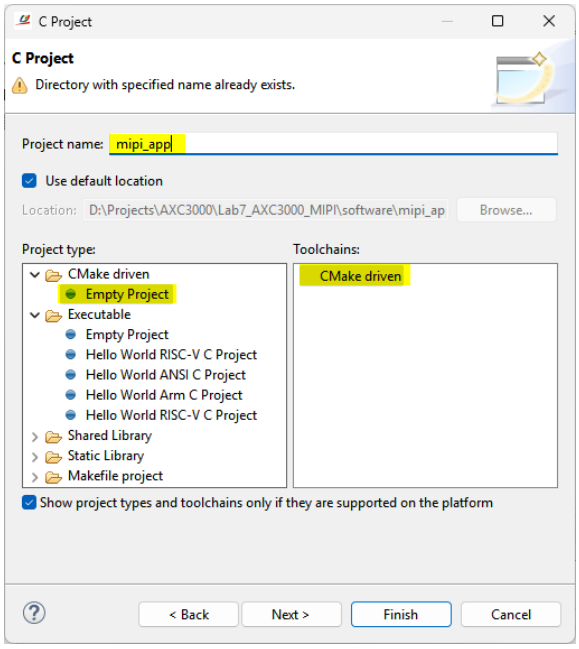


Figure 7-5 : Create CMake driven axe5000_mipi_app project

In the **Ashling RiscFree IDE**, you should see the ***mipi_app*** folder.

Right-click the ***mipi_app*** project and select **Build Project**.

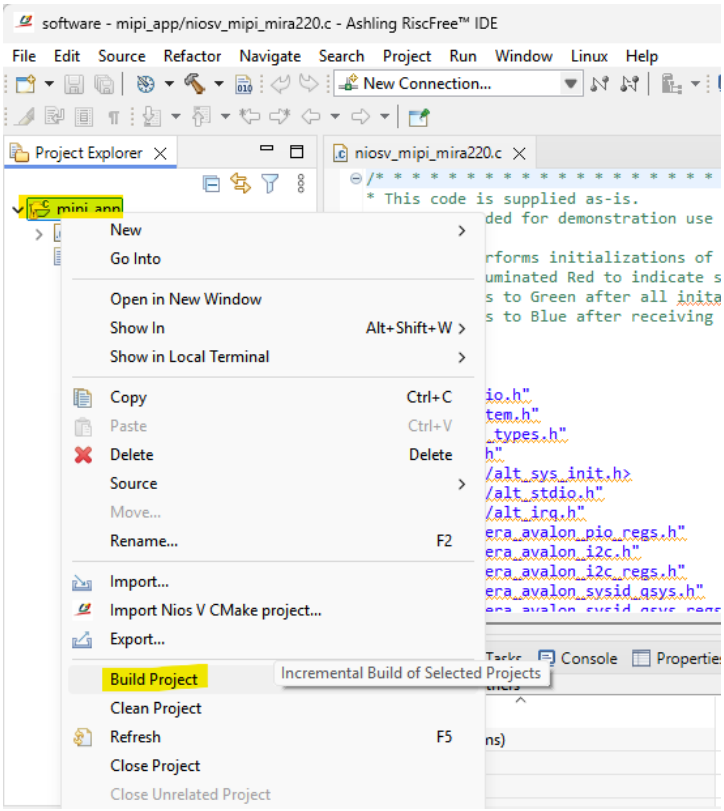


Figure 7-6 : Build software project

When the ***Build Project*** progress meter in the lower right corner reached 100%, always check the ***Problems*** messages window for error message.

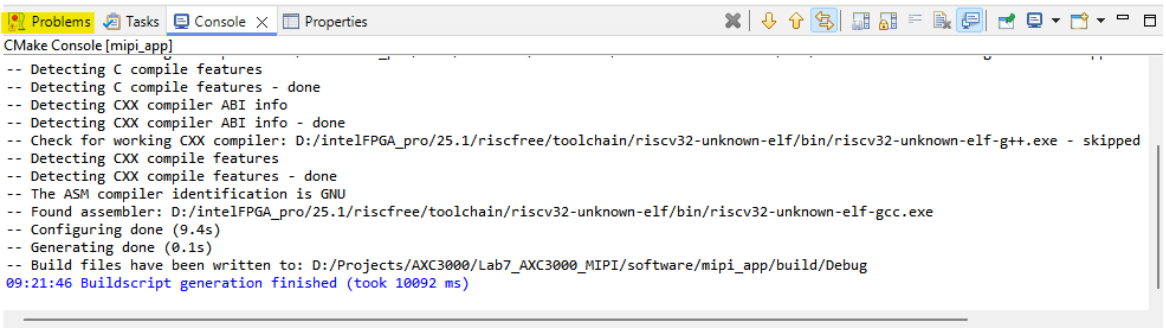


Figure 7-7 : Compiler Console Window

7.2 Merge the .hex file into the .sof

Before we can configure the FPGA with a .sof file, we need to incorporate the .hex file into it, which goes into the ***onchip_memory*** for NIOS-V to execute.

If you recall, when we added the ***onchip_memory*** in the ***niosv_system***, we specified a place holder path to the HEX file
../software/mipi_app/build/Debug/niosv_system_0_onchip_memory.hex.

7.2.1 Update memory initialization file (HEX file into SOF)

From the Quartus menu bar, execute Processing → Update Memory Initialization File.

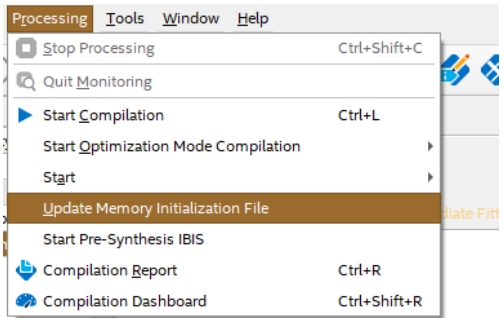


Figure 7-8 : Update Memory Initialization File

After this process is done, re-run the Assembler compile step by executing ***Processing→Start→Start Assembler***. Answer ***Yes***, when prompted to start the process.

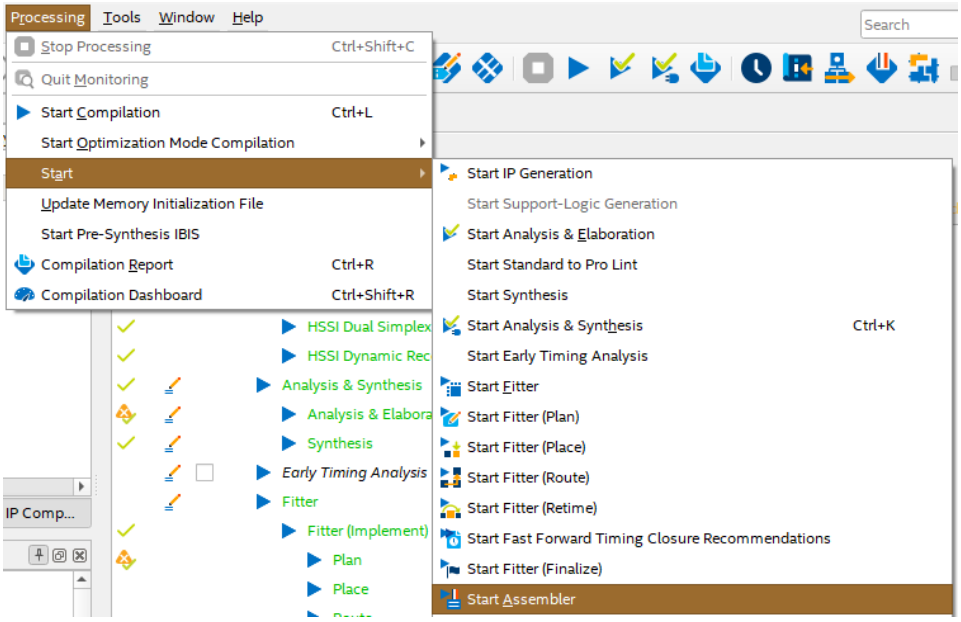


Figure 7-9 : re-run Assembler

8 Validate the Design

Depending on which of the 3 methods you are using to connect to the board, consult the right section. These methods are:

- PC direct to local AXC3000
- CloudLabs Virtual Machine to local AXC3000
- CloudLabs Virtual Machine to Remote AXC3000

Note: If you have installed the Quartus Standalone Programmer or Quartus using an installation path different to that recommended in the installation manual, you will need to modify the path in the python file. It is located in the scripts sub folder

- visualize_vfb.py (lines 26 or 27)

8.1 PC direct to local AXC3000

The next step is to run the demo. For this, the following steps will need to be done:

- Connect the camera to the AXC3000 board.
- Configure the Agilex-3 FPGA with the .sof.
- Run the script to extract the image.

8.1.1 Connect the camera to the AXC3000 board

The amsOSRAM Mira220 camera comes with a 15-pin flat ribbon cable. The connection to the AXC3000 board is done via the CRUVI adapter (TEC0278) J2 connector. The figure below shows the assembled camera demo.

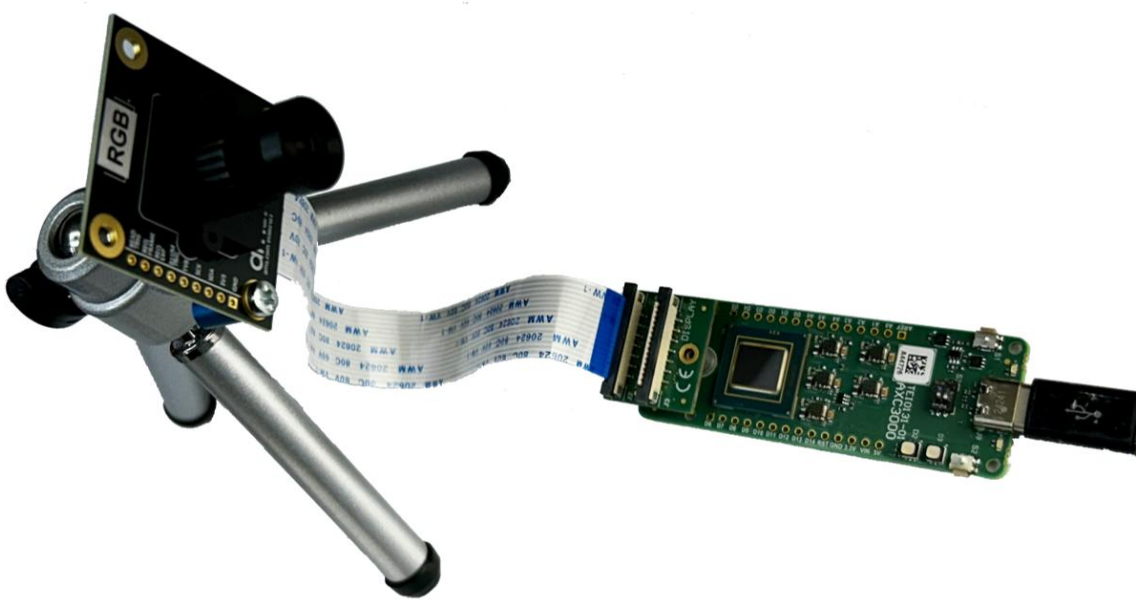


Figure 8-1 : Connected Hardware

8.1.1.1 Connecting the Camera to the AXC3000

The camera has an FPC connector which mates to the flat ribbon cable. Both the FPC connector and the cable have conductors on only one side. To make the connection, make sure that the FPC connector has its black tab lifted out on both ends. Insert any end of the cable, with the conductors facing the white side of the connector, then push the tabs down into the connector body.

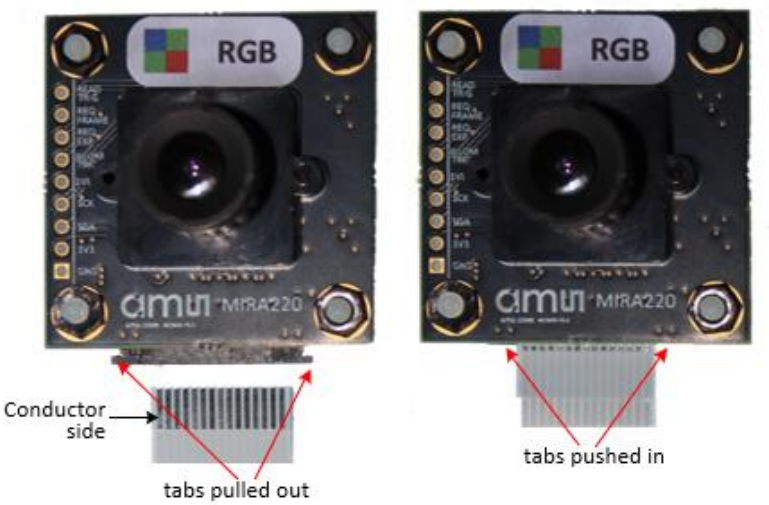


Figure 8-2 : Connect cable to camera

Similarly, insert the other end of the cable into J2 (CAMERA) of the CRUVI adapter, and insert the TEC0278 into the AXC3000 CRUVI receptacle (J3), as shown in the image below.

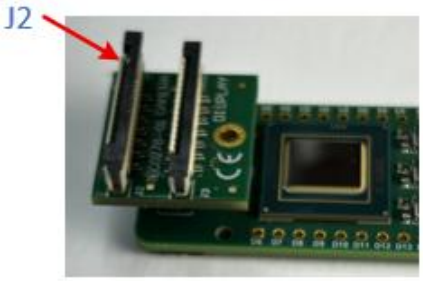


Figure 8-3 : Connect cable to TEC0278 CRUVI adapter, and plug into AXC3000

8.1.2 Configure the Agilex-3 FPGA

Connect the USB-C cable between the AXC3000 board and a PC USB port.

8.1.2.1 Open the Quartus Prime Programmer

On the Quartus Prime Pro, click **Tools → Programmer**, or click on the Programmer icon in the tool bar.

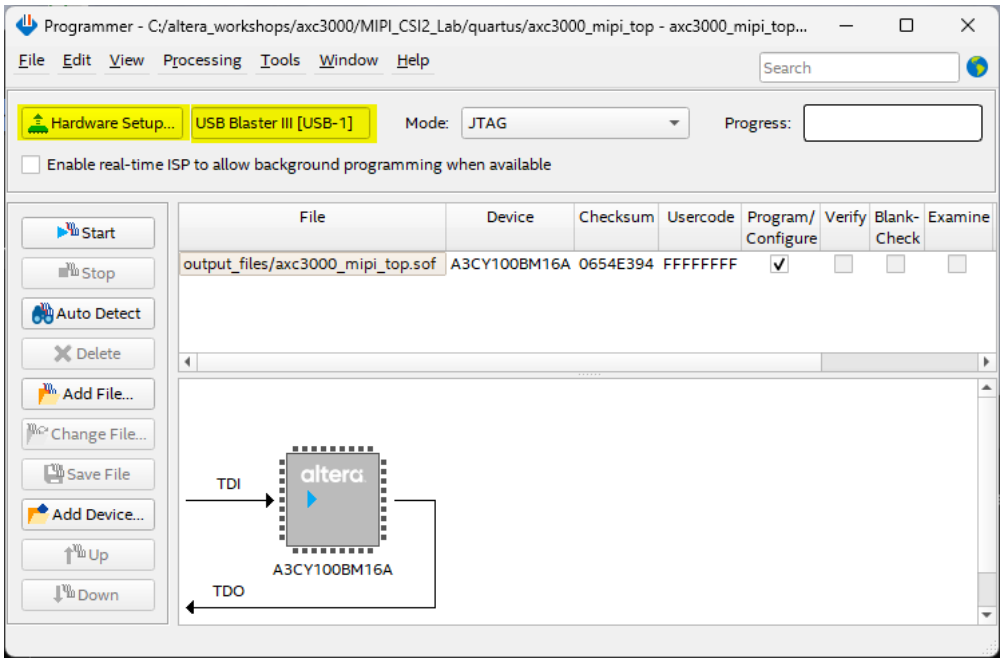


Figure 8-4 : Quartus Programmer

Your window contents might look different.

Click the ***Hardware Setup*** button and **double-click** on the ***USB-Blaster III***, then click **Close**.

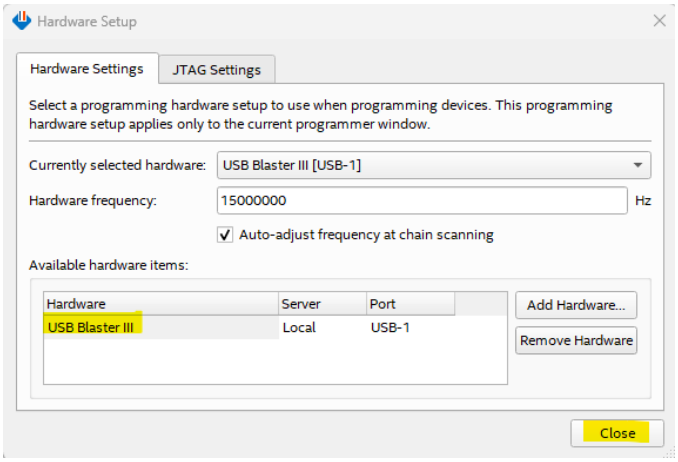


Figure 8-5 : USB Blaster Hardware Setup

Ensure that the ***USB-Blaster III*** is selected, and the mode is set to ***JTAG***.

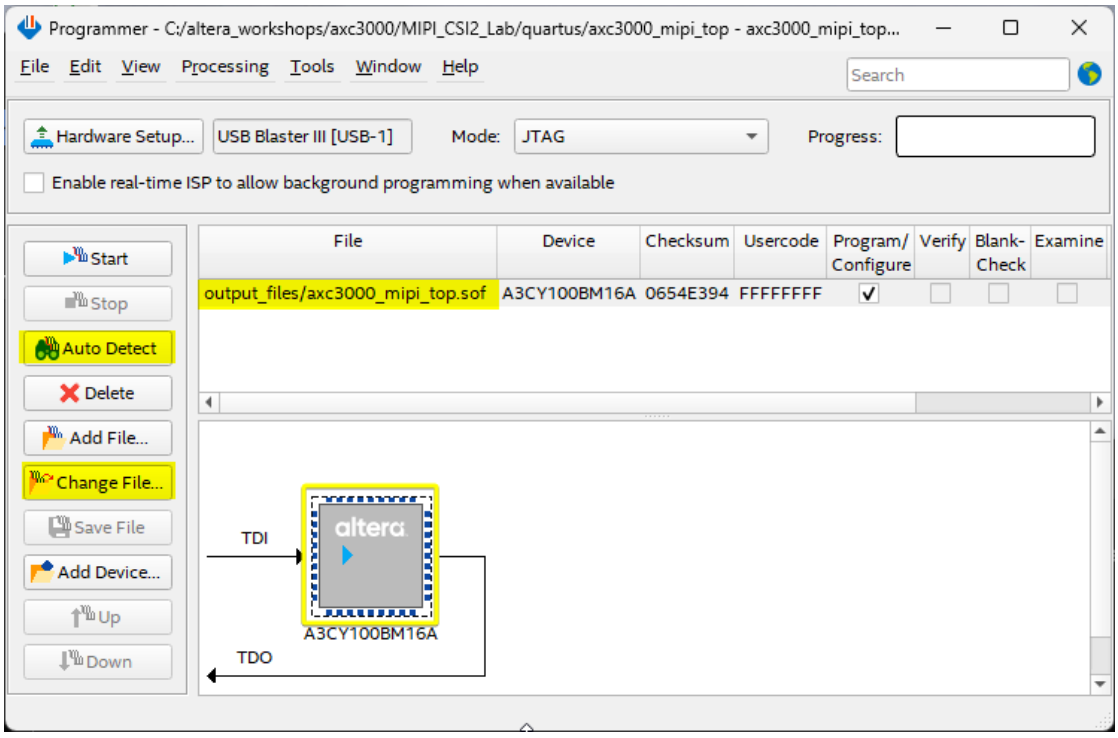


Figure 8-6 : Detect device and assign .sof file

If you don't see a device showing in the main window, click on **Auto Detect**.

If the .sof file is not already selected, ***select*** the FPGA and click on the ***Change File*** button and navigate to the ***axc3000_mipi.sof***. It should be in the ***output_files*** sub-folder, or in the ***quartus*** sub-folder.

Click the ***Start*** button to begin the programming operation and wait for the progress bar to hit 100%. If the ***Start*** button is not available, make sure the ***Program/Configure*** box for the selected SOF is **checked**.

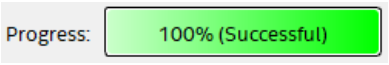


Figure 8-7 : Programmer progress bar

When the NIOS-V program runs successfully and data is captured into the Video Frame Buffer, LED D2 should go from Red to Blue. If it gets stuck at Green, it means that there is a problem, and the frame buffer is not being written to. In the case of a Green D2, unplug the USB cable from the PC, then plug it back in and re-configure the Agilex-3.

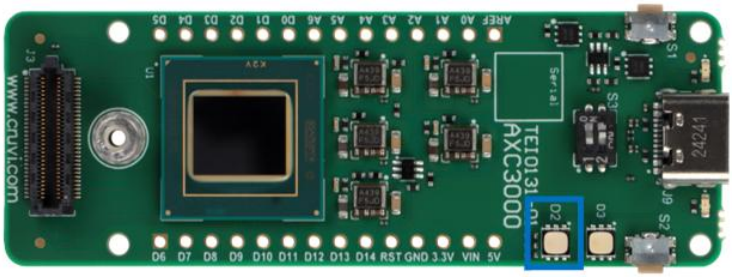


Figure 8-8 : LED D2 indicator

8.1.3 Run script to extract Video Frame Buffer content

In the ***scripts*** subfolder, there are 2 scripts which use ***System Console*** under the hood to extract the video frame buffer and display it on the screen.

Open a Windows ***Command Prompt*** (CMD) window.

Change directory to be at the ***scripts*** sub-folder.

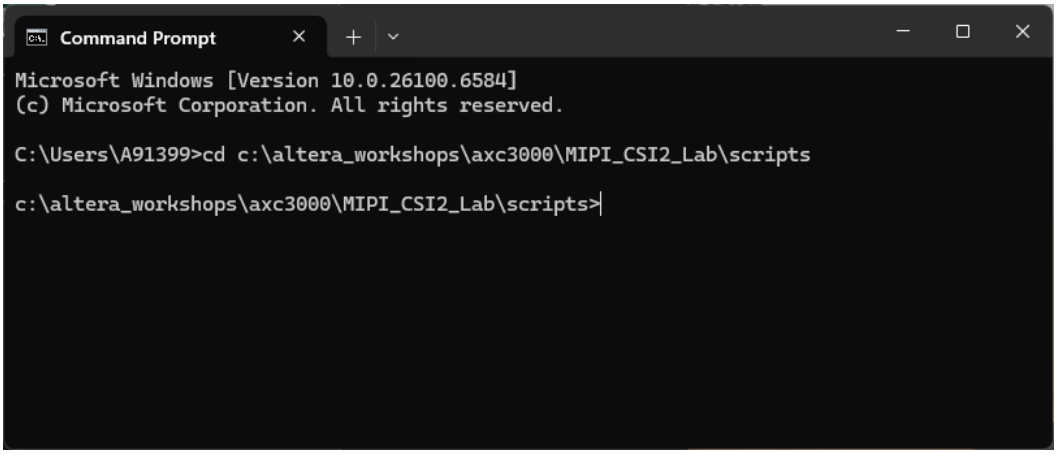


Figure 8-9 : Command Prompt Window

In the ***CMD*** window, type **py visualize_vfb.py**, to start the frame buffer extraction Python script.

You will be prompted to enter the USB COM port which the board is connected to. **Enter** the ***digit*** corresponding to that port and hit ***Enter*** on the keyboard.

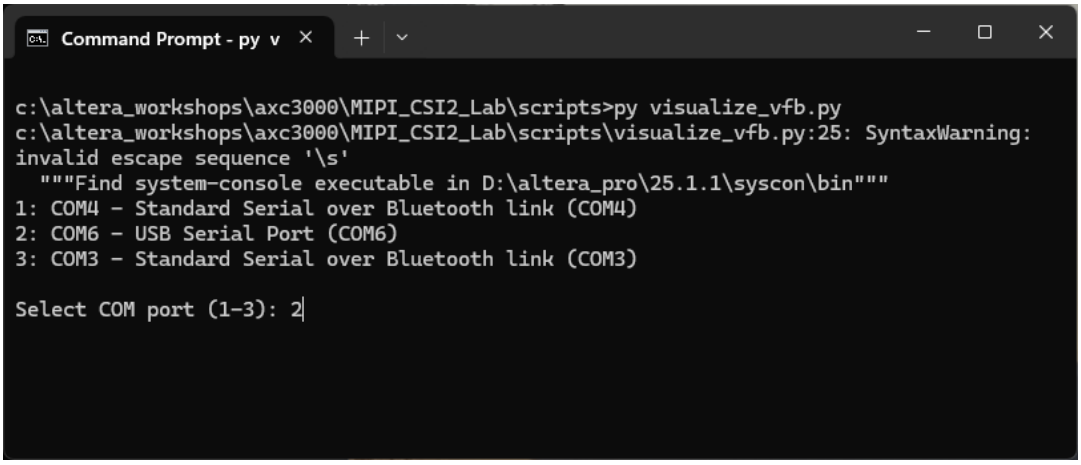


Figure 8-10 : Run the Python scripts and set COM port number

You should see a viewer window come up and start showing the 320x200 (multiplied by 3).

Turn the camera lens to focus the image. There is a few seconds lag for the new image to be re-drawn.

If you need more image exposure, press the S2 button and wait a few seconds to see a new image.

The image will look pixelated because it is blown up 3x.

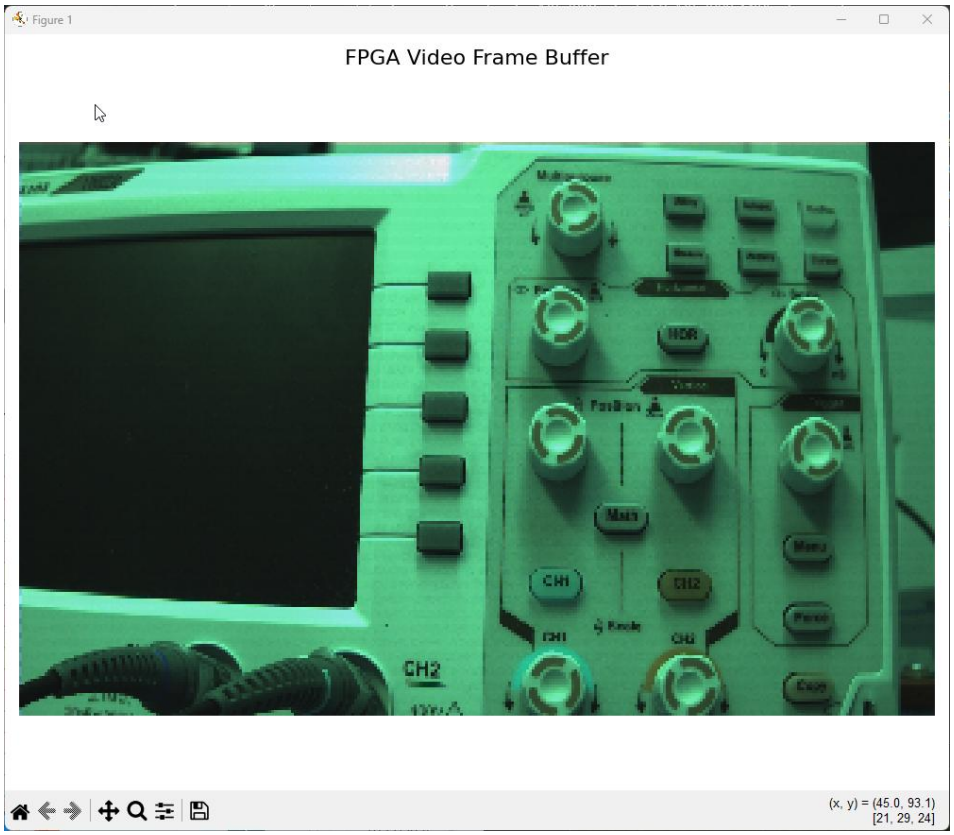


Figure 8-11: Image viewer

To exit the viewer, hit ***Ctrl-C*** in the CMD window.

8.2 CloudLabs Virtual Machine to local AXC3000

The following instructions are for the user who is building the design on a CloudLabs Windows Virtual Machine and using a native PC to connect to a local AXC3000 board. In this setup, the configuration file needs to be transferred from the Virtual Machine to the native PC for use with a local AXC3000.

8.2.1 Clone the repository

There are 2 scripts needed when running the demo in this mode. So, the user needs to clone the repository onto their PC.

In a Windows Command Prompt shell, do the following:

cd C:

git clone https://github.com/ArrowElectronics/altera_workshops.git

8.2.2 Transfer SOF file from VM to Local PC

Click on the **ArrowDTD** icon in the CloudLabs Virtual Machine Task Manager to launch the tool.

Figure 8-12: Launch Arrow DTD application

Click the **Shared Folder** utility in the Arrow DTD application to open a shared File Explorer window.

Figure 8-13: Arrow DTD application

Open an Internet browser on your native PC and navigate to the **URL** indicated and **enter** the provided ***username*** and ***password***.

In the CLoudLabs VM, **Drag and Drop** the ***c:\altera_workshops\axc3000\MIPI_CSI2_Lab\output_files\axc3000_mipi_top.sof*** file into the ***Shared folder***.

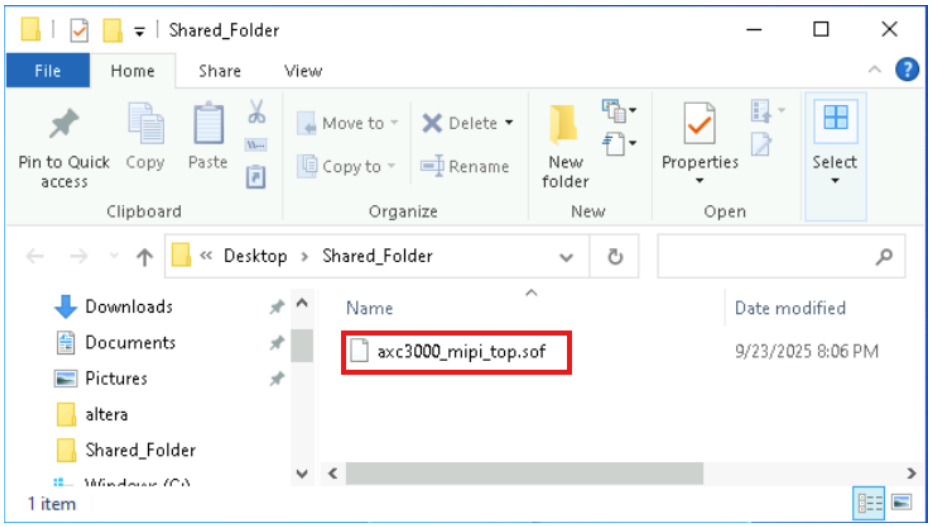


Figure 8-14: Deposit the .sof file in the shared folder

This will be reflected in the **browser** on the native PC.

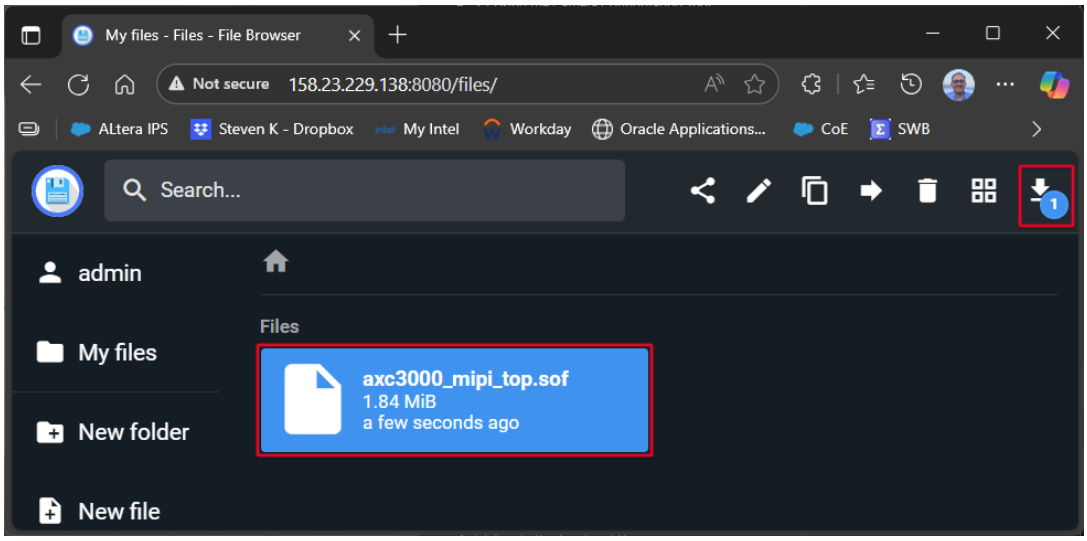


Figure 8-15: Download the mirrored .sof file to the native PC

Download the SOF file to a convenient location on the local PC, by clicking the download icon.

The next step is to run the demo. For this, the following steps will need to be done:

- Connect the camera to the AXC3000 board.
- Configure the Agilex-3 FPGA with the .sof.
- Run the script to extract the image.

8.2.3 Connect the camera to the AXC3000 board

The amsOSRAM Mira220 camera comes with a 15-pin flat ribbon cable. The connection to the AXC3000 board is done via the CRUVI adapter (TEC0278) J2 connector. The figure below shows the assembled camera demo.

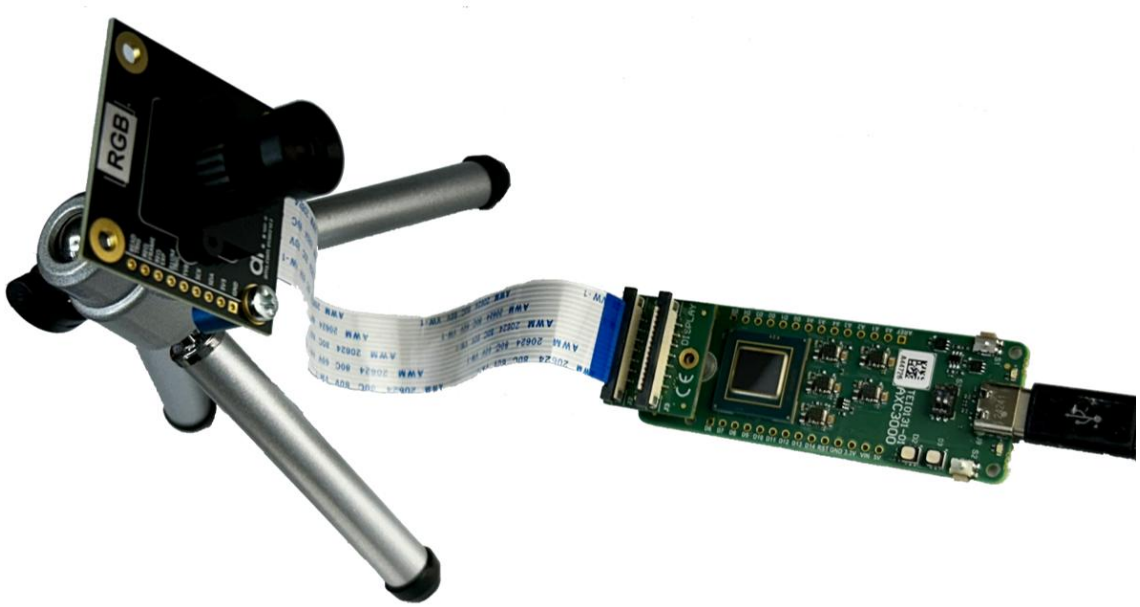


Figure 8-16 : Connected Hardware

8.2.3.1 Connecting the Camera to the AXC3000

The camera has an FPC connector which mates to the flat ribbon cable. Both the FPC connector and the cable have conductors on only one side. To make the connection, make sure that the FPC connector has its black tab lifted out on both ends. Insert any end of the cable, with the conductors facing the white side of the connector, then push the tabs down into the connector body.

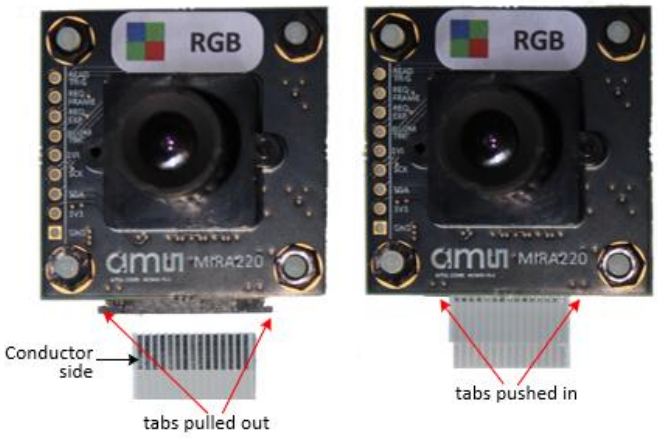


Figure 8-17 : Connect cable to camera

Similarly, insert the other end of the cable into J2 (CAMERA) of the CRUVI adapter, and insert the TEC0278 into the AXC3000 CRUVI receptacle (J3), as shown in the image below.

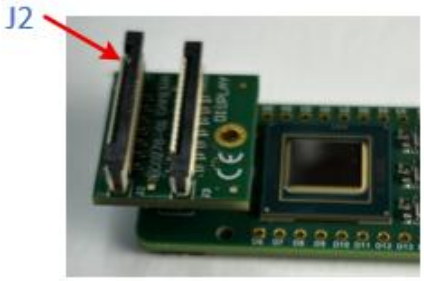


Figure 8-18 : Connect cable to TEC0278 CRUVI adapter, and plug into AXC3000

8.2.4 Configure the Agilex-3 FPGA

Connect the USB-C cable between the AXC3000 board and the native PC USB port.

8.2.4.1 Open the Quartus Prime Programmer on native PC

From the Quartus Programmer installation site, open the ***Programmer (Quartus Pro 25.1.1)***.

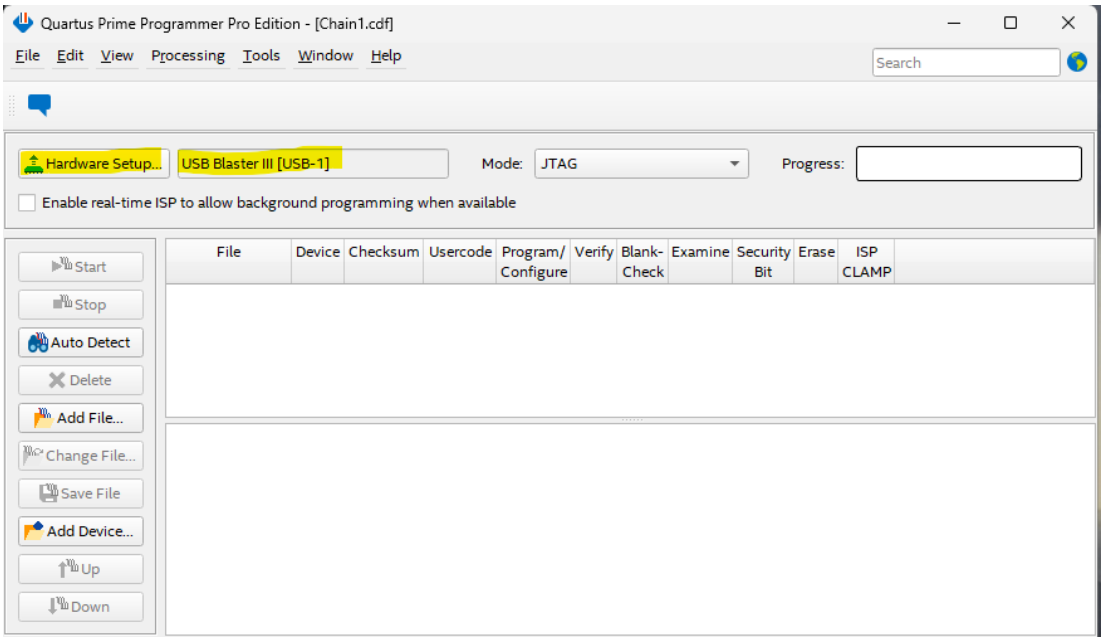


Figure 8-19 : Quartus Programmer

Click the ***Hardware Setup*** button and **double-click** on the ***USB-Blaster III***, then click **Close**.

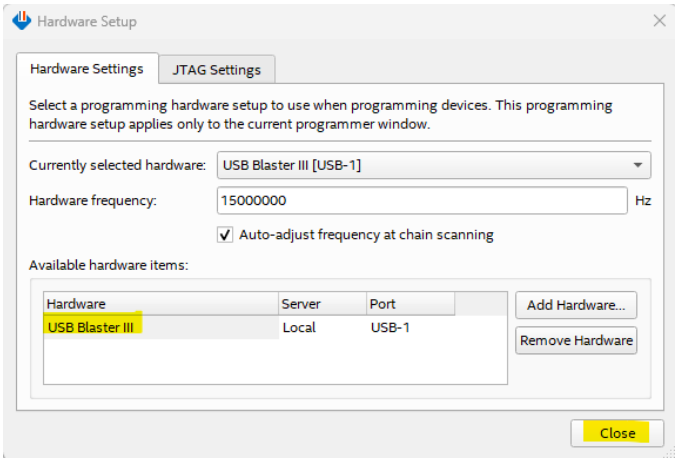


Figure 8-20 : USB Blaster Hardware Setup

Ensure that the ***USB-Blaster III*** is selected, and the mode is set to ***JTAG***.

If you don't see a device showing in the main window, click on **Auto Detect** button to query the JTAG chain and display what is connected to it.

Select the FPGA, to activate the ***Change File...*** button.

Click the ***Change File...*** button and navigate to the downloaded .sof file and **double-click** it.

Check the box for ***Program/Configure***.

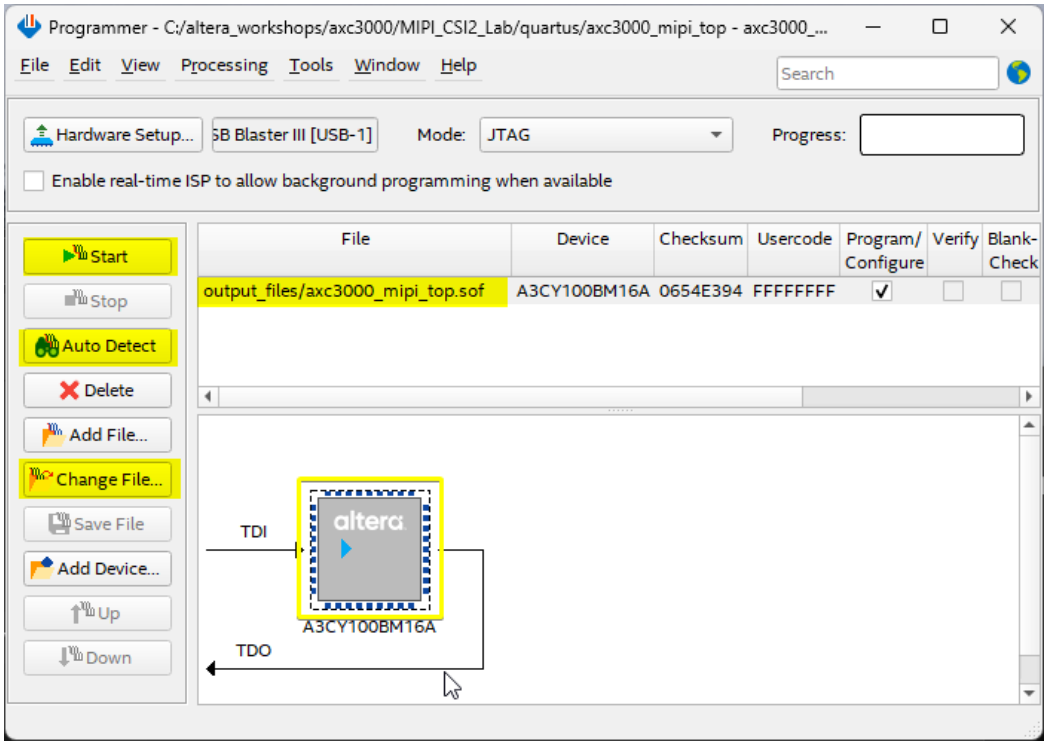


Figure 8-21 : Detect device and assign .sof file

Click the ***Start*** button to begin the programming operation and wait for the progress bar to hit 100%. If the ***Start*** button is not available, make sure the ***Program/Configure*** box for the selected SOF is **checked**.

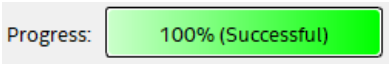


Figure 8-22 : Programmer progress bar

When the NIOS-V program runs successfully and data is captured into the Video Frame Buffer, LED D2 should go from Red to Blue. If it gets stuck at Green, it means that there is a problem, and the frame buffer is not being written to. In the case of a Green D2, unplug the USB cable from the PC, then plug it back in and re-configure the Agilex-3.

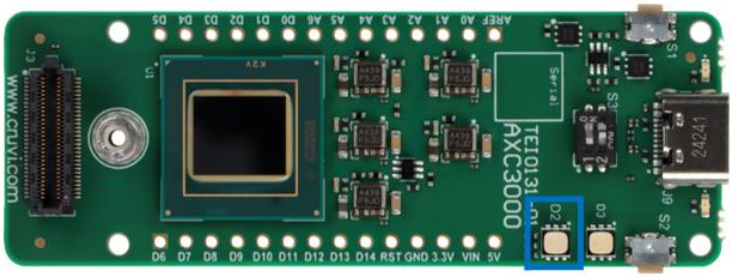


Figure 8-23 : LED D2 indicator

8.2.5 Run script to extract Video Frame Buffer content

In the ***scripts*** subfolder, there are 2 scripts which use ***System Console*** under the hood to extract the video frame buffer and display it on the screen.

Open a Windows ***Command Prompt*** (CMD) window.

Change directory to be at the ***scripts*** sub-folder.

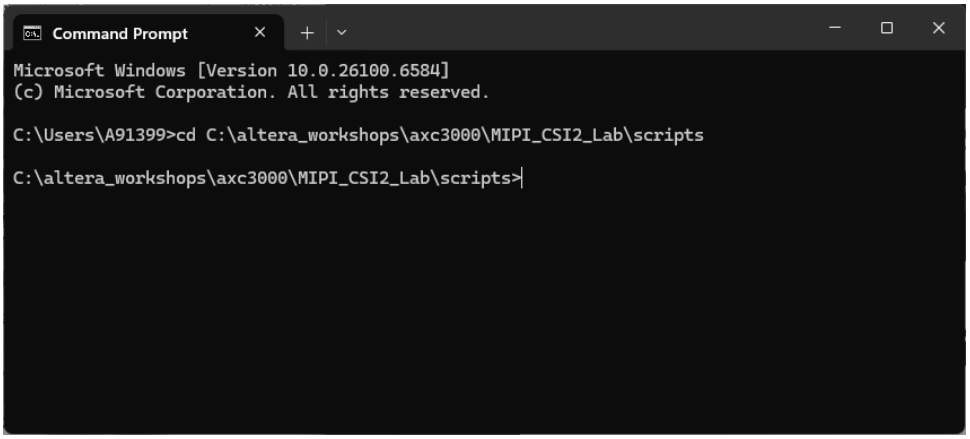


Figure 8-24 : Command Prompt Window

In the ***CMD*** window, type **py visualize_vfb.py**, to start the frame buffer extraction Python script.

You will be prompted to enter the USB COM port which the board is connected to. **Enter** the ***digit*** corresponding to that port and hit ***Enter*** on the keyboard.

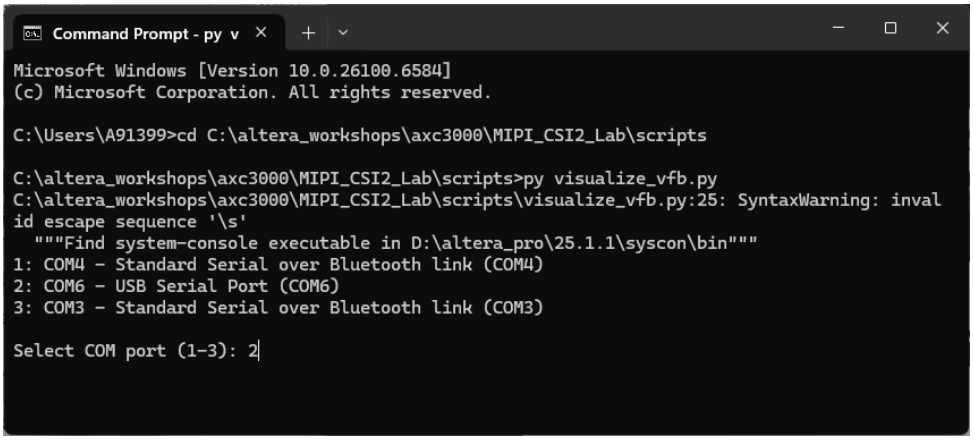


Figure 8-25 : Run the Python scripts and set COM port number

You should see a viewer window come up and start showing the 320x200 (multiplied by 3).

Turn the camera lens to focus the image. There is a few seconds lag for the new image to be re-drawn.

If you need more image exposure, press the S2 button and wait a few seconds to see a new image.

The image will look pixelated because it is blown up 3x.

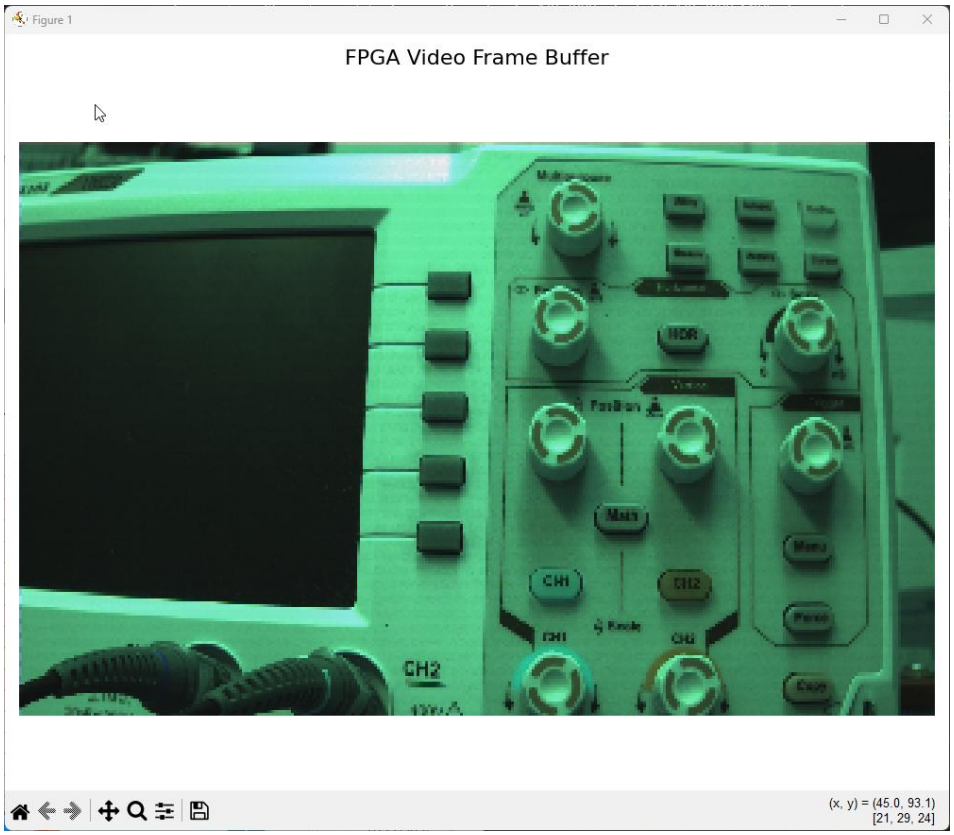


Figure 8-26: Image viewer

To exit the viewer, hit ***Ctrl-C*** in the CMD window.

8.3 CloudLabs Virtual Machine to Remote AXC3000

The following instructions require the user to be utilizing a CloudLabs Windows Virtual Machine.

Connect to the remote board farm

Click on the **ArrowDTD** icon in the CloudLabs Virtual Machine Task Manager to launch the tool.

Figure 8-27: Launch Arrow DTD application

Select an available board from the dropdown menu and **click** the ***Connect Board*** button.

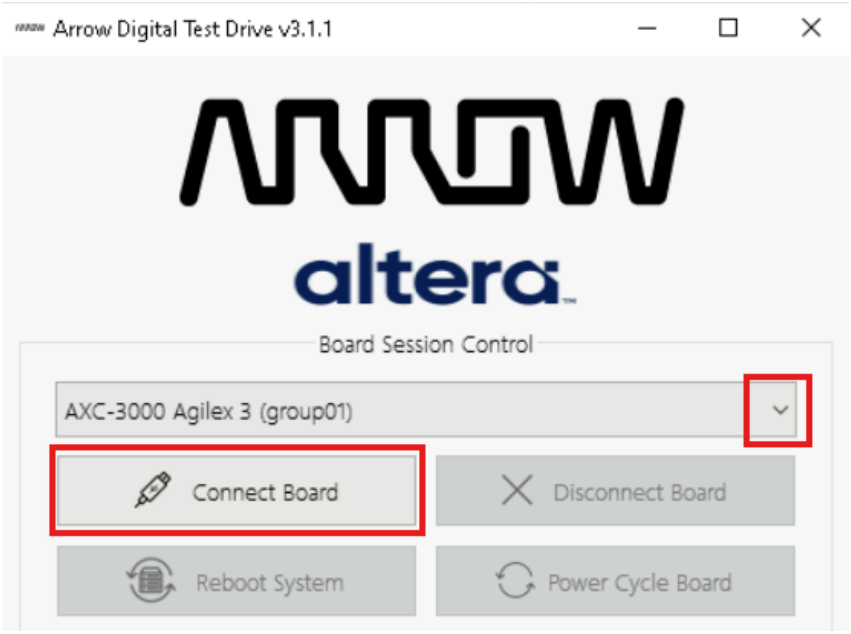


Figure 8-28: Launch Arrow DTD application

Wait until the board is ready.

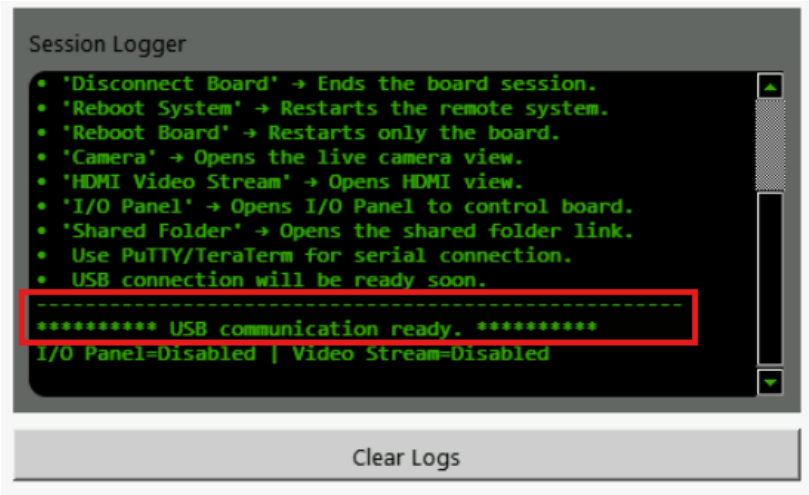


Figure 8-29: Board connection status

Click the ***Camera*** button to view the hardware you are connected to, then **select** the box for ***Flip Vertical***.

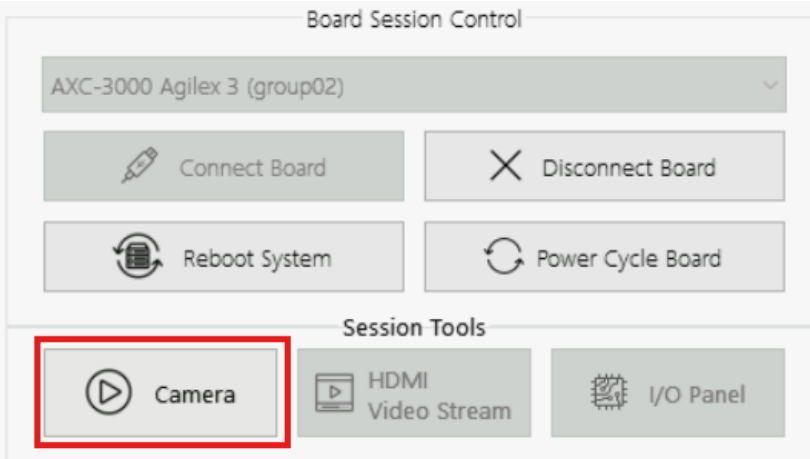


Figure 8-30: Open Camera view

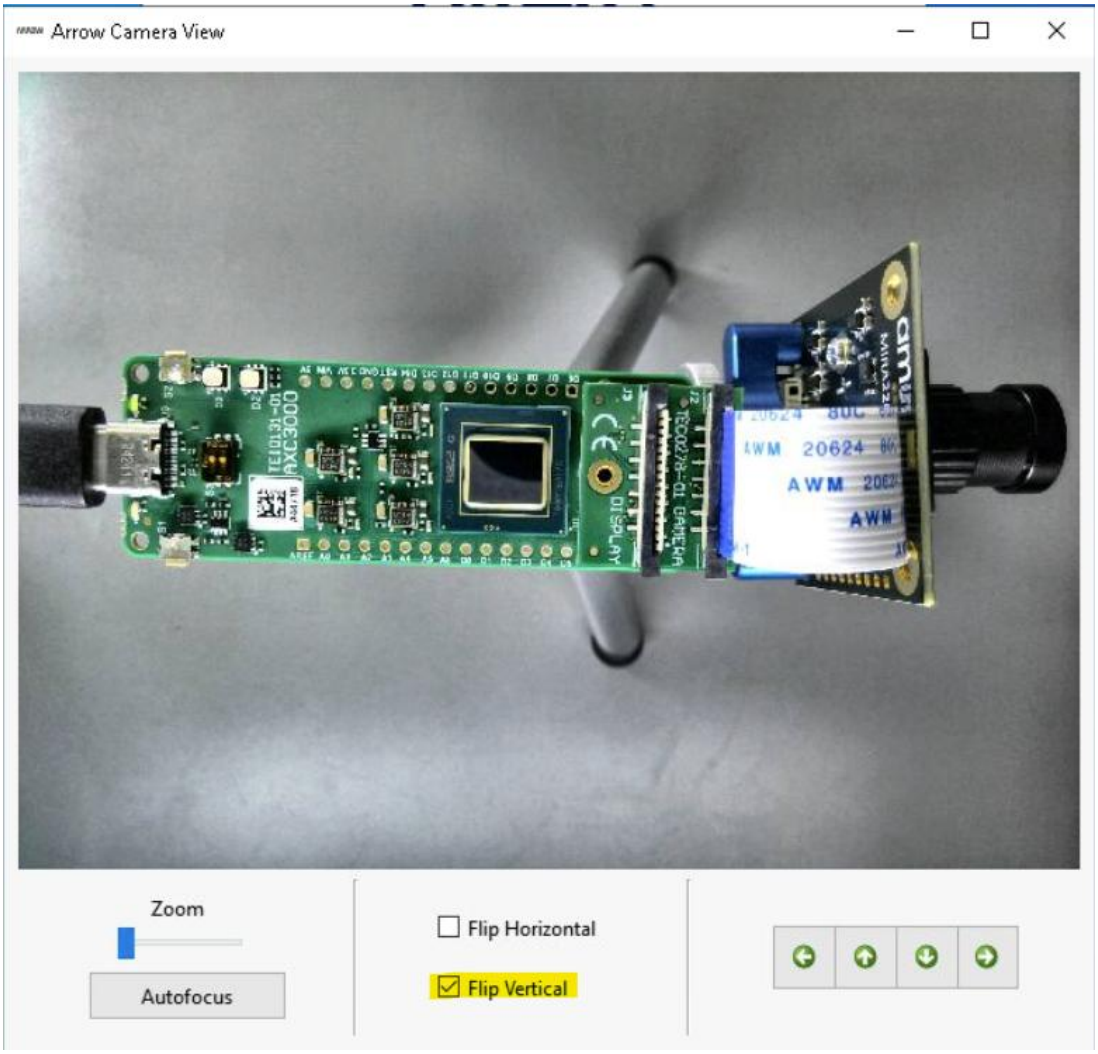


Figure 8-31: Camera view

Programming – Configuration

Configuring the Agilex-3 FPGA is done remotely.

Open the Quartus Prime Programmer in the CloudLabs VM

Double-click the ***Programmer (Quartus Prime Pro 25.1.1)*** icon on the desktop.

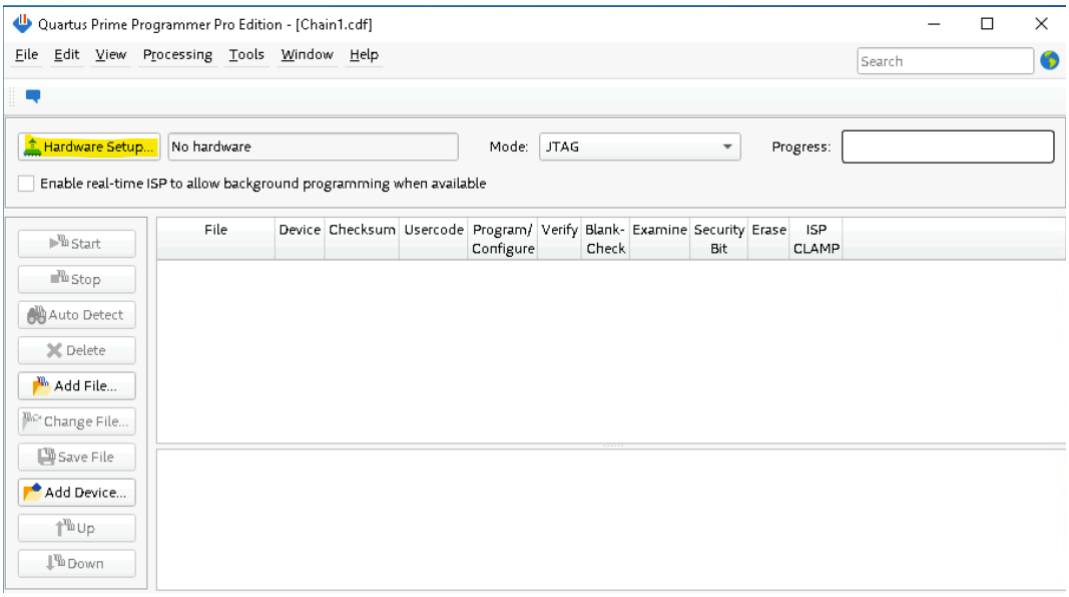


Figure 8-32 : Quartus Programmer

Click the ***Hardware Setup*** button and **double-click** on the ***USB-Blaster III***, then click **Close**.

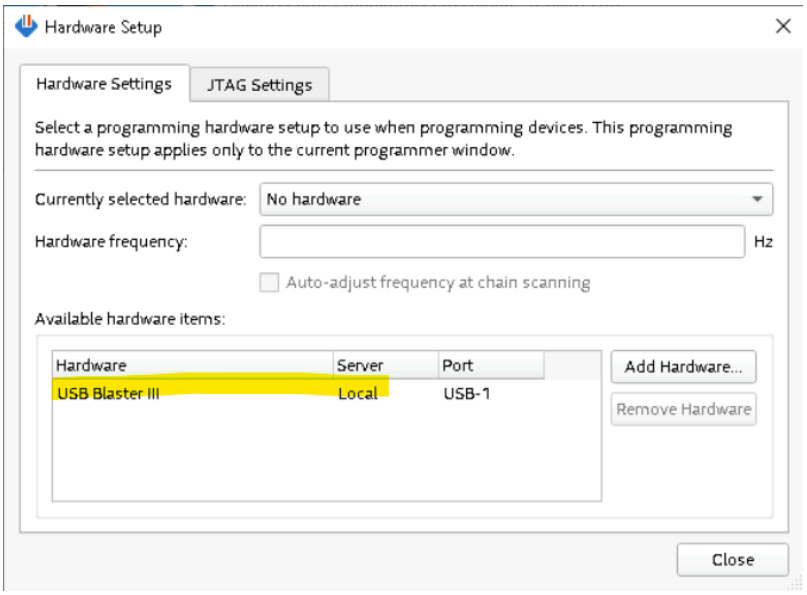
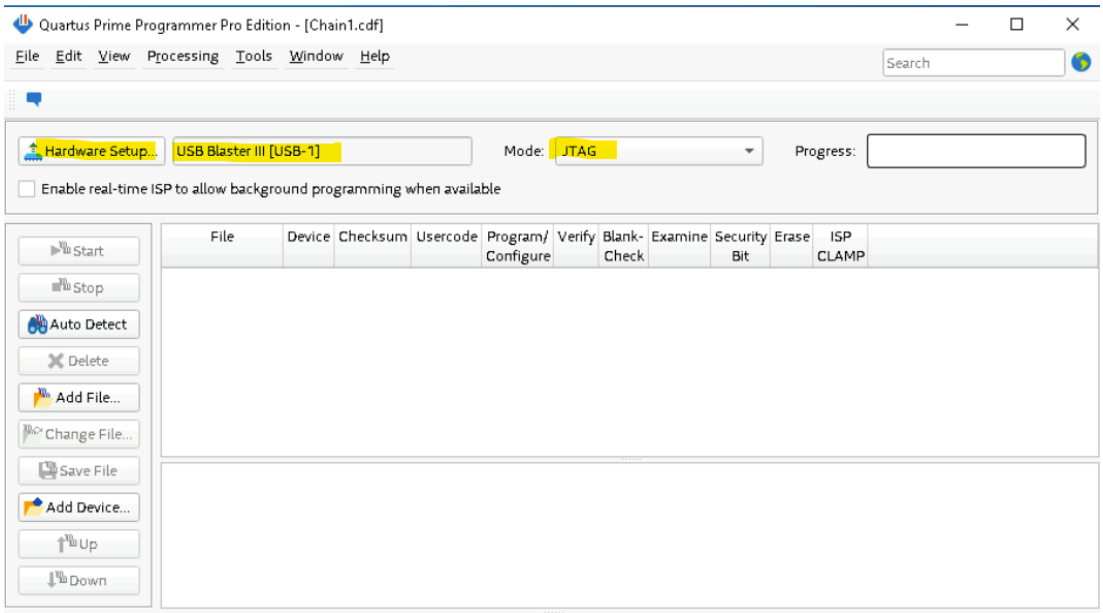


Figure 8-33 : USB Blaster Hardware Setup

Ensure that the ***USB-Blaster III*** is selected, and the mode is set to ***JTAG***.



If you don't see a device showing in the main window, click on **Auto Detect** button to query the JTAG chain and display what is connected to it.

Select the FPGA, to activate the ***Change File…*** button.

Click the ***Change File…*** button and navigate to the downloaded .sof file and **double-click** it.

Check the box for ***Program/Configure***.

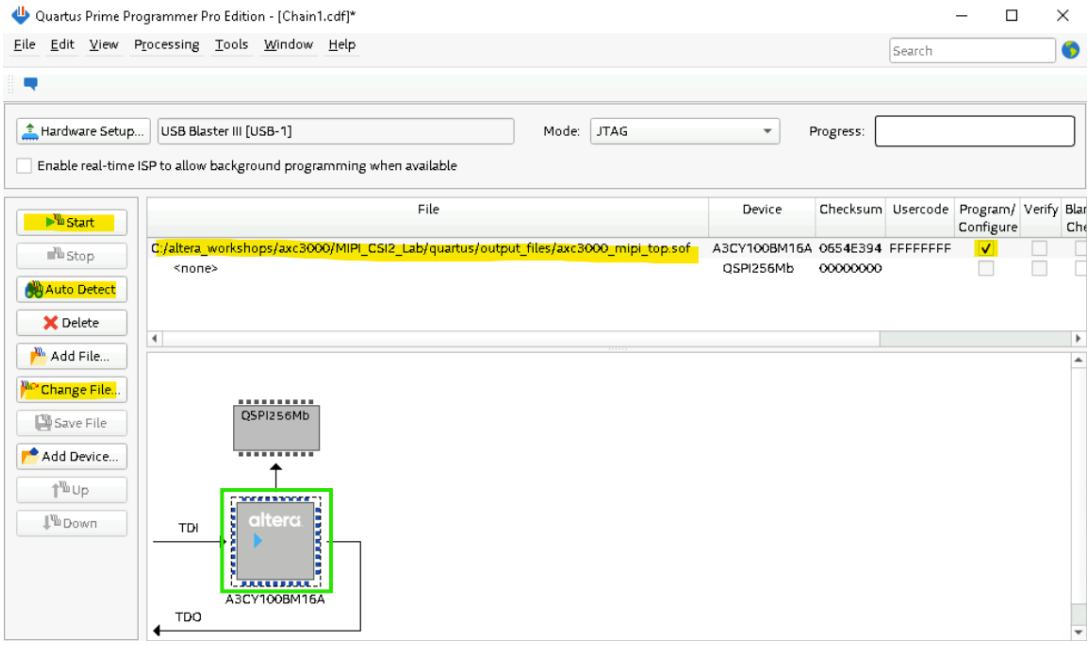


Figure 8-34 : Detect device and assign .sof file

Click the ***Start*** button to begin the programming operation and wait for the progress bar to hit 100%. If the ***Start*** button is not available, make sure the ***Program/Configure*** box for the selected SOF is **checked**.

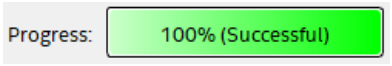


Figure 8-35 : Programmer progress bar

When the NIOS-V program runs successfully and data is captured into the Video Frame Buffer, LED D2 should go from Red to Blue. If it gets stuck at Green, it means that there is a problem, and the frame buffer is not being written to. In the case of a Green D2, unplug the USB cable from the PC, then plug it back in and re-configure the Agilex-3.

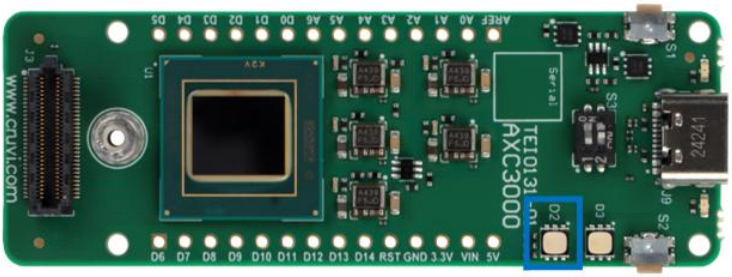


Figure 8-36 : LED D2 indicator

Run script to extract Video Frame Buffer content

In the ***scripts*** subfolder, there are 2 scripts which use ***System Console*** under the hood to extract the video frame buffer and display it on the screen.

Open a Windows ***Command Prompt*** (CMD) window.

Change directory to be at the ***scripts*** sub-folder.

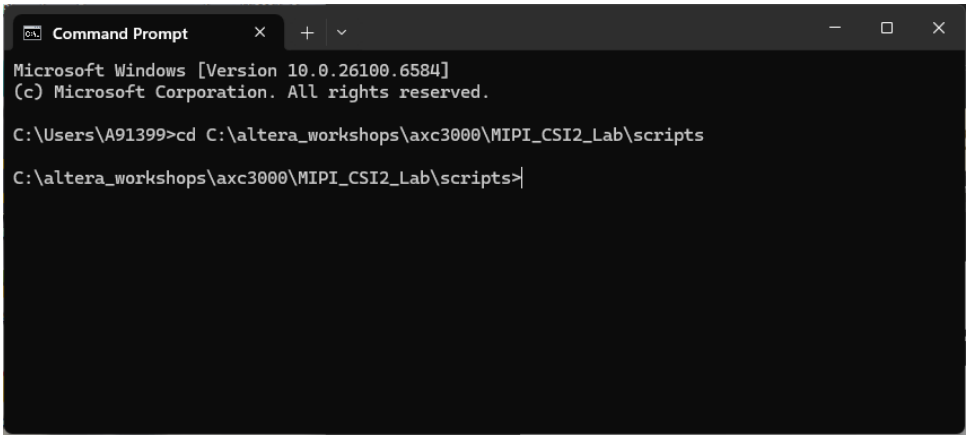


Figure 8-37 : Command Prompt Window

In the ***CMD*** window, type **py visualize_vfb.py**, to start the frame buffer extraction Python script.

You will be prompted to enter the ***USB Serial Port*** which the board is connected to. **Enter** the ***digit*** corresponding to that port and hit ***Enter*** on the keyboard.

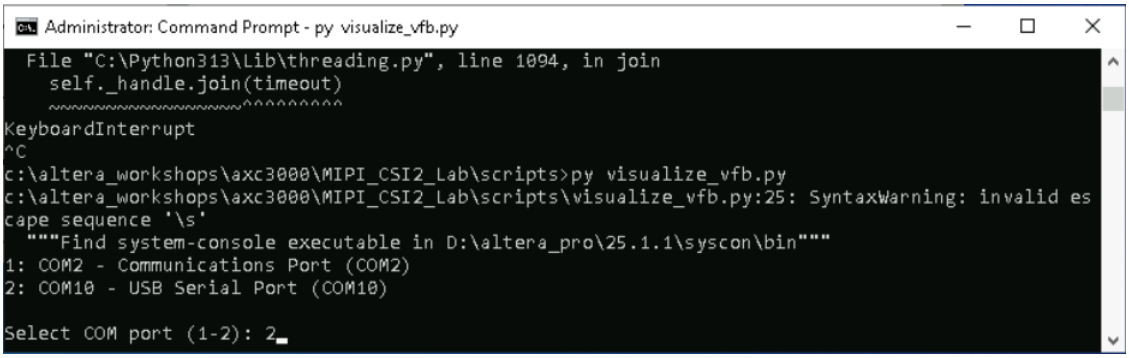


Figure 8-38 : Run the Python scripts and set COM port number

You should see a viewer window come up and start showing the 320x200 (multiplied by 3).

Turn the camera lens to focus the image. There is a few seconds lag for the new image to be re-drawn.

If you need more image exposure, press the S2 button and wait a few seconds to see a new image.

The image will look pixelated because it is blown up 3x.

Figure 8-39: Image viewer

To exit the viewer, hit ***Ctrl-C*** in the CMD window.

9 Legal Disclaimer

ARROW ELECTRONICS

EVALUATION BOARD LICENSE AGREEMENT

By using this evaluation board or kit (together with all related software, firmware, components, and documentation provided by Arrow, “Evaluation Board”), You (“You”) are agreeing to be bound by the terms and conditions of this Evaluation Board License Agreement (“Agreement”). Do not use the Evaluation Board until You have read and agreed to this Agreement. Your use of the Evaluation Board constitutes Your acceptance of this Agreement.

PURPOSE

The purpose of this evaluation board is solely intended for evaluation purposes. Any use of the Board beyond these purposes is at your own risk. Furthermore, according to the applicable law, the offering Arrow entity explicitly does not warrant, guarantee or provide any remedies to you with regard to the board.

LICENSE

Arrow grants You a non-exclusive, limited right to use the enclosed Evaluation Board offering limited features only for Your evaluation and testing purposes in a research and development setting. Usage in a living environment is prohibited. The Evaluation Board shall not be, in any case, directly or indirectly assembled as a part in any production of Yours as it is solely developed to serve evaluation purposes and has no direct function and is not a finished product.

EVALUATION BOARD STATUS

The Evaluation Board offers limited features allowing You only to evaluate and test purposes. The Evaluation Board is not intended for consumer or household use. You are not authorized to use the Evaluation Board in any production system, and it may not be offered for sale or lease, or sold, leased or otherwise distributed for commercial purposes.

OWNERSHIP AND COPYRIGHT

Title to the Evaluation Board remains with Arrow and/or its licensors. This Agreement does not involve any transfer of intellectual property rights (“IPR) for evaluation board. You may not remove any copyright or other proprietary rights notices without prior written authorization from Arrow or its licensors.

RESTRICTIONS AND WARNINGS

Before You handle or use the Evaluation Board, You shall comply with all such warnings and other instructions and employ reasonable safety precautions in using the Evaluation Board. Failure to do so may result in death, personal injury, or property damage.

You shall not use the Evaluation Board in any safety critical or functional safety testing, including but not limited to testing of life supporting, military or nuclear

applications. Arrow expressly disclaims any responsibility for such usage which shall be made at Your sole risk.

WARRANTY

Arrow warrants that it has the right to provide the evaluation board to you. This warranty is provided by Arrow in lieu of all other warranties, written or oral, statutory, express or implied, including any warranty as to merchantability, non-infringement, fitness for any particular purpose, or uninterrupted or error-free operation, all of which are expressly disclaimed. The evaluation board is provided “as is” without any other rights or warranties, directly or indirectly.

You warrant to Arrow that the evaluation board is used only by electronics experts who understand the dangers of handling and using such items, you assume all responsibility and liability for any improper or unsafe handling or use of the evaluation board by you, your employees, affiliates, contractors, and designees.

LIMITATION OF LIABILITIES

In no event shall Arrow be liable to you, whether in contract, tort (including negligence), strict liability, or any other legal theory, for any direct, indirect, special, consequential, incidental, punitive, or exemplary damages with respect to any matters relating to this agreement. In no event shall arrow’s liability arising out of this agreement in the aggregate exceed the amount paid by you under this agreement for the purchase of the evaluation board.

IDENTIFICATION

You shall, at Your expense, defend Arrow and its Affiliates and Licensors against a claim or action brought by a third party for infringement or misappropriation of any patent, copyright, trade secret or other intellectual property right of a third party to the extent resulting from (1) Your combination of the Evaluation Board with any other component, system, software, or firmware, (2) Your modification of the Evaluation Board, or (3) Your use of the Evaluation Board in a manner not permitted under this Agreement. You shall indemnify Arrow and its Affiliates and Licensors against and pay any resulting costs and damages finally awarded against Arrow and its Affiliates and Licensors or agreed to in any settlement, provided that You have sole control of the defense and settlement of the claim or action, and Arrow cooperates in the defense and furnishes all related evidence under its control at Your expense. Arrow will be entitled to participate in the defense of such claim or action and to employ counsel at its own expense.

RECYCLING

The Evaluation Board is not to be disposed of as urban waste. At the end of its life cycle, differentiated waste collection must be followed, as stated in the directive 2002/96/EC. In all the countries belonging to the European Union (EU Dir. 2002/96/EC) and those following differentiated recycling, the Evaluation Board is subject to differentiated recycling at the end of its life cycle, therefore: It is forbidden

to dispose the Evaluation Board as an undifferentiated waste or with other domestic wastes. Consult the local authorities for more information on the proper disposal channels. An incorrect Evaluation Board disposal may cause damage to the environment and is punishable by the law.